

DESIGN Output - Stage 1

Task: Generated by LLM
Agent: design
Stage: 1
Timestamp: 2026-09-27T09:40:53.917593
Model: deepseek-v4.1-flash
Provider: opencode-go

Token Usage

- **Input Tokens:** 143744
- **Output Tokens:** 154221
- **Cached Tokens:** 68608
- **Reasoning Tokens:** 58149
- **Total Tokens:** 297965
- **Cost:** \$0.114094
- **Cost per 1K input:** \$0.00015
- **Cost per 1K output:** \$0.0006

Output

F-1: add command entry point

Feature ID: F-1
Feature name: `add` command entry point
Summary: The single feature of the product. A stateless, dependency-free Python 3 CLI that accepts exactly two integer operands, prints their sum to stdout, and exits `0`. All failures are diagnosed on stderr with a non-zero exit status.
Priority (feature-level): must-have

Requirements

Functional requirements (FR ids)

ID	Requirement	Priority
FR-1	The program MUST be invocable as `add <int1> <int2>` from a POSIX-style shell, accepting exactly two positional operands.	must-have
FR-2	The program MUST parse each operand as an integer using a strict, explicitly-defined decimal integer grammar (see Business rules `BR-1`–`BR-4`).	must-have
FR-3	The program MUST compute the exact arithmetic sum `int1 + int2` using arbitrary-precision integer arithmetic.	must-have
FR-4	On success the program MUST write the result as a base-10 integer string followed by exactly one newline (U+000A) to stdout , and write nothing to stderr.	must-have
FR-5	On success the program MUST terminate with exit status `0`.	must-have
FR-6	If argument arity is not exactly two operands, the program MUST write a usage/diagnostic message to stderr , write nothing to stdout, and terminate with exit status `2`.	must-have

| FR-7 | If any operand fails the integer grammar (`BR-1`), the program MUST write a single-line diagnostic to `**stderr**` naming the offending argument, write nothing to stdout, and terminate with exit status `2`. | must-have |

| FR-8 | The program MUST print the usage line to stderr on arity error, in the form `usage: add <int1> <int2>`, followed by the result-free newline-terminated diagnostic. | must-have |

| FR-9 | The program MUST support `-h` / `--help`: print the usage line to `**stdout**` and exit `0`. The help text MUST share the same usage string used by `FR-8`. | should-have |

| FR-10 | The program MUST be a pure function of its argv: no reads or writes to files, stdin, environment variables, network, or global mutable state; the only observable side effect is the stdout/stderr byte stream and the exit status. | must-have |

| FR-11 | The program MUST consume input `**only**` from argv. It MUST NOT read stdin (even when stdin is a pipe or TTY), MUST NOT accept operands from environment variables, and MUST NOT read a config file. | must-have |

| FR-12 | The program MUST rely on the Python 3 standard library only and MUST NOT declare any third-party runtime dependency. | must-have |

| FR-13 | The program MUST NOT evaluate, compile, or `eval()`/`exec()` any user-supplied text, and MUST NOT invoke a shell. | must-have |

| FR-14 | The output for a given `(int1, int2)` pair MUST be byte-identical across runs, processes, and machines, independent of locale and current working directory. | must-have |

| FR-15 | The program MUST NOT emit any output on stdout when it exits with a non-zero status, so that `$(add 2 3)` never captures partial or diagnostic text. | must-have |

****Non-functional requirements (NFR ids)****

ID	Requirement	Target / measurement
NFR-1	Latency: end-to-end wall time for <code>add 2 3</code> on a reference machine, including interpreter start-up.	p50 < 100 ms, p95 < 250 ms; measured with <code>time</code> over 100 runs from a warm cache.
NFR-2	Throughput/statelessness: no state is retained between invocations; N concurrent invocations produce identical, independent results.	100 parallel invocations yield 100 identical correct outputs and 100 zero exit codes.
NFR-3	Operand scalability: correctness is preserved for integers far beyond 64-bit range.	<code>add</code> returns the exact sum for operands of $\geq 1,000$ decimal digits (subject to <code>EC-6</code>).
NFR-4	Determinism/reliability: identical argv always yields identical stdout bytes and exit status.	Byte-for-byte comparison across 1,000 repeated runs.
NFR-5	Security: no file, network, subprocess, or dynamic-code-execution capability is exercised at runtime.	Static review of the single entry-point module; no imports of <code>os.system</code> , <code>subprocess</code> , <code>socket</code> , <code>eval</code> , <code>exec</code> , or dynamic <code>import</code> .
NFR-6	Input-safety: malicious or oversized operands cannot cause code execution, crashes, or unbounded resource use.	Fuzzing with generated non-numeric, control-character, and 1 MB-length argv strings yields exit <code>2</code> or <code>EC-6</code> behaviour, never a traceback or hang.
NFR-7	Deployment/environment: single-file, zero-configuration executable; runs in any environment with Python 3 available.	Runs from an empty directory with no env vars set other than <code>PATH</code> .
NFR-8	Portability: identical behaviour on CPython 3.8+ across Linux, macOS, and Windows (cmd/PowerShell) including exit status and newline handling.	Same acceptance suite passes on all three platforms.
NFR-9	Availability model: a short-lived, single-process, foreground command with no daemon, service, or listening port; there is no uptime target beyond "runs when invoked".	No process outlives the command; no sockets opened.
NFR-10	Data/residency: no user data is persisted, logged, cached, or transmitted; operands and results exist only in process memory for the lifetime of the invocation.	No file or socket writes observed under filesystem/network tracing.
NFR-11	Locale independence: output digits and separators must not change under any <code>LC_ALL</code> / <code>LANG</code> value.	<code>LC_ALL=de_DE.UTF-8 add 1000 1</code> outputs <code>1001</code> (never <code>1.001</code>).

| NFR-12 | Observability: the only diagnostic channel is stderr; diagnostics are single-line and machine-greppable. | Diagnostics match a stable prefix; no stack traces leak on the error paths defined in `EC-1`-`EC-6`. |

| NFR-13 | Solo robustness: unexpected internal failure must not surface a Python traceback to the user. | Any unhandled internal exception is converted to a stderr diagnostic with exit status `2` (see `EC-7`). |

| NFR-14 | Terminal-safety: output MUST NOT contain ANSI escapes or other control characters other than the single trailing newline. | Output byte inspection. |

| NFR-15 | Maintainability: the entire feature is implementable in a single module of ≤ 100 source lines with no conditional platform branches in the core arithmetic path. | Source line count and review. |

****User stories (US ids)****

| ID | Story |

|---|---|

| US-1 | As a developer at a shell prompt, I want to type `add 2 3` and see `5`, so that I can compute a value without leaving the terminal. |

| US-2 | As a script author, I want the result on stdout with nothing else, so that I can capture it with `\${add 2 3}` or a pipe. |

| US-3 | As a script author, I want a deterministic exit status, so that I can branch on success (`0`) versus input error (`2`) in shell conditionals. |

| US-4 | As a user who mistyped a command, I want a clear one-line error and a usage line, so that I can correct the invocation immediately. |

| US-5 | As a user with large numbers, I want arbitrary-precision results, so that I am not limited by 32-/64-bit overflow. |

| US-6 | As a new user, I want `add --help` to show the usage line, so that I can learn the command without reading source. |

| US-7 | As a security-conscious operator, I want the tool to touch no files, no network, and no shell, so that I can run it safely in constrained environments. |

| US-8 | As an automation author, I want the command to be stateless and side-effect-free, so that parallel invocations never interfere. |

| US-9 | As an international user, I want digits formatted the same regardless of my locale, so that downstream parsers never break. |

| US-10 | As a maintainer, I want a stdlib-only single-file tool, so that there is nothing to install or vendor. |

Behaviour

1. The process is invoked with `argv` of the form `["add", <operand1>, <operand2>]`.
2. `-h` / `--help` appearing anywhere in argv is intercepted first (FR-9): the usage line is written to stdout followed by `\n`, and the process exits `0`.
3. Otherwise the argument count is checked (FR-1, FR-6). Exactly two operands are required; zero, one, or three-or-more operands is an arity error.
4. Each operand is validated against the integer grammar (`BR-1`) in left-to-right order (FR-2). The first offending operand is reported (FR-7).
5. The sum is computed with Python's arbitrary-precision `int` addition (FR-3).
6. The result is rendered as a base-10 string with a leading `-` only when negative, then written to stdout with exactly one trailing newline (FR-4), and the process exits `0` (FR-5).
7. No other output, prompt, spinner, colour, or timing information is produced on any path (FR-10, NFR-14).

8. The process is fully synchronous: it reads argv, computes, writes, and exits. There is no interaction loop, no stdin polling, and no retry (FR-11).

Business rules

ID	Rule
BR-1	The accepted integer grammar is: ``-?[0-9]+`` — an optional single ASCII hyphen-minus ``U+002D``, followed by one or more ASCII digits ``0`–`9`` (``U+0030`–`U+0039``). Nothing else is accepted as an operand.
BR-2	A leading ``+`` is rejected. Numeric separators (``_``, `` ``, spaces, thin spaces, apostrophes) are rejected. Leading zeros are accepted and normalised in the result (``007`` → parsed as ``7``).
BR-3	Only ASCII digits are accepted. Non-ASCII decimal digits (e.g. Arabic-Indic ``٠``, fullwidth ``۰``), superscripts, Roman numerals, and spelled-out numbers are rejected with exit ``2``.
BR-4	Floating-point, scientific, hexadecimal (``0x10``), octal (``0o17``), binary (``0b101``), and ``inf`/`nan`` style literals are rejected with exit ``2``.
BR-5	The operation is mathematically exact integer addition. There is no rounding, saturation, wrap-around, modulus, or floating-point path.
BR-6	``-0`` is a valid operand and is equal to ``0``; the result of any sum equal to zero is rendered as ``0``, never ``-0`` or ``+0``.
BR-7	Addition is commutative and associative for the two-operand case; ``add a b`` and ``add b a`` produce identical stdout bytes.
BR-8	Exit statuses are fixed: ``0`` = success, ``2`` = usage/operand error. No other status is produced on a designed path.
BR-9	Exit status ``2`` is the only failure status; the tool never returns ``1`` for input problems, so scripts can distinguish "bad input" from "unexpected internal failure".
BR-10	Operands are consumed positionally by index. There is no flag parsing other than ``-h`/`--help``; flags such as ``--base``, ``--precision``, or ``-v`` are **not** part of this feature and **MUST** be treated as invalid operands, not as options.
BR-11	An operand that begins with ``-`` and consists solely of digits is a negative number, not an option. The only reserved tokens are the exact strings ``-h`` and ``--help``.
BR-12	The command is stateless: no history, no memoisation, no temp files, no environment mutation.

Validation

Validation occurs in this strict order; the first failing check determines the diagnostic.

Step	Check	On failure
V-1	``-h`` or ``--help`` present in argv	Print usage to stdout, exit ``0`` (FR-9)
V-2	``len(argv) - 1 == 2``	Arity error ``EC-1`/`EC-2`/`EC-3``, exit ``2``
V-3	Operand 1 matches ``BR-1``	Diagnostic ``EC-4``, exit ``2``
V-4	Operand 2 matches ``BR-1``	Diagnostic ``EC-4``, exit ``2``
V-5	Both operands parse under ``BR-2`–`BR-4`` without accepted-but-unintended forms	Diagnostic ``EC-4``, exit ``2``
V-6	Result is convertible to a decimal string within the interpreter's configured limit	Diagnostic ``EC-6``, exit ``2``

Validation MUST be performed before any arithmetic, and no partial result may be written to stdout on a failed validation (FR-15).

Edge cases

ID	Case	Required behaviour
EC-1	No operands: <code>`add`</code>	Usage + diagnostic → stderr, exit <code>`2`</code> .
EC-2	One operand: <code>`add 2`</code>	Usage + diagnostic → stderr, exit <code>`2`</code> .
EC-3	Three or more operands: <code>`add 2 3 4`</code>	Usage + diagnostic → stderr, exit <code>`2`</code> . No implicit folding of extra operands.
EC-4	Non-integer operand: <code>`add 2 x`</code> , <code>`add 2 3.0`</code> , <code>`add 2 1e3`</code> , <code>`add 2 0x10`</code> , <code>`add 2 +3`</code> , <code>`add 2 1,000`</code> , <code>`add 2 1_000`</code> , <code>`add 2 █`</code>	Diagnostic naming the offending argument (" <code>x</code> ") → stderr, exit <code>`2`</code> .
EC-5	Empty-string operand: <code>`add "" 3`</code>	Treated as invalid (zero digits, fails <code>`BR-1`</code>), exit <code>`2`</code> .
EC-6	Very large operands whose decimal rendering exceeds the interpreter's int→str digit limit (CPython 3.11+ default 4300 digits)	Diagnostic → stderr, exit <code>`2`</code> . This is an explicit, documented operands-scaling limit, not a silent truncation. See <code>`OQ-1`</code> .
EC-7	Unexpected internal exception on any code path	Converted to a single-line stderr diagnostic, exit <code>`2`</code> , never a traceback (<code>`NFR-13`</code>).
EC-8	Zero results reached by negative operands: <code>`add -5 5`</code> , <code>`add 0 -0`</code> , <code>`add -0 -0`</code>	stdout <code>`0\n`</code> , exit <code>`0`</code> .
EC-9	Negative results: <code>`add 2 -5`</code>	stdout <code>`-3\n`</code> , exit <code>`0`</code> .
EC-10	Both operands negative: <code>`add -2 -3`</code>	stdout <code>`-5\n`</code> , exit <code>`0`</code> .
EC-11	Leading zeros: <code>`add 007 0003`</code>	stdout <code>`10\n`</code> , exit <code>`0`</code> .
EC-12	Extremely large positive/negative operands within <code>`EC-6`</code> limits	Exact sum, exit <code>`0`</code> (<code>`NFR-3`</code>).
EC-13	Leading/trailing whitespace in an operand: <code>`add " 2" 3`</code> , <code>`add "2 " 3`</code>	Rejected, exit <code>`2`</code> (whitespace is not part of <code>`BR-1`</code>); the tool never trims argv silently.
EC-14	<code>`add -h 3`</code> / <code>`add 3 --help`</code>	Help wins (<code>`V-1`</code>), usage to stdout, exit <code>`0`</code> . Documented precedence, not an error.
EC-15	argv[0] differs (renamed binary, invoked via symlink or <code>`python -m add`</code>)	Behaviour is unchanged; usage text still names <code>`add`</code> .
EC-16	stdin is a pipe/TTY with buffered data, or closed entirely	Never read; behaviour identical (<code>`FR-11`</code>).
EC-17	Non-UTF-8 argv bytes on a byte-oriented locale	No decode exception surfaces; operand is treated as invalid → exit <code>`2`</code> with diagnostic.
EC-18	Output stream closed early (<code>`add 2 3   head -c0`</code>)	Broken-pipe condition must not produce a Python traceback (<code>`NFR-13`</code>); process terminates without stdout corruption.

Error handling

All error paths share one contract: **diagnostic** → stderr, stdout empty, exit ``2``, single line, no traceback.

ID	Condition	stderr content	Exit
EH-1	*(folded into <code>`EC-1`</code> / <code>`EC-2`</code> / <code>`EC-3`</code> messaging)*	Wrong operand count <code>`usage: add <int1> <int2>`</code> followed by <code>`add: expected exactly 2 integer operands, got <n>`</code>	<code>`2`</code>

API-2	`argv`	`[ "add", "-h" ]` or `[ "add", "--help" ]` → help path, exit `0`.
API-3	`argv`	Any other length or any non-conforming operand → error path, exit `2`.
API-4	stdout	On success: the exact decimal sum as ASCII/UTF-8 bytes, terminated by one `\n`. On error: zero bytes.
API-5	stderr	On success: zero bytes. On error: usage line plus one diagnostic line (each `\n`-terminated).
API-6	Exit status	`0` success or help; `2` for arity error, operand error, operand-too-large, and internal error.
API-7	stdin	Never read, never blocked on, never closed by the program.
API-8	Filesystem / network / environment	No reads, no writes, no sockets; no environment variable changes behaviour.
API-9	Concurrency	Reentrant by construction; no locks, temp files, or shared state.
API-10	Versioning	The CLI contract is versioned by the tool itself; any change to stdout bytes, exit codes, or diagnostic text is a breaking change for `API-4`/`API-5`/`API-6` consumers.

Data model (informational)

ID	Concept	Definition
DM-1	Operand	A validated decimal integer literal conforming to `BR-1`, mapped to a Python arbitrary-precision `int`.
DM-2	Result	The exact mathematical sum of `DM-1` and `DM-1`; rendered as `-[0-9]+` with no leading zeros except the single value `0`.
DM-3	Exit status	Enumerated set `{0, 2}` only, per `BR-8`.
DM-4	Diagnostic	Immutable single-line string written to stderr on failure; part of the observable contract (`NFR-12`).

Test obligations

ID	Test	Covers
T-1	Positive, negative, zero, mixed-sign, and leading-zero addition cases.	AC-1–AC-6
T-2	Arity matrix: 0, 1, 3, 4 operands.	AC-8, AC-9
T-3	Malformed-operand matrix: `x`, `3.0`, `1e3`, `0x10`, `+3`, `1,000`, `1\_000`, `█`, `""`, `" 2"`, `"2 "`.	AC-7
T-4	Help precedence: `-h`, `--help`, `add -h 3`, `add 3 --help`.	AC-10, EC-14
T-5	Large-integer cases at and beyond the int→str digit limit.	AC-11, EC-6
T-6	Locale matrix over `LC\_ALL`/`LANG`.	AC-12
T-7	Determinism and parallel-invocation harness.	AC-13, AC-14
T-8	Padding and sanitisation checks on stdout/stderr bytes.	AC-15
T-9	Resource-access audit (files/sockets) under a sandbox.	AC-16
T-10	Cross-platform acceptance run (Linux/macOS/Windows).	AC-20

Open questions

| ID | Question | Blocks |

---|---|---

| OQ-1 | For operands whose decimal rendering exceeds CPython 3.11+'s default 4,300-digit int→str limit: is raising the limit (`sys.set_int_max_str_digits`) the desired behaviour, or is the documented "operand too large" error (`EC-6`) the correct contract? | `EC-6`, `NFR-3`, `AC-11` |

| OQ-2 | Should `--version` (or any version string) be added, or is the brief's "one command" scope limited to `-h/--help` (`FR-9`) only? Adding it requires scope approval. | `FR-9`, `BR-10` |

| OQ-3 | Are the exact diagnostic message texts and the choice of exit code `2` (versus `1`) acceptable as a frozen contract, given they are consumed by scripts (`API-5`, `API-6`)? | `BR-8`, `BR-9`, `EH-1`–`EH-4` |

Priority: must-have (the sole feature of the product; the tool has no purpose without F-1).

F-2: Two positional integer operands

Feature ID: F-2

Requirements

Functional Requirements (FR)

| ID | Requirement | Priority |

----|-----|-----|

| FR-26 | The program MUST treat the first command-line argument after the program name as `int1` and the second argument as `int2`, with no option parsing or flag recognition. | must-have |

| FR-27 | Each operand MUST be validated against the canonical decimal integer grammar: an optional single sign character (`+` `U+002B` or `-` `U+002D`) followed by one or more ASCII decimal digits (`0`–`9`), and nothing else. | must-have |

| FR-28 | The grammar MUST accept leading zeros (e.g., `007`, `-000`, `+00042`) and interpret them as their base-10 integer value. | must-have |

| FR-29 | The grammar MUST accept an explicit `+` sign (e.g., `+5`, `+0`, `+007`). | must-have |

| FR-30 | The grammar MUST accept a single leading `-` sign for negative numbers (e.g., `-5`, `-0`, `-007`). | must-have |

| FR-31 | The grammar MUST reject any operand containing whitespace, underscores, decimal points, commas, exponent notation (`e`/`E`), base prefixes (`0x`, `0o`, `0b`), or any character outside `[0-9+-]` (with sign permitted only at position 0). | must-have |

| FR-32 | The grammar MUST reject a lone sign (`+` or `-`), an empty operand, and multiple signs (e.g., `--5`, `++5`, `+-5`, `-+5`). | must-have |

| FR-33 | The program MUST convert each valid operand to an arbitrary-precision integer value without loss of precision, using only integer arithmetic (no floating-point conversion). | must-have |

| FR-34 | If either operand fails validation, the program MUST follow the error path defined in F-1 `FR-7`: write a single-line diagnostic to `stderr` naming the offending operand, write nothing to `stdout`, and exit with status `2`. | must-have |

| FR-35 | The program MUST NOT impose any artificial limit on the number of digits in an operand; the only practical limit is available process memory. | must-have |

| FR-36 | The program MUST treat `--` as an ordinary operand token, not as an end-of-options marker; since `--` does not match the integer grammar, it MUST be rejected as invalid. | must-have |

Non-Functional Requirements (NFR)

| ID | Target | Measurement |

----|-----|-----|

| NFR-16 | Validation of a single operand MUST complete in $O(n)$ time, where n is the operand length. | Time complexity analysis; linear-time scan of characters. |

| NFR-17 | The program MUST handle operands of at least 10,000 digits without stack overflow, excessive memory use (≤ 10 MB extra heap), or recursion. | Benchmark with a 10,000-digit operand; measure peak RSS. |

| NFR-18 | All parsing and arithmetic MUST be exact integer operations; no floating-point types or operations may be used for parsing or summing. | Code review / static analysis for `float` usage. |

| NFR-19 | The program MUST be deterministic: identical argv inputs always produce identical stdout, stderr, and exit status. | Repeated runs with same argv; compare outputs. |

| NFR-20 | The program MUST NOT require network access, filesystem writes (other than normal process I/O), or any external service. | Run in a sandbox with no network and read-only filesystem; verify success. |

| NFR-21 | The program MUST have no third-party runtime dependencies; only the Python 3 standard library may be used. | Inspect imports; verify no `pip`-installed packages required. |

| NFR-22 | The program MUST NOT evaluate or execute any part of an operand as code (e.g., no `eval`, `exec`, `pickle`, or shell invocation). | Static analysis for dangerous functions. |

| NFR-23 | The program MUST treat operands as data only; no injection vector is possible via the integer grammar. | Security review of parsing path. |

| NFR-24 | Operands and results MUST remain local to the process; no data is transmitted over a network. | Confirm no network APIs are called. |

| NFR-25 | The program MUST run as a single command-line executable without requiring a persistent daemon, configuration file, or database. | Run in a minimal environment with no config files present. |

User Stories (US)

| ID | Story |

|----|-----|

| US-16 | As a shell user, I want to pass two integer operands as positional arguments, so that I can get their sum without specifying any flags or options. |

| US-17 | As a shell user, I want to use negative numbers (e.g., `add -5 3`) without the program treating `-5` as an option, so that I can add negative integers directly. |

| US-18 | As a shell user, I want to use a leading `+` sign (e.g., `add +5 3`), so that I can be explicit about positive numbers. |

| US-19 | As a shell user, I want leading zeros to be accepted (e.g., `add 007 3`), so that I can paste numbers without stripping them first. |

| US-20 | As a shell user, I want clear error messages when I provide a non-integer operand (e.g., `add 1.5 2`), so that I can correct my input. |

| US-21 | As a shell user, I want the program to handle very large integers (e.g., 1000 digits), so that I can use it for arbitrary-precision arithmetic. |

| US-22 | As a shell user, I want the program to reject operands with whitespace or other non-numeric characters, so that I don't get silently wrong results. |

Behaviour

1. The program receives exactly two positional operands: `int1` and `int2`.
2. Each operand is independently validated against the canonical decimal integer grammar (BR-1).
3. If both operands are valid, they are converted to arbitrary-precision integers and summed (delegated to F-1 `FR-3`).
4. If either operand is invalid, no arithmetic is performed; the program writes a diagnostic to stderr, writes nothing to stdout, and exits with status `2` (F-1 `FR-7`).
5. The program does not recognize any option flags; every argument is a positional operand.
6. The program does not read from stdin; all input comes from argv.

7. The program does not use environment variables or configuration files to alter operand parsing.

Business rules

ID	Rule
BR-1	The integer grammar is <code>^[+-]?[0-9]+\$</code> (ASCII characters only).
BR-2	The sign is optional; if present it must be exactly one character: <code>`+`</code> or <code>`-`</code> .
BR-3	At least one digit (<code>`0`–`9`</code>) must follow the optional sign.
BR-4	Leading zeros are permitted and do not change the numeric value (e.g., <code>`007`</code> = 7).
BR-5	A <code>`-`</code> sign before zero yields zero (e.g., <code>`-0`</code> = 0).
BR-6	The program does not implement option flags; all arguments are positional operands.
BR-7	The token <code>`--`</code> is not a separator; it is an invalid operand.
BR-8	Operands are parsed as base-10 integers only; no other bases are recognized.
BR-9	Only ASCII digits <code>`0`–`9`</code> are accepted; Unicode digits or full-width digits are rejected.
BR-10	No whitespace is allowed within an operand.

Validation

ID	Validation step
V-1	Check that the argument is not an empty string.
V-2	Check that the argument matches the regex <code>^[+-]?[0-9]+\$</code> .
V-3	If the argument starts with <code>`+`</code> or <code>`-`</code> , ensure there is at least one digit following.
V-4	Ensure that the argument contains no characters outside <code>`[0-9+-]`</code> (sign only at position 0).
V-5	Ensure that the argument contains no whitespace (spaces, tabs, newlines, etc.).
V-6	Ensure that the argument contains no underscores, dots, commas, or letters.
V-7	Ensure that the argument does not use scientific notation (e.g., <code>`1e3`</code>).
V-8	Ensure that the argument does not use base prefixes (<code>`0x`</code> , <code>`0o`</code> , <code>`0b`</code>).
V-9	Ensure that the argument is not just a sign (<code>`+`</code> or <code>`-`</code>).
V-10	If validation fails, produce the diagnostic specified in EH-1..EH-5.

Edge cases

ID	Case	Expected result
EC-1	<code>`+0`</code>	valid, value 0
EC-2	<code>`-0`</code>	valid, value 0
EC-3	<code>`007`</code>	valid, value 7
EC-4	<code>`-007`</code>	valid, value -7
EC-5	<code>`+007`</code>	valid, value 7
EC-6	<code>`0`</code>	valid, value 0
EC-7	<code>`-1`</code>	valid, value -1
EC-8	<code>`+1`</code>	valid, value 1
EC-9	<code>``</code> (empty string)	invalid
EC-10	<code>`+`</code>	invalid
EC-11	<code>`-`</code>	invalid
EC-12	<code>`--1`</code>	invalid
EC-13	<code>`++1`</code>	invalid
EC-14	<code>`+-1`</code>	invalid
EC-15	<code>`-+1`</code>	invalid
EC-16	<code>`1.0`</code>	invalid

Error handling

Acceptance criteria

API behaviour

ID	Behaviour
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
10	10
11	11
12	12
13	13
14	14
15	15
16	16
17	17
18	18
19	19
20	20
21	21
22	22
23	23
24	24
25	25
26	26
27	27
28	28
29	29
30	30
31	31
32	32
33	33
34	34
35	35
36	36
37	37
38	38
39	39
40	40
41	41
42	42
43	43
44	44
45	45
46	46
47	47
48	48
49	49
50	50
51	51
52	52
53	53
54	54
55	55
56	56
57	57
58	58
59	59
60	60
61	61
62	62
63	63
64	64
65	65
66	66
67	67
68	68
69	69
70	70
71	71
72	72
73	73
74	74
75	75
76	76
77	77
78	78
79	79
80	80
81	81
82	82
83	83
84	84
85	85
86	86
87	87
88	88
89	89
90	90
91	91
92	92
93	93
94	94
95	95
96	96
97	97
98	98
99	99
100	100

|---|-----|

| API-1 | Command-line interface: ``add <int1> <int2>``. There are no options, flags, or environment variables. |

| API-2 | The program reads only from ``argv``; it does not read stdin. |

| API-3 | On success, the program writes only the result to stdout; on failure, it writes only diagnostics to stderr. |

| API-4 | Exit status is ``0`` on success and ``2`` on invalid operand (and per F-1, ``2`` on arity error). |

| API-5 | The program does not implement ``--help`` or ``--version``; invoking with such tokens results in either an arity error or an invalid-operand error. |

| API-6 | The program does not use any configuration files or environment variables to alter operand parsing. |

Priority

****must-have**** — both positional integer operands are fundamental to the core function of the ``add`` utility.

F-3: Integer addition

****Feature ID:**** F-3

****Depends on:**** F-1 (``add`` command entry point), F-2 (two positional integer operands) — F-3 consumes operands that have already been accepted by the F-2 grammar; F-3 does not re-define argument handling, arity rules, or the digit grammar.

Requirements

Functional Requirements (FR)

| id | Requirement | Priority |

|---|---|---

| FR-51 | The program **MUST** compute ``int1 + int2`` as the exact mathematical sum for every pair of operands accepted under the F-2 operand grammar. | must-have |

| FR-52 | The computation **MUST** use arbitrary-precision integer arithmetic; the result **MUST NOT** overflow, truncate, round, saturate, or wrap at any magnitude or sign. | must-have |

| FR-53 | The parse $\rightarrow$ add $\rightarrow$ render path **MUST NOT** use binary floating-point or decimal floating-point values at any point. | must-have |

| FR-54 | Sign rules: same-sign operands **MUST** produce a result with that sign and magnitude equal to the sum of the magnitudes; opposite-sign operands **MUST** produce a result whose magnitude is the difference of the magnitudes and whose sign is the sign of the larger-magnitude operand; if the magnitudes are equal the result **MUST** be ``0``. | must-have |

| FR-55 | ``0`` **MUST** be the additive identity: for every accepted ``x``, ``x + 0 = 0 + x = x`` (including ``+0`` and ``-0`` as operand spellings). | must-have |

| FR-56 | Negative zero **MUST NOT** be emitted: whenever the computed sum is zero, the rendered result **MUST** be exactly ``0``. | must-have |

| FR-57 | Canonical rendering: the sum **MUST** be rendered as an optional single leading ``-`` (U+002D, only when the value is negative) followed by one or more ASCII decimal digits ``0`–`9``, with no leading zeros except the single digit ``0``, no ``+`` prefix, no whitespace, no thousands separator, no underscore, and no exponent notation. | must-have |

| FR-58 | The rendered sum **MUST** be written to ****stdout**** followed by exactly one newline (``\n``, U+000A); nothing else (no banner, prompt, label, padding, or ``\r``) **MAY** be written to stdout. | must-have |

| FR-59 | On a successful addition the program MUST write nothing to **stderr** and MUST terminate with exit status `0`. | must-have |

| FR-60 | Addition MUST be attempted only after both operands pass F-2 validation. If either operand is invalid, or if arity is not exactly two, the program MUST NOT compute a sum and MUST take the F-1 error path (one-line stderr diagnostic, empty stdout, exit status `2`). | must-have |

| FR-61 | Addition MUST be a pure function of its two operands: environment variables, locale, config files, clock, filesystem, network, stdin contents, and process state MUST NOT influence the result. | must-have |

| FR-62 | Addition MUST be commutative in effect: for any accepted `a` and `b`, `add a b` and `add b a` MUST produce byte-identical stdout, stderr, and exit status. | should-have |

| FR-63 | The printed result MUST be re-parseable under the F-2 grammar and MUST denote exactly the mathematical sum of the two operands (round-trip equality). | must-have |

Non-Functional Requirements (NFR)

| id | Requirement | Priority |

---|---|---

| NFR-31 | Performance: for two operands of up to 1,000 decimal digits each, the parse → add → render cycle MUST complete in under 100 ms wall clock on a 2-core 2 GHz reference machine, excluding interpreter start-up. | should-have |

| NFR-32 | Scalability: operands of at least 10,000 decimal digits MUST be handled without algorithmic failure, stack overflow, or non-zero exit; runtime SHOULD scale no worse than linearly in operand length. | should-have |

| NFR-33 | Availability: addition MUST require no network, no daemon, no service, and no auxiliary process; it MUST succeed on a host with no network interface and no writable filesystem. | must-have |

| NFR-34 | Security: the addition path MUST NOT use `eval`, `exec`, `compile`, `ast.literal_eval`, shell interpolation, or any dynamic evaluation of operand text; operands MUST reach integers only through a strict parser. | must-have |

| NFR-35 | Resource bound: peak additional memory for the addition step SHOULD be $O(n)$ in combined operand length; no buffer MAY be allocated whose size is independent of and larger than the input. | should-have |

| NFR-36 | Data residency: the program MUST NOT read, write, persist, cache, log, or transmit operand or result data; values exist only in process memory and on stdout. | must-have |

| NFR-37 | Deployment: the implementation MUST run on CPython 3 using only the standard library — no third-party packages, no virtualenv, no filesystem writes, no installed data files. | must-have |

| NFR-38 | Determinism: identical arguments on the same interpreter version MUST yield byte-identical stdout, empty stderr, and identical exit status on every invocation. | must-have |

| NFR-39 | Locale independence: parsing and rendering MUST NOT depend on `LANG`, `LC_ALL`, `LC_NUMERIC`, or any locale/encoding setting; ASCII digits MUST be emitted regardless of environment. | must-have |

| NFR-40 | Portability: results MUST be identical across POSIX shells on Linux and macOS, and MUST NOT depend on host endianness, word size, or CPU architecture. | must-have |

| NFR-41 | Interpretability: success vs. failure MUST be distinguishable by exit status alone (`0` vs `2`), and no failure path MAY leave a partial or plausible-looking number on stdout. | must-have |

| NFR-42 | Testability: the addition operation MUST be separable from CLI argument handling so it can be exercised directly with integer inputs and compared against an exact reference result. | should-have |

| NFR-43 | Observability: diagnostics for failed addition MUST be a single line on stderr, human-readable and naming the offending input, with no multi-line traceback and no internal paths. | should-have |

| NFR-44 | Encoding safety: the entire stdout payload MUST be ASCII (`0x30`–`0x39`, `0x2D`, `0x0A`), so that pipeline consumers and non-UTF-8 terminals cannot corrupt or block it. | must-have |

| NFR-45 | Concurrency safety: concurrent invocations MUST be independent — no shared temp files, lock files, or mutable global state MAY affect results. | must-have |

User Stories (US)

id	Story	Priority
---	---	---
US-31	As a shell user, I want `add 2 3` to print `5` so that I get an exact integer sum without leaving the terminal.	must-have
US-32	As a shell user, I want `add -7 4` to print `-3` so that negative and mixed-sign operands behave like ordinary integer arithmetic.	must-have
US-33	As a script author, I want the sum on stdout and nothing on stderr on success so that `sum=\$(add "\$a" "\$b")` captures the number uncontaminated.	must-have
US-34	As a script author, I want a stable exit status so that I can branch on success (`0`) versus invalid input (`2`) inside `if`, `&&`, and `  ` chains.	must-have
US-35	As a user handling large identifiers, I want `add 99999999999999999999 1` to print `10000000000000000000000000000000` exactly so that no precision is silently lost.	must-have
US-36	As a user, I want `add 5 -5` and `add -0 0` to print `0` so that signed zeros never leak into output.	must-have
US-37	As a user, I want `add 5 0` and `add 0 5` to print `5` so that zero behaves as the identity element in both positions.	should-have
US-38	As a pipeline author, I want the result to be a plain newline-terminated decimal integer so that downstream tools consume it without cleaning.	should-have

Behaviour

1. ****Precondition**** — The entry point (F-1) has established exactly two operands and each operand has passed F-2 validation. F-3 starts from two validated integer values.
2. ****Convert**** — Each validated operand's digit string is converted to an exact integer value. No rounding, scaling, or intermediate float representation occurs.
3. ****Add**** — Compute ``int1` + `int2`` exactly, with unbounded magnitude and correct sign per ``BR-2``.
4. ****Normalise**** — If the sum is zero, normalise it to unsigned zero (``BR-4``, ``FR-56``).
5. ****Render**** — Produce the canonical decimal string per ``FR-57``.
6. ****Emit**** — Write the rendered string plus exactly one ``\n`` to stdout; write nothing to stderr.
7. ****Exit**** — Terminate with status ``0``.
8. If any step before 6 fails (validation was not reached, an input exceeds a supported limit, or an internal error occurs), no sum is emitted and the error path in ``EH-1``–``EH-4`` applies.

Business rules

- **BR-1 — Sum definition.** The result is the unique integer `s`` such that `s - int1 = int2`` under exact arithmetic. There is no modular, bounded, saturating, or floating variant.
- **BR-2 — Sign resolution (exhaustive).**

```
| int1 sign | int2 sign | Result | | | | |
|---|---|---|---|---|---|---|
| + | + | positive, magnitude `|int1| + |int2|` |
| - | - | negative, magnitude `|int1| + |int2|` |
| + | - | sign of the larger magnitude; magnitude `abs(|int1| - |int2|)` |
| - | + | sign of the larger magnitude; magnitude `abs(|int1| - |int2|)` |
| equal magnitudes, opposite signs | | `0` |
```

- **BR-3 — Identity.** `0` (however spelled: `0`, `+0`, `-0`, `000`) contributes nothing to the sum.
- **BR-4 — Zero canonicalisation.** Every computed zero renders as `0`. `-0`, `+0`, `00`, and `0` are never valid outputs.
- **BR-5 — No expression parsing.** Operands are values, not expressions. Operators, parentheses, function names, or multiple terms inside a single argument (e.g. `2+3`, `(2)`, `2 3`) are not parsed; such an argument is simply a malformed operand and is rejected by the F-2 grammar (`FR-27`–`FR-32` there). Exactly one addition occurs per invocation; the tool is not an accumulator or REPL.
- **BR-6 — Sign spelling is not arithmetic.** An explicit `+` prefix on an operand has no effect beyond the value; `-` denotes negation of the following digits only.
- **BR-7 — Leading zeros are not arithmetic.** Leading zeros affect neither value nor output; the output never carries leading zeros (`FR-57`).
- **BR-8 — Bound of this feature.** F-3 does not re-validate grammar, does not decide arity, does not format diagnostics beyond the error-handling rules below, and does not define exit codes for arity errors (those belong to F-1).

- **V-1 (precondition).** Both operands MUST already be valid per the F-2 grammar; F-3 MUST reject-by-not-running if either is not. Addition on unvalidated text is forbidden (`FR-60`).
- **V-2 (postcondition).** The rendered result MUST match the canonical pattern `^-?(0|[1-9][0-9]*)$` and, when parsed back, MUST equal the exact sum (`FR-63`, `AC-12`).
- **V-3 (no result-side limits).** There is no maximum magnitude, no minimum, and no range check on the sum. Arbitrarily large and negative results are valid.
- **V-4 (no operand-side value checks).** F-3 MUST NOT reject an operand for being "too large", "zero", or "negative" — those are value properties, not errors.
- **V-5 (single newline).** Output validation MUST confirm exactly one terminating `\n` and no other control characters (`NFR-44`, `AC-15`).
- **V-6 (silence on success).** `stderr` MUST be zero bytes on every successful path (`FR-59`).

[illegible]

- **EC-14** — Digit-limit guard (CPython ≥ 3.11). `int()`/`str()` conversion is subject to `sys.set_int_max_str_digits` (default 4300 digits). An operand or a rendered result exceeding that limit will raise `ValueError`/`int` conversion error at the interpreter level. This MUST surface as the clean error path (`EH-3`), never as a raw traceback. Resolution deferred to `OQ-1`.
- **EC-15** — Memory exhaustion. An operand large enough to exhaust addressable memory may cause the process to be killed by the OS (SIGKILL/OOM). This is outside the graceful-handling contract and MUST be documented as such; the program MUST NOT attempt to pre-allocate based on claimed lengths.
- **EC-16** — Expression-shaped single argument. `add 2+3 1` → both arguments are malformed; exit `2` (`BR-5`).
- **EC-17** — Non-integer shaped operands passed through. `add 1.0 2`, `add 0x10 1`, `add 1e3 1` → rejected by F-2 grammar; exit `2`, no stdout.
- **EC-18** — Identical operands. `add 7 7` → `14`; the two operands are independent and MUST NOT be deduplicated or memoised into a wrong result.
- **EC-19** — Sum equals an operand. `add 0 42` → `42`; `add 42 0` → `42`; byte-identical to each other (`FR-62`).

Error handling

- **EH-1** — Invalid operand. F-3 MUST NOT run. The F-1 path applies: one-line diagnostic on stderr naming the offending argument, stdout empty, exit status `2`.
- **EH-2** — Wrong arity. F-3 MUST NOT run; the F-1 arity path applies (stderr usage/diagnostic, stdout empty, exit status `2`).
- **EH-3** — Internal conversion/limit failure. If operand conversion or result rendering raises (e.g. `EC-14`), the program MUST catch it at the top level and emit a single-line stderr diagnostic (e.g. identifying the argument or the limit involved), leave stdout empty, and exit `2`. A Python traceback MUST NOT be printed.
- **EH-4** — Output write failure. If writing the result fails (broken pipe, `ENOSPC`, `/dev/full`, closed stdout), the program MUST NOT exit `0` and MUST NOT retry silently; it MUST terminate non-zero with a one-line stderr diagnostic. A partially written number MUST NOT be treated as success.
- **EH-5** — No silent failure. There is no case in which the program exits `0` without having written a complete canonical result to stdout.
- **EH-6** — Encoding/terminal failure. Because the payload is pure ASCII (`NFR-44`), an unset or exotic `PYTHONIOENCODING`/`locale` MUST NOT cause a crash or an altered number; the bytes written MUST be identical regardless of the environment's text encoding.
- **EH-7** — Interrupt. On SIGINT/SIGPIPE the program MAY terminate before writing; it MUST NOT write a partial number followed by the newline in a way that a consumer could mistake for a complete result.

Acceptance criteria

id	Criterion
AC-1	<code>add 2 3</code> → stdout is exactly the bytes <code>5\n</code> ; stderr is empty; exit <code>0</code> .
AC-2	<code>add -7 4</code> → <code>-3\n</code> ; stderr empty; exit <code>0</code> .
AC-3	<code>add 7 -4</code> → <code>3\n</code> ; stderr empty; exit <code>0</code> .
AC-4	<code>add -7 -4</code> → <code>-11\n</code> ; stderr empty; exit <code>0</code> .
AC-5	<code>add 5 -5</code> → <code>0\n</code> (not <code>-0</code> , not blank); exit <code>0</code> .
AC-6	<code>add 0 0</code> → <code>0\n</code> ; exit <code>0</code> .
AC-7	<code>add -0 -0</code> and <code>add +0 -000</code> → <code>0\n</code> ; exit <code>0</code> .
AC-8	<code>add +00042 -0002</code> → <code>40\n</code> ; exit <code>0</code> .

- **DM-2** — No stored state. Nothing about an invocation persists after exit: no cache, no history file, no temp file, no environment mutation (`NFR-36`).

Priority

- **must-have**: `FR-51`–`FR-61`, `FR-63`; `NFR-33`, `NFR-34`, `NFR-36`, `NFR-37`, `NFR-38`, `NFR-39`, `NFR-40`, `NFR-41`, `NFR-44`, `NFR-45`; `US-31`–`US-36`.
- **should-have**: `FR-62`; `NFR-31`, `NFR-32`, `NFR-35`, `NFR-42`, `NFR-43`; `US-37`, `US-38`.
- **nice-to-have**: none. F-3 is the core value-producing feature of the tool; nothing in it is optional for a correct `add`.

Open Questions

- **OQ-1** — Digit-length limit. CPython ≥ 3.11 caps `int`↔`str` conversion via `sys.set_int_max_str_digits` (default 4300 digits), which affects both operand parsing (`EC-14`) and result rendering. Should the tool (a) accept the interpreter default and report a clean exit-`2` error beyond it, (b) raise the limit at start-up to support arbitrary lengths at extra CPU cost, or (c) document a deliberate maximum input length? This determines whether `NFR-32`'s 10,000-digit target is achievable as written. Decision belongs to the USER/Architect; F-3 specifies only that the failure mode **MUST** be clean (`EH-3`).
- **OQ-2** — Exit status for limit-exceeded input. Should an oversized-but-well-formed operand exit `2` (same as malformed, current spec) or a distinct status? Current spec: `2`, for consistency with F-1.
- **OQ-3** — `stdin`. The brief defines two positional operands and no piped input. F-3 therefore specifies that `stdin` **MUST** be ignored (`API-5`). Confirm this is intended and that no `add` reading a line from `stdin` is ever required.
- **OQ-4** — `SIGPIPE` policy. Should the tool silently exit on a broken pipe (`add 1 2 | head -n0` style) or exit non-zero with a diagnostic (`EH-4`)? Current spec requires non-zero; the user may prefer POSIX-conventional silent termination.

F-4: Print result to stdout

Feature ID: F-4

Depends on: F-1 (`add` command entry point), F-2 (two positional integer operands), F-3 (integer addition) — F-4 consumes the already-computed exact sum from F-3 and owns only the *rendering and emission* of that value to stdout. F-4 does **not** re-define argument handling, arity rules, the digit grammar, or arithmetic; those belong to F-1, F-2, and F-3 respectively.

Scope of this feature: given a successfully computed sum `S` (arbitrary-precision integer) and an accepted invocation, produce the canonical textual rendering of `S`, emit it on stdout as a single line, keep stderr empty, and signal success via the exit status.

Requirements

Functional Requirements (FR)

ID	Requirement	Priority
----	-------------	----------

FR-76	The program MUST write the textual rendering of the computed sum <code>S</code> to stdout on every successful run (i.e., whenever F-1 arity rules and F-2 operand grammar are satisfied).	must-have
-------	---	-----------

| FR-77 | `S` MUST be rendered in **base 10** using ASCII digit characters `0`–`9` (U+0030–U+0039) only. No other radix, and no radix prefix (`0x`, `0o`, `0b`), may appear. | must-have |

| FR-78 | The rendered value MUST be followed by **exactly one** line terminator: `\n` (U+000A, byte `0x0A`). Nothing may be written after it. | must-have |

| FR-79 | The rendering MUST NOT contain leading zeros, except that the value zero itself is rendered as the single character `0`. (`0` → `0`; `5` → `5`; `007`-style operands never leak into output.) | must-have |

| FR-80 | A negative sum MUST be prefixed with exactly one `-` (U+002D, byte `0x2D`) placed immediately before the first digit. A positive sum MUST NOT be prefixed with `+`. | must-have |

| FR-81 | The value zero MUST be rendered as `0` — never as `-0`, `+0`, `00`, `0.0`, an empty string, or any whitespace. Zero is a **successful result**, not an absence of output. | must-have |

| FR-82 | The output line MUST contain no padding, no thousands separators (`,`/`\_`/space), no exponent or decimal point, no unit, no label, and no prefix such as `=` or `sum:`. The line is the bare number. | must-have |

| FR-83 | stdout MUST be the sole channel for the result. No banner, version string, prompt, progress text, or diagnostic may be written to stdout under any circumstance. | must-have |

| FR-84 | On the success path, **stderr** MUST receive zero bytes. | must-have |

| FR-85 | On the success path, the process MUST terminate with exit status `0`, and it MUST do so only after the result bytes have been flushed to stdout. | must-have |

| FR-86 | The program MUST NOT apply terminal-dependent formatting: no line wrapping, paging, colour, or ANSI escape sequences. Output bytes MUST be identical whether stdout is a TTY, a pipe, a regular file, or `/dev/null`. | must-have |

| FR-87 | The emitted byte sequence MUST be ASCII-only: `0x30`–`0x39` for digits, an optional single leading `0x2D`, and a final `0x0A`. No UTF-8 BOM, no non-breaking spaces, no Unicode minus (U+2212) or en/em dash. | must-have |

| FR-88 | The complete line MUST be emitted as one logical unit (a single write, or writes that are flushed before exit) such that the **last byte** observable on stdout is the `0x0A` terminator. | must-have |

| FR-89 | If writing to stdout fails (e.g., closed pipe / `EPIPE`, disk full on a redirected file), the program MUST NOT surface a language traceback; it MUST write at most one diagnostic line to **stderr** and terminate with a non-zero status. | must-have |

| FR-90 | On any output-write failure the program MUST NOT exit with status `0` (no false success). | must-have |

| FR-91 | stdout MUST be flushed before the process terminates, so the result is never silently lost when stdout is a pipe or a file. | must-have |

| FR-92 | Rendering MUST be exact for sums of **arbitrary magnitude**: a result with thousands of digits MUST be printed in full, with no truncation, rounding, scientific notation, or ellipsis. | must-have |

| FR-93 | Rendering MUST be deterministic and pure: for a given integer value `S`, the produced byte sequence is always the same, independent of locale, environment variables, host, or run order. | must-have |

| FR-94 | No result value may be suppressed. Only argument/arity/parse failures (owned by F-1 and F-2) suppress output; a computed result — including `0` — is always printed. | must-have |

| FR-95 | The rendering MUST be idempotent under shell consumer use: a consumer capturing stdout MUST receive exactly the number plus one newline, with no trailing blank line. | must-have |

Non-Functional Requirements (NFR)

| ID | Requirement | Target / Measurement |

|---|---|---|

| NFR-46 | Output performance: rendering + emission of a 1000-digit result MUST add less than 50 ms over the F-1/F-2/F-3 processing time on a modern commodity CPU. | Measured wall-clock delta of render+write step, 100 runs, p95 < 50 ms. |

| NFR-47 | Memory: the output path MUST use memory $O(n)$ in the number of output digits, and MUST NOT buffer the whole pipeline beyond the single result line. | Peak RSS with a 100 000-digit result stays proportional to the digit count; no accumulated buffers. |

| NFR-48 | Availability: output MUST have no external dependency — no network, no filesystem read/write, no service, no daemon. The command works fully offline and in a sandbox with only a read-only filesystem. | Run with network disabled and CWD read-only; stdout contract still holds. |

| NFR-49 | Security: the result MUST be written verbatim as data. It MUST NOT be passed through a format string as a template, evaluated, interpreted, or expanded by a shell, and MUST never be used to construct a command. | Code review + test with an adversarial digit string; no evaluation/format expansion path exists. |

| NFR-50 | Privacy: operands and results MUST NOT be logged, telemetered, or duplicated to stderr on the success path. stderr byte count is exactly 0 on success. | Assert captured stderr length == 0 for a corpus of successful runs. |

| NFR-51 | Data residency/persistence: the process MUST NOT persist operands or results to disk, temp files, caches, history, or environment variables during normal operation. | Filesystem-snapshot diff before/after a run shows no writes outside the redirected stdout sink. |

| NFR-52 | Determinism/portability of output: `LANG`, `LC\_ALL`, `LC\_NUMERIC`, `PYTHONHASHSEED`, and terminal type MUST NOT change the output bytes. | Same invocation across locales `C`, `en\_US.UTF-8`, `de\_DE.UTF-8`, `ar\_EG.UTF-8` yields byte-identical stdout. |

| NFR-53 | Deployment/environment: Python 3 standard library only; no third-party packages; runs on Linux, macOS, and Windows invoked from a POSIX-style shell; no build step beyond byte-compilation. | Runs green in a clean interpreter with an empty site-packages for third-party imports. |

| NFR-54 | Output-sink robustness: the contract MUST hold when stdout is a TTY, a pipe, a regular file, a FIFO, or `/dev/null`. | Byte-level assertions per sink type. |

| NFR-55 | Reliability: the success path emits exactly one line and exactly one terminator; partial lines are never left terminated mid-number. | `wc -l` == 1 and `wc -c` == len(rendered)+1 on success. |

| NFR-56 | Observability: exit status is the machine-readable outcome signal (`0` success, non-zero failure); stderr carries at most one diagnostic line per failure and is empty on success. | Exit-code and stderr-length assertions across the test matrix. |

| NFR-57 | Composability/usability: stdout MUST be directly consumable by shell substitution and pipelines, e.g. `x=\$(add 2 3)` yields `5` with no whitespace stripping required. | Shell-level end-to-end test asserting the captured variable equals the bare number. |

| NFR-58 | Internationalization: output digits are ASCII and MUST NOT be subject to locale digit shaping, grouping, or RTL bidi marks. | Locale matrix comparison (see NFR-52); no U+200E/U+200F/U+0660-U+0669 in output. |

| NFR-59 | Encoding robustness: emission MUST succeed under a restrictive stdout encoding (e.g., `PYTHONIOENCODING=ascii` or `latin-1`) because the payload is ASCII. | Run with `PYTHONIOENCODING=ascii`; output bytes unchanged, exit status `0`. |

| NFR-60 | Testability: the output contract MUST be verifiable byte-for-byte by capturing stdout/stderr and the exit status of a subprocess. | Test suite asserts raw byte sequences (hex) and exit codes. |

User Stories (US)

| ID | Story | Priority |

|---|---|---|

| US-46 | As a command-line user, I want the sum printed on stdout, so that I can read the answer directly in my terminal. | must-have |

| US-47 | As a shell scripter, I want stdout to be exactly the number followed by one newline, so that `x=\$(add 2 3)` gives me `5` without extra text. | must-have |

| US-48 | As a shell scripter, I want exit status `0` on a printed result, so that I can branch on ` \$? ` reliably. | must-have |

| US-49 | As a pipeline author, I want stdout to contain nothing but the result, so that ``add 2 3 \| wc -c`` prints ``2``. | must-have |

| US-50 | As a user redirecting output to a file, I want exactly one trailing newline, so the file is a valid POSIX text file and ``wc -l`` prints ``1``. | must-have |

| US-51 | As a user computing negative sums, I want a single leading ``-`` on negative results, so that sign is unambiguous downstream. | must-have |

| US-52 | As a user computing very large sums, I want the complete exact digits, so that I can copy the value into another tool without losing precision. | must-have |

| US-53 | As a user, I want no banners, labels, or prompts around the number, so that output can be pasted straight into another command. | must-have |

| US-54 | As a user on a system with a non-UTF-8 locale, I want the result to still print correctly, so that the tool is usable everywhere. | should-have |

| US-55 | As a user whose stdout sink fails (broken pipe, full disk), I want a short error message and a non-zero exit rather than a Python traceback, so that the failure is diagnosable in a pipeline. | should-have |

| US-56 | As a maintainer, I want the rendering to be locale-independent and deterministic, so that tests are stable across CI environments. | must-have |

| US-57 | As a user capturing output into a variable, I want no trailing blank line, so that downstream string comparisons are not off by a newline. | should-have |

Behaviour (what it must do)

1. F-4 is entered only after F-1 has validated argument arity and F-2 has validated both operands against the integer grammar, and F-3 has produced the exact sum ``S``.
2. F-4 converts ``S`` into its canonical decimal string ``R``:
 - sign handling per ``BR-2``,
 - zero handling per ``BR-3``,
 - no leading zeros per ``BR-4``,
 - no separators or exponents per ``BR-5``.
3. F-4 appends exactly one ``\n`` to ``R``, producing the payload ``P = R + "\n"`` (``BR-6``).
4. F-4 writes ``P`` to stdout, then flushes stdout so the bytes are observable by the parent process before termination (``BR-9``, ``FR-91``).
5. F-4 writes nothing to stderr and returns exit status ``0`` (``FR-84``, ``FR-85``).
6. If any step of the write/flush fails, F-4 follows the error-handling path (``EH-1``–``EH-5``) instead of reporting success.
7. F-4 performs no arithmetic, no re-validation of operands, and no re-parsing of arguments.

****Data model (DM)****

ID	Item	Description
DM-1	<code>`S`</code>	The exact integer sum produced by F-3. Arbitrary precision; may be negative, zero, or positive; unbounded magnitude. Consumed read-only.
DM-2	<code>`R`</code>	Canonical decimal string form of <code>`S`</code> . Matches <code>`/^-(0 [1-9][0-9]*)\$/`</code> . ASCII only, non-empty, no leading zeros (except <code>`"0"`</code>), no <code>`+`</code> .
DM-3	<code>`P`</code>	The emitted payload: <code>`R`</code> concatenated with exactly one <code>`\n`</code> . Byte length <code>`= len(R) + 1`</code> .
DM-4	Exit status	Process outcome signal: <code>`0`</code> = result emitted; non-zero = failure (usage/parse errors owned by F-1/F-2; output-write failure owned by F-4, see <code>`EH-1`</code>).

Business rules

| ID | Rule |

|---|---|

| BR-1 | ****Single-line contract.**** The result is exactly one line: `R` followed by one `\n`. There is no preceding blank line, no trailing blank line, and no second line. |

| BR-2 | ****Sign rendering.**** Sign is emitted only for negative values, as a single leading `-` (U+002D). Positive values carry no sign character. `+` is never emitted, even if the operands carried `+` signs. |

| BR-3 | ****Zero is canonical.**** Any result with value zero renders as the single character `0`. `-0`, `+0`, `00`, `-00`, and empty output are all forbidden representations of zero. |

| BR-4 | ****No leading zeros.**** The digit sequence begins with a non-zero digit unless the value is exactly zero, in which case it is the single digit `0`. Input leading zeros accepted by F-2 are normalized away in output. |

| BR-5 | ****No decoration.**** No grouping separators, decimal point, exponent, unit, currency symbol, label, or prefix (e.g. `=`, `sum:`, `result:`) may be emitted. |

| BR-6 | ****Exactly one terminator.**** The line terminator is exactly `\n` (U+000A). `\r\n` (CRLF) and a missing terminator are both non-conforming. |

| BR-7 | ****stdout is result-only; stderr is diagnostics-only.**** The two streams are never mixed. On success stderr is byte-empty; on failure stdout carries no diagnostic. |

| BR-8 | ****Exit status reflects emission.**** Status `0` is emitted only if the full payload was written and flushed. Any output failure yields a non-zero status. |

| BR-9 | ****Flush before exit.**** The payload must be flushed before process termination so that a parent reading a pipe or file sees the complete line. |

| BR-10 | ****Presentation is value-determined, not environment-determined.**** Locale, terminal type, redirection target, and environment variables must not change the bytes produced for a given `S`. |

| BR-11 | ****Zero-length rendering is impossible.**** A successful run always emits at least two bytes: the shortest case is `0\n`. |

| BR-12 | ****No partial-line success.**** The program must never exit `0` having emitted part of a line, and must never emit the terminator before the full digit sequence is written (except as a consequence of an unrecoverable failure, which must be non-zero per `BR-8`). |

Validation

| ID | Validation | Failure handling |

|---|---|---|

| V-1 | ****Conformance to `DM-2`:**** after rendering, `R` must match `/^-(0|[1-9][0-9]\*)\$/`. | Internal invariant violation; must not occur. If it does, treat as an internal error, emit one diagnostic line to stderr, exit non-zero (`EH-5`). |

| V-2 | ****Non-empty:**** `R` must contain at least one character. | As V-1. |

| V-3 | ****ASCII-only:**** every byte of `P` must be in `{0x2D} ∪ [0x30..0x39] ∪ {0x0A}`, with `0x2D` permitted only at index 0 and `0x0A` only at the final index. | As V-1. |

| V-4 | ****Single terminator:**** `P` must end with exactly one `0x0A`, and `0x0A` must not appear anywhere else in `P`. | As V-1. |

| V-5 | ****No whitespace other than the terminator:**** `R` must contain no spaces, tabs, non-breaking spaces, or Unicode line separators. | As V-1. |

| V-6 | ****Round-trip value fidelity:**** parsing `R` back as a base-10 integer must yield exactly `S` (no digit loss, no truncation, no sign error). | As V-1; asserted in tests. |

| V-7 | ****Byte-length consistency:**** observed stdout byte count must equal `len(R) + 1`. | Test assertion failure; indicates a rendering/emission defect. |

Edge cases

ID	Situation	Required behaviour
EC-1	Sum is `0` (e.g. `add 0 0`, `add 5 -5`).	Emit `0\n`; exit `0`; stderr empty. Zero is printed, never suppressed.
EC-2	Operand is `-0` or `+0` and the sum is zero (e.g. `add -0 0`).	Emit `0\n`. Never `-0`, `+0`, or `00`.
EC-3	Both operands carry leading zeros (e.g. `add 007 +00042`).	Emit the normalized value `49\n`; the leading zeros must not survive into output.
EC-4	Result is a single-digit negative (e.g. `add -2 -3`).	Emit exactly three bytes: `2D 35 0A` (`-5\n`).
EC-5	Result crosses a magnitude boundary (e.g. `add 9999999999999999999 1`).	Emit the full multi-digit result `10000000000000000000\n`, no separators, no exponent.
EC-6	Result is negative and large (e.g. `add -9999999999999999999 -1`).	Emit `-10000000000000000000\n` with a single leading `-`.
EC-7	Result has thousands of digits (e.g. operands of 2000 digits each).	Emit every digit in full on one logical line; no truncation, no ellipsis, no wrapping inserted by the program.
EC-8	Result would be positive but an operand had an explicit `+` (e.g. `add +2 +3`).	Emit `5\n` — the `+` is never propagated to output (`BR-2`).
EC-9	stdout is a terminal.	Emit the same bytes as any other sink; no colour, paging, or TTY-specific formatting (`FR-86`).
EC-10	stdout is a pipe whose reader exits early (broken pipe / `EPIPE`).	Do not emit a traceback; emit at most one stderr diagnostic line; exit non-zero (`EH-1`, `FR-89`, `FR-90`).
EC-11	stdout is redirected to a file on a full filesystem (`ENOSPC`).	Same as EC-10: no traceback, one stderr line, non-zero exit.
EC-12	stdout is `/dev/null`.	Write succeeds; exit `0`; stderr empty. Sink semantics are the OS's concern, not the program's.
EC-13	stdout encoding is restricted (`PYTHONIOENCODING=ascii`).	Emission still succeeds byte-identically because the payload is ASCII (`NFR-59`).
EC-14	Locale sets a grouping/decimal convention (`LC_NUMERIC=de_DE.UTF-8`).	Output bytes are unchanged (no `.` or `,` grouping) (`BR-10`, `NFR-52`).
EC-15	Result is exactly one digit, positive (e.g. `add 2 3`).	Emit exactly two bytes: `35 0A` (`5\n`).
EC-16	Consumer uses command substitution (`x=\$(add 2 3)`).	`x` equals `5`; no trailing blank line (`US-57`).

Error handling

ID	Condition	Required handling
EH-1	stdout write fails with `EPIPE` / `BrokenPipeError` (downstream consumer closed the pipe).	Suppress the interpreter traceback. Write at most one single-line diagnostic to stderr. Terminate with a non-zero status (`FR-89`, `FR-90`). Exit status used: `1` (see `OQ-1`).
EH-2	stdout write fails with an I/O error other than a broken pipe (e.g. `ENOSPC`, `EIO`, `EACCES` on the redirect target).	Same as `EH-1`: single stderr diagnostic, non-zero exit, no traceback.
EH-3	stdout flush fails at process shutdown.	The failure MUST NOT be swallowed into a `0` exit status; non-zero exit and (best-effort) one stderr diagnostic. Suppress any traceback emitted by interpreter shutdown.

| EH-4 | `S` is an internal value that cannot be rendered per `V-1` (invariant violation; should be unreachable). | Emit one diagnostic line to stderr, emit nothing to stdout, exit non-zero. Never emit a malformed line with exit `0`. |

| EH-5 | Any unexpected exception occurs while rendering or emitting. | Suppress the traceback, emit at most one stderr diagnostic line, emit nothing further to stdout, exit non-zero. |

| EH-6 | Arity or operand-grammar failure (owned by F-1/F-2). | F-4 is not entered. stdout receives nothing; the diagnostic and exit status `2` are F-1/F-2's contract and are not re-specified here. |

| EH-7 | A diagnostic is required (any of `EH-1`–`EH-5`). | The diagnostic MUST go to stderr only, MUST be a single line, and MUST NOT contain a language traceback or the operand/result values (per `NFR-50`). |

| EH-8 | Partial output has already been written when a failure is detected. | Do not attempt to "repair" the line; ensure the exit status is non-zero so consumers do not treat the partial line as a successful result (`BR-12`, `FR-90`). |

Acceptance criteria

| ID | Criterion | Verified by |

|---|---|---|

| AC-1 | `add 2 3` writes exactly the bytes `35 0A` (`5\n`) to stdout, writes 0 bytes to stderr, and exits `0`. | Subprocess capture of raw bytes + exit code. |

| AC-2 | `add -2 -3` writes exactly `2D 35 0A` (`-5\n`), stderr empty, exit `0`. | Byte-level assertion. |

| AC-3 | `add 2 -3` writes exactly `2D 31 0A` (`-1\n`), stderr empty, exit `0`. | Byte-level assertion. |

| AC-4 | `add -2 3` writes exactly `31 0A` (`1\n`) — no `+`, no leading zero, stderr empty, exit `0`. | Byte-level assertion. |

| AC-5 | `add 0 0` writes exactly `30 0A` (`0\n`), exit `0`. Zero is printed, not suppressed. | Byte-level assertion. |

| AC-6 | `add -0 0` writes exactly `30 0A` (`0\n`) — never `-0`. | Byte-level assertion. |

| AC-7 | `add 007 +00042` writes exactly `34 39 0A` (`49\n`) — input leading zeros and the `+` sign do not appear in output. | Byte-level assertion. |

| AC-8 | For a sum of arbitrary magnitude (e.g. 1000+ digits), stdout contains the full exact digits with no separators, no exponent, no truncation; byte count equals digit count + sign + 1. | Length + regex + round-trip parse assertions. |

| AC-9 | On every success case, captured stderr length is exactly `0`. | `len(stderr) == 0` across the success corpus. |

| AC-10 | On every success case, stdout ends with exactly one `0x0A`, contains no other `0x0A`, no `0x0D`, and no whitespace bytes other than the terminator. | Byte scan assertion. |

| AC-11 | `add 2 3 \| wc -c` prints `2` and `add 2 3 \| wc -l` prints `1`. | Shell pipeline integration test. |

| AC-12 | `x=\$(add 2 3)` results in `x` equal to `5` (no trailing blank line, no surrounding whitespace). | Shell substitution test. |

| AC-13 | Output bytes are byte-identical across `LC\_ALL=C`, `LC\_ALL=en\_US.UTF-8`, `LC\_ALL=de\_DE.UTF-8`, and `PYTHONIOENCODING=ascii`. | Locale/encoding matrix byte comparison (`NFR-52`, `NFR-59`). |

| AC-14 | Output bytes are byte-identical when stdout is a TTY (pty), a pipe, a regular file, and `/dev/null`. | Sink matrix byte comparison (`NFR-54`). |

| AC-15 | When the stdout pipe is closed before the write (`add 2 3 \| head -c 0`), the process does not print a traceback, emits at most one stderr line, and exits non-zero. | Integration test inspecting stderr for traceback markers and checking exit code $\neq 0$. |

| AC-16 | When stdout is redirected to a full filesystem, the process exits non-zero with at most one stderr line and no success indication. | Fault-injection test (small tmpfs). |

****Non-conforming outputs (explicitly forbidden):**** `5` with no newline; `5\n\n`; `5\n\n`; `5\n`; `5 \n`; `+5\n`; `05\n`; `-0\n`; `0.0\n`; `5e0\n`; `sum: 5\n`; `=5\n`; `\n` alone; empty stdout with exit `0`; ANSI-coloured digits; any stdout diagnostic on a failure.

Priority

****Priority: must-have.****

Rationale: printing the result is the sole observable purpose of the command — `add 2 3` → `5` is the stated vision of the brief. Without F-4 the program computes a value that no user or downstream process can observe. All FRs in this section are `must-have`; the should-have items are limited to the degradation-path and convenience behaviours captured in US-54, US-55, and US-57 (robust failure reporting and shell-substitution ergonomics), which do not alter the success-path contract.

Open questions

| ID | Question | Why it blocks nothing right now |

|---|---|---|

| OQ-1 | F-1/F-2 define exit status `2` for usage/parse errors and `0` for success. This section proposes exit status `1` for an output-write/flush failure (broken pipe, disk full) to keep the categories distinguishable. Confirm `1` is the desired code, or supply the preferred code. | Defaulted to `1` (`API-4`, `EH-1`, `EH-2`); tests assert only "non-zero" for these paths in `AC-15`/`AC-16` so the spec is usable either way. |

| OQ-2 | On a broken-pipe failure (`EC-10`), should the program write a diagnostic line to stderr at all, or exit silently with a non-zero status (the common Unix convention for SIGPIPE-style exits)? | Defaulted to "at most one diagnostic line" (permissive: silent is compatible with "at most one"); confirm if silence is mandatory. |

| OQ-3 | Should an output-write failure attempt to align with shell convention by terminating on `SIGPIPE` rather than catching the error? | Not required by the brief; the brief constrains only the success path. Defaulted to catching and exiting non-zero without a traceback (`FR-89`). |

F-5: Exit-status contract

****Feature ID:**** F-5

****Depends on:**** F-1 (`add` command entry point), F-2 (two positional integer operands), F-3 (integer addition), F-4 (print result to stdout) — F-5 consumes the **outcome categories** already produced by F-1, F-2, F-3 and F-4 and owns only the ****process exit status**** as an externally observable interface. F-5 does ****not**** re-define arity rules (F-1), the operand grammar (F-2), arithmetic (F-3), or stdout rendering (F-4); it fixes the status value that each of those outcomes must yield, and the invariants that hold between the status and the two byte streams.

Requirements

Functional Requirements (FR)

| ID | Requirement | Priority |

|---|---|---|

| FR-101 | The process MUST terminate with exit status `0` **iff** exactly two operands were supplied, both satisfied the F-2 operand grammar, and the F-4 rendering of the sum was written to stdout as a complete line. | must-have |

| FR-102 | The process MUST terminate with exit status `2` when argument arity is not exactly two operands (zero, one, or three-plus arguments), writing nothing to stdout. | must-have |

| FR-103 | The process MUST terminate with exit status `2` when any supplied operand fails the F-2 operand grammar, writing nothing to stdout. | must-have |

| FR-104 | The process MUST terminate with exit status `1` when an unexpected internal failure or an output (I/O) failure prevents completion, including resource exhaustion and a closed/broken stdout destination. Status `1` MUST NOT be produced by any input-condition error. | must-have |

| FR-105 | The set of exit statuses the process may produce for **any** input is exactly `{0, 1, 2}`. No other status, no negative status, and no termination by signal-as-control-flow may occur on a defined input. | must-have |

| FR-106 | Every outcome MUST map deterministically to exactly one row of the normative status table (BR-1). The same argv, environment, and stream destinations MUST always yield the same status. | must-have |

| FR-107 | On exit status `0`, stderr MUST be empty (zero bytes) and stdout MUST contain exactly the F-4 rendering (base-10 digits followed by exactly one `\n`), with nothing else. | must-have |

| FR-108 | On exit status `2`, stdout MUST be empty (zero bytes) and stderr MUST contain at least one non-empty, single-line diagnostic terminated by `\n`. | must-have |

| FR-109 | On exit status `1`, stderr MUST contain at least one non-empty, single-line diagnostic terminated by `\n`, unless stderr itself is unwritable, in which case the status `1` is still returned. | must-have |

| FR-110 | The process MUST NOT emit an unhandled-exception traceback on any outcome reachable from user input. Any caught internal failure is reported as a single-line diagnostic, retaining status `1` (FR-104). | must-have |

| FR-111 | Validation precedence MUST be: arity first (F-1), then operand grammar (F-2), then arithmetic and rendering (F-3/F-4). The first failing stage determines the status; later stages MUST NOT run once an earlier stage fails. | must-have |

| FR-112 | When both an arity error and an operand-grammar error are present (e.g., three arguments where the second is also malformed), the status MUST be `2` and the diagnostic MUST describe the arity error. | should-have |

| FR-113 | Exit status MUST be delivered after stdout and stderr contents are flushed, so that observing status `0` implies the sum line was fully written to the stream (subject to EC-4). | must-have |

| FR-114 | Exit status `0` MUST NOT be returned if any byte was written to stderr, or if a write of the sum line failed or was truncated. | must-have |

| FR-115 | Exit status MUST be independent of locale, `LANG`/`LC\_\*`, `TERM`, environment variables, current working directory, TTY-vs-pipe destination, file descriptors 0/1/2 reassignment, umask, and wall-clock time. | must-have |

| FR-116 | Exit status MUST NOT depend on the magnitude, sign, or textual length of the operands or the result, beyond whether they pass the F-2 grammar. | must-have |

| FR-117 | Exit status MUST be observable as-is by the invoking shell (e.g., `add 2 3; echo \$?` → `0`; `add 2; echo \$?` → `2`) and MUST be in the range `0`–`255`. | must-have |

| FR-118 | Statuses `3`–`125` MUST remain unallocated and MUST NOT be produced. Statuses `126`, `127`, and `128`+ MUST NOT be produced (they are reserved to the shell for command-not-executable, command-not-found, and signal termination). | must-have |

| FR-119 | "No arguments" and "empty-string argument(s)" MUST both yield status `2`, with nothing on stdout. | must-have |

| FR-120 | Exit status for a *successful* run MUST be reachable when stdin is closed, absent, or not readable; the process MUST NOT read from stdin in any case. | must-have |

| FR-121 | The status value MUST be set by an explicit, documented path in the program. The program MUST NOT rely on implicit interpreter defaults, implicit fall-through, or the value of the last evaluated expression to produce status `0`. | must-have |

| FR-122 | The exit-status mapping is part of the public interface. Any change to a status value or to which condition yields it is a breaking interface change requiring a spec revision; it MUST NOT be altered silently (‘FR-105’, ‘FR-106’). | must-have |

Non-Functional Requirements (NFR)

| ID | Requirement | Priority |

|---|---|---|

| NFR-61 | ****Performance**** — Total wall-clock time from process start to exit-status availability MUST be ≤ 250 ms for operand strings up to 1,000 digits on a modern desktop CPU; status determination MUST add no measurable latency beyond the F-4 output emission. | must-have |

| NFR-62 | ****Performance**** — Determining the status MUST be O(1) in the number of operands (fixed at two) and MUST NOT introduce any extra pass over the result beyond what F-4 already performs. | should-have |

| NFR-63 | ****Scalability / availability**** — The status mapping MUST be total and deterministic: for 100% of invocations, exactly one table row matches. There is no retry, no partial availability, and no stateful degradation. | must-have |

| NFR-64 | ****Robustness**** — Correct status behaviour MUST hold under fuzzed/random argv (including binary bytes, control characters, very long arguments, and non-UTF-8 byte sequences) with no unhandled exceptions and no status outside {0, 1, 2}. | must-have |

| NFR-65 | ****Security**** — Diagnostics MUST NOT leak interpreter internals, stack frames, absolute filesystem paths, environment dumps, or the raw unescaped argument. Echoed argument text MUST be control-character-escaped and truncated (≤ 64 characters) before being written to stderr. | must-have |

| NFR-66 | ****Security**** — Arguments MUST be treated as inert data only; no evaluation, no shell invocation, no path resolution, and no file access may be triggered by argument content, and no argument content may influence the status except through F-2 grammar validation. | must-have |

| NFR-67 | ****Data handling / residency**** — The process MUST persist nothing and read nothing from disk or network; operands and result exist only in process memory, and the only bytes leaving the process are on stdout and stderr. No telemetry, no logs, no temp files. | must-have |

| NFR-68 | ****Deployment / environment**** — Runs on Linux and macOS with a POSIX-style shell and a Python 3 standard-library runtime only; no environment variables, config files, or companion processes are required to produce the documented statuses. | must-have |

| NFR-69 | ****Portability**** — Statuses ‘0’ and ‘2’ (and the reserved ‘1’) MUST be observable identically under ‘sh’, ‘dash’, ‘bash’, and ‘zsh’, and MUST remain meaningful when the status is truncated to the low 8 bits. | must-have |

| NFR-70 | ****Observability**** — The pair (exit status, single-line stderr diagnostic) MUST be sufficient to diagnose any failure without additional tooling or debug output. | should-have |

| NFR-71 | ****Stream safety**** — Behaviour MUST be defined and non-throwing for stdout/stderr redirected to a file, a pipe, ‘/dev/null’, or closed before invocation. | must-have |

| NFR-72 | ****Resource limits**** — Peak memory MUST be O(n) in operand/result length; exhausting memory MUST resolve to status ‘1’ with a diagnostic, never to an unhandled exception or a wrong success status. | should-have |

| NFR-73 | ****Terminal usability**** — Diagnostics MUST be plain ASCII, single-line, and free of ANSI escape sequences, so they render correctly in dumb terminals, log files, and screen readers. | should-have |

| NFR-74 | ****Testability**** — The status table MUST be fully verifiable by an automated harness that spawns the process and asserts (status, stdout bytes, stderr bytes) triples, with one case per table row plus each edge case. | must-have |

| NFR-75 | ****Compatibility**** — Status values MUST remain stable across all patch releases of the tool; only a spec revision may change them. | must-have |

User Stories (US)

| ID | Story | Priority |

|---|---|---|

| US-61 | As a **shell user**, I want a successful ``add 2 3`` to exit ``0``, so that ``add 2 3 && echo ok`` behaves as expected. | must-have |

| US-62 | As a **shell user**, I want a failing ``add`` to exit non-zero, so that ``&&/set -e`` pipelines stop instead of continuing with a bad value. | must-have |

| US-63 | As a **script author**, I want usage and grammar errors to exit ``2`` and unexpected failures to exit ``1``, so that I can distinguish "bad input, fix the caller" from "tool broke, retry/report". | must-have |

| US-64 | As a **CI author**, I want exit statuses to be deterministic for identical inputs, so that build outcomes are reproducible and cacheable. | must-have |

| US-65 | As a **script author**, I want stdout to be empty when the command fails, so that captured output is never polluted by a partial or bogus number. | must-have |

| US-66 | As a **pipeline author**, I want a broken/closed stdout to produce a clean non-zero status rather than a traceback, so that logs stay readable. | should-have |

| US-67 | As a **security-conscious operator**, I want no stack traces or environment details in stderr, so that failures cannot leak internals into shared logs. | must-have |

| US-68 | As a **user**, I want a one-line explanation on stderr whenever the command fails, so that I know what to change without reading docs. | must-have |

| US-69 | As a **shell user**, I want statuses within ``0`–`255``, so that ``$?`` checks and shell conditionals behave predictably. | must-have |

| US-70 | As an **automation author**, I want the status to be independent of locale, cwd, TTY, and environment, so that containerized runs match my laptop. | must-have |

| US-71 | As a **user**, I want status ``0`` to guarantee the full sum line was actually delivered, so that I can trust the output I just redirected to a file. | must-have |

| US-72 | As a **tester**, I want a documented, normative status table, so that I can write one assertion per row and prove coverage. | must-have |

| US-73 | As a **maintainer**, I want the status mapping to be frozen as a contract, so that refactors cannot silently break downstream scripts. | must-have |

| US-74 | As a **user** who redirects output to ``/dev/null``, I want the exit status to be unaffected by where output goes, so that silencing output does not change control flow. | must-have |

| US-75 | As a **user** adding very large integers, I want the exit status to stay ``0`` regardless of magnitude, so that big numbers are never mistaken for an error. | must-have |

Behaviour

``add`` exposes a four-way observable interface: the argv it receives, the bytes it writes to stdout, the bytes it writes to stderr, and the integer exit status it returns. This feature specifies the last of those and the invariants tying it to the other three.

1. **Evaluate arity (F-1).** Count the operands. If the count is not exactly two, produce the usage diagnostic on stderr, write nothing to stdout, and stop with status ``2``.
2. **Validate operands (F-2).** If either operand fails the digit grammar, produce a single-line diagnostic on stderr naming (escaped, truncated) the offending argument, write nothing to stdout, and stop with status ``2``.
3. **Compute and render (F-3, F-4).** Compute the exact sum and render it as a base-10 line. Write it to stdout.
4. **Determine status.** If step 3 completed and the write to stdout was not detected as failed, return ``0``. If an internal or output failure occurred at any point, return ``1`` after emitting a single-line diagnostic.

Stages 1–3 are ordered and mutually exclusive: the first failing stage fixes the status, and no stage may run after an earlier stage has failed (``FR-111``).

Business Rules

- **BR-1** (Normative status table). The following mapping is total over all possible inputs and is the single source of truth for exit status:

#	Outcome	Condition	stdout	stderr	Exit
T0	Success	Exactly two operands; both pass the F-2 grammar; sum line written completely	<base-10 sum>\n	empty	0
T1	Usage error	Operand count is 0, 1, or ≥ 3	empty	usage diagnostic + \n	2
T2	Operand error	Operand count is 2; ≥ 1 operand fails the F-2 grammar	empty	single-line operand diagnostic + \n	2
T3	Internal / I/O failure	Resource exhaustion, unwritable stdout detected, or any unexpected internal failure	empty or partially written	single-line diagnostic (best effort) + \n	1

- **BR-2** (Precedence). Arity before grammar before arithmetic before rendering. Rows T1 and T2 both yield 2; when both conditions are present, T1's diagnostic (arity) is reported.

- **BR-3** (Status is not a severity ladder). 1 is reserved for "the tool did not work as designed", never for "the user typed something wrong". Input errors are always 2.

- **BR-4** (Claim of success). Status 0 is a claim that stdout holds exactly one line: the base-10 rendering of the sum followed by one \n, and stderr is empty. This claim MUST hold whenever 0 is returned.

- **BR-5** (Freshness). No file, lock, cache, or persistent state participates in status determination; the status is a pure function of argv and the writability of the output streams.

- **BR-6** (Frozen contract). The table in BR-1 is an interface. Additions require a spec revision; removals or remappings are breaking changes.

- **BR-7** (Reserved space). 3–125 unallocated; 126, 127, and 128+ reserved to the shell and never emitted by this program.

Validation

- **V-1** — Before returning any status, the program MUST assert that the chosen value is an element of {0, 1, 2}.

- **V-2** — Exactly one BR-1 row MUST match the observed outcome; zero or two matching rows is a defect.

- **V-3** — Status 0 MUST be accompanied by empty stderr; a non-empty stderr with status 0 is a defect (FR-114).

- **V-4** — Statuses 2 and 1 MUST be accompanied by a non-empty, single-line stderr diagnostic, except the unwritable-stderr case in FR-109.

- **V-5** — For statuses 2 and 1, stdout MUST be empty (status 1 may allow a detected partial write, per EC-4).

- **V-6** — stderr diagnostics MUST contain no traceback markers (Traceback (most recent call last), File ()), no absolute paths, and no newline-separated multi-line bodies.

- **V-7** — Each BR-1 row MUST be exercised by an automated case asserting the (status, stdout, stderr) triple (NFR-74).

- **V-8** — The status MUST be identical across re-runs with the same argv under varied LANG, LC_ALL, cwd, and stdout destinations (FR-115).

Edge Cases

ID	Case	Required behaviour
----	------	--------------------

ID	Failure	Handling	Status
EC-1	Zero arguments (<code>`add`</code>)	Status <code>`2`</code> , stdout empty, usage diagnostic on stderr (row T1).	
EC-2	One argument (<code>`add 5`</code>)	Status <code>`2`</code> , stdout empty, usage diagnostic (row T1).	
EC-3	Three or more arguments (<code>`add 1 2 3`</code>)	Status <code>`2`</code> , stdout empty, usage diagnostic; the extra operands are never summed (row T1).	
EC-4	stdout closed or connected to a pipe whose reader has exited (broken pipe / <code>`add 2 3   head -0`</code>)	No traceback. Status <code>`1`</code> with a single-line diagnostic on stderr; no status <code>`0`</code> is reported.	
EC-5	stdout redirected to <code>`/dev/null`</code>	Status <code>`0`</code> ; redirection destination does not change the status.	
EC-6	Empty-string operand (<code>`add "" 3`</code>)	Status <code>`2`</code> (rows T1/T2), stdout empty; the empty argument is named in the diagnostic.	
EC-7	Malformed operand that looks numeric (<code>`add 0x10 3`</code> , <code>`add 1.5 2`</code> , <code>`add " 3" 4`</code> , <code>`add 1_000 2`</code>)	Status <code>`2`</code> , stdout empty (delegated to F-2 validation).	
EC-8	Both an arity error and a malformed operand (<code>`add 1 x 3`</code>)	Status <code>`2`</code> ; diagnostic describes the arity error (<code>`BR-2`</code>).	
EC-9	Very large operands (e.g., 100,000 digits)	Status <code>`0`</code> on success; magnitude never alters the status (<code>`FR-116`</code>). If memory is exhausted, status <code>`1`</code> (<code>`NFR-72`</code>).	
EC-10	Negative / zero result (<code>`add -5 5`</code> → <code>`0`</code>)	Status <code>`0`</code> ; the value of the result never alters the status.	
EC-11	stdin closed or absent	Status is unchanged; stdin is never read (<code>`FR-120`</code>).	
EC-12	Non-UTF-8 / binary argv bytes	Status <code>`2`</code> (grammar failure) or <code>`0`</code> (if the bytes are valid digits), never a traceback; diagnostic escapes non-printable bytes.	
EC-13	stderr closed	Status still reflects the outcome; status <code>`2`/`1`</code> are returned even though the diagnostic cannot be delivered (<code>`FR-109`</code>).	
EC-14	Non-ASCII digit look-alikes (e.g., Arabic-Indic digits)	Status <code>`2`</code> ; only ASCII <code>`0`–`9`</code> are accepted (delegated to F-2).	

Error Handling

ID	Failure	Handling	Status
EH-1	Wrong arity	Catch at the entry stage; print usage line to stderr; emit no stdout bytes	<code>`2`</code>
EH-2	Operand fails grammar	Catch before arithmetic; print one line naming the escaped, truncated offending argument	<code>`2`</code>
EH-3	Unexpected internal exception during parse/add/render	Catch at the top level; print a generic single-line diagnostic; suppress the traceback	<code>`1`</code>
EH-4	stdout write failure / broken pipe	Detect the failed write; suppress the traceback; print a single-line diagnostic to stderr if possible; never report <code>`0`</code>	<code>`1`</code>
EH-5	Memory/resource exhaustion	Reduce to a single-line diagnostic where possible; otherwise exit <code>`1`</code> silently on stderr but never <code>`0`</code>	<code>`1`</code>
EH-6	stderr unwritable	Do not attempt retry or re-open; return the status that the outcome dictates	<code>`2`</code> or <code>`1`</code>

Acceptance Criteria

ID	Criterion
AC-1	Given <code>`add 2 3`</code> , when run in a POSIX shell, then stdout is exactly <code>`5\n`</code> , stderr is empty, and <code>`\$?`</code> is <code>`0`</code> .
AC-2	Given <code>`add -5 5`</code> , then stdout is exactly <code>`0\n`</code> , stderr is empty, and <code>`\$?`</code> is <code>`0`</code> .
AC-3	Given <code>`add`</code> , <code>`add 5`</code> , or <code>`add 1 2 3`</code> , then stdout is empty, stderr is non-empty with one line, and <code>`\$?`</code> is <code>`2`</code> .

AC-4	Given `add 1 x`, then stdout is empty, stderr names the offending argument as a single line, and `\$?` is `2`.	
AC-5	Given `add "" 3`, then stdout is empty and `\$?` is `2`.	
AC-6	Given any invocation, the observed `\$?` is always an element of `{0, 1, 2}` (FR-105).	
AC-7	Given `add 2 3	head -0` (reader exits early), no traceback appears on stderr and `\$?` is `1`, never `0` (EC-4, FR-114).
AC-8	Given `add 2 3 > /dev/null; echo \$?`, the result is `0` (EC-5).	
AC-9	Given the same argv run 100 times with varied `LANG`, `LC\_ALL`, cwd, and stdout destination, the status is identical every time (FR-115, FR-106).	
AC-10	Given 100,000-digit operands, the run exits `0` with a correct sum line (FR-116, EC-9).	
AC-11	Given any failing run, stderr contains no traceback marker, no absolute path, and exactly one line (V-6, NFR-65).	
AC-12	Given `add 1 x 3` (arity ****and**** grammar error), `\$?` is `2` and the diagnostic describes the arity error (BR-2, FR-112).	
AC-13	Given a fuzzed corpus of random/binary argv values, no run produces a status outside `{0, 1, 2}` and no run produces an unhandled traceback (NFR-64, FR-110).	
AC-14	Given status `0`, stdout parses as exactly one base-10 integer line and stderr is zero bytes (BR-4, FR-107).	

API Behaviour

The "API" of this feature is the process boundary.

ID	Contract element	Specification
API-1	****Invocation****	`add <int1> <int2>` — two positional operands, no flags, no options, no environment switches.
API-2	****Return value (exit status)****	Integer in `{0, 1, 2}`: `0` = success (T0); `2` = user/input error (T1, T2); `1` = internal or output failure (T3).
API-3	****stdout contract****	Success: exactly `` + `\n` (owned by F-4). Error: zero bytes, except the detected partial-write case which forces status `1`.
API-4	****stderr contract****	Empty on success. Exactly one `\n`-terminated diagnostic line on `2` and `1` (best effort when stderr is unwritable). Diagnostics are ASCII, control-character-escaped, and ≤ 64 chars of echoed argument.
API-5	****Idempotency****	`status(argv, stdout\_dest, stderr\_dest)` is a pure function; no hidden state, no caching, no ordering effects across invocations.
API-6	****Error signalling****	Out-of-band: the status itself is the machine-readable signal; stderr is the human-readable signal. Neither is emitted in the other's channel.
API-7	****Compatibility surface****	The status values and the condition→status mapping are contractual; changes require a spec revision (BR-6, NFR-75).
API-8	****Unsupported inputs****	Anything other than exactly two grammar-valid operands is an unsupported input and returns `2`; unsupported inputs MUST NOT be partially processed.

Priority

****Overall:**** must-have. The exit-status contract is the only machine-readable result of the tool and is a prerequisite for shell composability (`&&`, `||`, `set -e`), scripting, and automated verification.

- ****must-have:**** FR-101–FR-111, FR-113–FR-122; all of NFR-61, NFR-63–NFR-69, NFR-71, NFR-74, NFR-75; US-61–US-65, US-67–US-75.
 - ****should-have:**** FR-112; NFR-62, NFR-70, NFR-72, NFR-73; US-66.

- **F-5 local ID scope (non-global):** `AC-1`–`AC-14``, `BR-1`–`BR-7``, `V-1`–`V-8``, `EC-1`–`EC-14``, `EH-1`–`EH-6``, `API-1`–`API-8`` are scoped to this feature section and may be reused with different meanings in other feature sections.

- **Cross-references:** F-1 (arity and entry point), F-2 (operand grammar), F-3 (integer addition), F-4 (stdout rendering) are referenced as plain text; their IDs are not re-defined here.

F-6: Input validation and error message

Feature ID: F-6

Depends on: F-1 (`add`` command entry point), F-2 (two positional integer operands), F-5 (exit-status contract).

Boundary note: F-2 owns *whether* an operand is accepted (the grammar). F-1 owns the arity rule and the fact that violations go to stderr with status `2``. F-5 owns the numeric exit status itself. **F-6** owns only the validation *pipeline orchestration* and the *human-readable diagnostic catalogue* — the ordering of checks, the exact text/format of messages, the safe echoing of offending input, and the invariant that a failing run produces no stdout bytes. F-6 redefines no grammar, no arity rule, and no exit-status value.

Requirements

Functional Requirements (FR)

| ID | Requirement | Priority |

|---|---|---|

| FR-126 | Validation MUST run as a fixed, deterministic pipeline: **(1)** arity check → **(2)** grammar check of the first operand → **(3)** grammar check of the second operand → **(4)** hand-off to computation/output. The first failing stage MUST abort the pipeline; later stages MUST NOT run. | must-have |

| FR-127 | An arity failure (argument count ≠ 2) MUST emit exactly two stderr lines: an error line naming the expected count and the received count, followed by a usage line. | must-have |

| FR-128 | A grammar failure MUST emit exactly one stderr line identifying the offending operand by 1-based position and echoing the offending value verbatim (subject to FR-137/FR-143). | must-have |

| FR-129 | Every emitted diagnostic line MUST be ASCII-only, free of leading and trailing whitespace, and terminated by exactly one `\n`` (U+000A). | must-have |

| FR-130 | On any validation failure the program MUST write **zero bytes** to stdout. | must-have |

| FR-131 | On any validation failure the program MUST terminate with exit status `2``, as defined by F-5. | must-have |

| FR-132 | All diagnostics MUST be written to stderr only; stderr MUST be empty on a successful run. | must-have |

| FR-133 | If more than one operand is invalid, the program MUST report **only the first** offender (first operand) and MUST suppress messages about subsequent operands — exactly one diagnostic line is emitted per run for grammar failures. | must-have |

| FR-134 | Validation MUST consume only the process argument vector; it MUST NOT read stdin, environment variables, or any file — with the single exception that it MUST NOT be perturbed by locale/encoding environment variables (see NFR-84). | must-have |

| FR-135 | No input, of any length or byte content, may cause an unhandled exception, traceback, or non-`0``/non-`2`` exit status. All rejections are controlled diagnostics. | must-have |

| FR-136 | Every diagnostic MUST begin with the stable, greppable prefix `add: error:`` (arity and grammar alike), so that a single log filter captures all failures. | must-have |

| FR-137 | The verbatim echo MUST be rendered in an escaped-printable form: characters outside printable ASCII (U+0020–U+007E) MUST be replaced by a backslash escape (e.g. ``n``, ``t``, ``x1b``, non-ASCII as ``xNN`/`uNNNN``). A diagnostic MUST be unable to inject extra lines or terminal control sequences. | must-have |

| FR-138 | The program MUST NOT coerce, trim, or auto-correct invalid input. ``2.0``, `` 3``, ``1e3``, ``0x10``, ``1_000`` and ``3 `` MUST all be rejected, never reinterpreted. | must-have |

| FR-139 | Diagnostics MUST NOT disclose internal implementation detail: no Python tracebacks, no exception class names, no source file paths, no module or function names, no version-build internals. | must-have |

| FR-140 | Arity MUST be checked before grammar: an invocation with three arguments where the first is also malformed MUST produce the `**arity**` diagnostic. | must-have |

| FR-141 | The diagnostic catalogue MUST be a fixed, closed set of message templates: exactly one distinguishable template per failure class, defined in §Business rules ``BR-5`–`BR-7``. | must-have |

| FR-142 | Diagnostic text MUST be emitted in a single fixed language (English) in the C locale and MUST NOT vary with ``LC_ALL``, ``LANG``, or ``PYTHONIOENCODING``. | should-have |

| FR-143 | A verbatim echo longer than 64 characters MUST be truncated to the first 64 characters followed by the three-character marker ``...``. | should-have |

| FR-144 | Exit-status selection for validation failures MUST be delegated to F-5; F-6 MUST NOT introduce any status value other than ``2``. | must-have |

| FR-145 | Because validation strictly precedes computation and output, no partial or speculative stdout write may ever occur on a failing run. | must-have |

Non-Functional Requirements (NFR)

| ID | Requirement | Priority |

|---|---|---|

| NFR-76 | ****Performance:**** On a failing invocation, the diagnostic MUST be emitted within 50 ms of process start on reference hardware, for total argv length up to 1 MiB. | must-have |

| NFR-77 | ****Complexity:**** Validation and diagnostic construction MUST be linear, $O(\text{total argv length})$; no quadratic scanning or repeated concatenation of long operand strings. | should-have |

| NFR-78 | ****Memory:**** Validation MUST NOT allocate a copy of the full argument string beyond the echo buffer bounded by FR-143. | should-have |

| NFR-79 | ****Atomicity:**** Each diagnostic line MUST be written with a single write operation where the platform permits, so a message is never observed partially interleaved. | should-have |

| NFR-80 | ****Scalability/Availability:**** The validator is stateless and side-effect-free; concurrency across processes is unlimited (no locks, no temp files, no shared state, no network). | must-have |

| NFR-81 | ****Security:**** Diagnostics MUST NOT echo environment variables, current working directory, hostname, username, PID, or timestamps; stderr output MUST be derived solely from argv. | must-have |

| NFR-82 | ****Determinism:**** Identical argv MUST always yield byte-identical stderr and an identical exit status, across runs and machines. | must-have |

| NFR-83 | ****Testability (black-box):**** Every clause of §Acceptance criteria MUST be verifiable externally by asserting on ``(exit_status, stdout_bytes, stderr_bytes)`` without importing program internals. | must-have |

| NFR-84 | ****Environment/Locale independence:**** Output bytes MUST be invariant under ``LC_ALL=C``, ``LC_ALL=C.UTF-8``, and any other locale setting, and under ``PYTHONIOENCODING`` overrides. | must-have |

| NFR-85 | ****Data/residency:**** No data leaves the machine and nothing is persisted — validation writes only to the inherited stderr stream and retains no file, cache, or log. | must-have |

| NFR-86 | ****Deployment/environment:**** Validation MUST depend only on the Python 3 standard library and the Python runtime; no third-party packages, no network access, no shelling out to external binaries. | must-have |

| NFR-87 | **Coverage:** Every rejection branch enumerated in the F-2 grammar MUST have at least one corresponding test vector in §Test vectors (`T-1`–`T-12`). | should-have |
| NFR-88 | **Robustness:** Argv containing NUL-free non-UTF-8/surrogate-escaped bytes MUST be handled without raising and rendered per FR-137. | should-have |

User Stories (US)

ID	Story	Priority

US-76	As a CLI user, I want a clear, single-line message when I supply the wrong number of arguments, so that I can correct the invocation without consulting documentation.	must-have
US-77	As a CLI user, I want the message to tell me *which* argument was rejected and show me exactly what I typed, so that I do not have to guess which of the two operands is at fault.	must-have
US-78	As a shell scripter, I want every failure to produce empty stdout, an explanatory stderr message, and exit status `2`, so that I can branch on the status while still surfacing a useful error to the user.	must-have
US-79	As a tool integrator, I want all failures to share a stable `add: error:` prefix, so that a single grep or regex can capture them in logs.	should-have
US-80	As a security-conscious operator, I want no tracebacks, paths, environment values, or terminal-escape sequences in error output, so that logs are safe to display and forward.	must-have

Behaviour

The validator is a pure function of argv producing a triple `(stdout\_bytes, stderr\_bytes, exit\_status)`:

1. **Stage 1 — Arity:** Count positional operands after the program name. If the count is not exactly 2, emit `BR-5` (two lines) and stop.
2. **Stage 2 — First operand grammar:** Apply the F-2 grammar to operand 1. If it fails, emit `BR-6` with position `1` and stop.
3. **Stage 3 — Second operand grammar:** Apply the F-2 grammar to operand 2. If it fails, emit `BR-6` with position `2` and stop.
4. **Stage 4 — Success path:** Hand both accepted operands to F-3/F-4. stderr stays empty; exit status `0`.

Properties of the whole pipeline:

- **Fail-fast, first-error-only (FR-126, FR-133):** exactly one failure is ever reported, even when several inputs are wrong.
- **Silence on success (FR-132):** a successful run writes 0 bytes to stderr.
- **Silence on failure (FR-130, FR-145):** a failing run writes 0 bytes to stdout, because stdout is never opened/written until after validation completes.
- **No side effects:** no files, no temp files, no network, no environment reads affecting output (NFR-85, FR-134).
- **Idempotence (NFR-82):** re-running with the same argv yields byte-identical results.

Business rules

ID	Rule

| BR-1 | **Failure classes are closed.** There are exactly two externally distinguishable failure classes: *arity* and *grammar*. No other class may be introduced. |

| BR-2 | **Arity precedes grammar (FR-140).** Count is validated for all arguments before any individual argument is inspected, so a three-argument call is always an arity error regardless of argument content. |

| BR-3 | **Operand order is significant (FR-133).** Operands are validated left-to-right; the first invalid operand is reported and validation stops there. |

| BR-4 | **Echo fidelity.** The rejected operand is echoed as submitted, after escape rendering (FR-137) and truncation (FR-143). It is never normalised, trimmed, lower-cased, or re-quoted beyond the surrounding single quotes of the template. |

| BR-5 | **Arity template (FR-127).** Exactly two lines, in this order:
`<br>`add: error: expected exactly 2 operands, got <N>`<br>`add: usage: add <int1> <int2>`<br>where `<N>` is the observed count rendered in decimal (e.g. `0`, `1`, `3`).` |

| BR-6 | **Grammar template (FR-128).** Exactly one line:
`<br>`add: error: invalid integer argument <P>: `<V>`
where `<P>` ∈ {`1`,`2`} is the 1-based operand position and `<V>` is the escape-rendered, possibly truncated echo of the rejected operand.` |

| BR-7 | **Template closure (FR-141).** No message other than BR-5 and BR-6 may be emitted by the validator. Additions require an amendment to this rule. |

| BR-8 | **No usage line on grammar failure.** The usage line is emitted only for arity failures; a grammar failure is a single line. |

| BR-9 | **No numeric analysis of invalid input.** The validator MUST NOT attempt to explain *why* an operand was invalid (no "did you mean", no range advice, no radix suggestions) — it reports only position and value. |

| BR-10 | **Zero-hypothesis on exit status (FR-144).** The validator signals failure as a boolean; the concrete status value `2` is applied by the F-5 contract. |

Validation

| ID | Validation | Applies to | Failure outcome |

---|---|---|---

| V-1 | Operand count must equal exactly `2`. | Full argv | BR-5, status `2` |

| V-2 | Operand 1 must satisfy the F-2 operand grammar (optional single sign + one or more ASCII digits, nothing else). | `argv[1]` | BR-6 (`P=1`), status `2` |

| V-3 | Operand 2 must satisfy the F-2 operand grammar. | `argv[2]` | BR-6 (`P=2`), status `2` |

| V-4 | Diagnostics must contain printable ASCII only. | Emitted lines | Internal assertion; never observable |

| V-5 | Emitted lines must be ≤ 256 bytes each, guaranteed by BR-5/BR-6 and truncation FR-143. | Emitted lines | Internal assertion |

| V-6 | stdout must be 0 bytes whenever status ≠ `0`. | Full run | Internal assertion before write |

Validation is *rejection-only*: F-6 performs no semantic normalisation and cannot alter an accepted value — once V-2/V-3 pass, the operand string is forwarded byte-for-byte to F-3.

Edge cases

| ID | Case | Required behaviour |

---|---|---

| EC-1 | No operands (`add`) | Arity error, `` = `0`, BR-5, status `2`. |

| EC-2 | One operand (`add 5`) | Arity error, `` = `1`, BR-5, status `2`. |

| EC-3 | Three-plus operands (`add 1 2 3`) | Arity error, `` = `3` (actual count), BR-5, status `2`. |

| EC-4 | Three-plus operands *and* a malformed first operand (`add 1.5 2 3`) | Arity error wins (BR-2/FR-140); `` = `3`; the malformed operand is not mentioned. |

| EC-5 | Both operands malformed (`add a b`) | Only `P=1` is reported (BR-3). |

| EC-6 | Second operand malformed only (`add 5 2.0`) | `P=2` reported; first operand not mentioned. |

| EC-7 | Empty-string operand (``add "" 2``) | Empty string fails V-2 (no digits) → BR-6 with ``<V>`` = empty, rendering ``invalid integer argument 1: ""``. |

| EC-8 | Operand that is only a sign (``add - 2``, ``add + 2``) | Fails V-2 → BR-6, sign echoed. |

| EC-9 | Whitespace-padded operand (``add " 5" 2``) | No trimming (FR-138) → fails V-2 → BR-6 with the leading space visible inside the quotes. |

| EC-10 | Embedded newline in an operand (``add $'1\n2' 3``) | Echo rendered as ``1\n2`` via FR-137; message remains a single line (never splits). |

| EC-11 | ANSI escape in an operand (``add $'\e[31m5' 2``) | Rendered as ``\x1b[31m5``; no control byte reaches the terminal (FR-137, NFR-81). |

| EC-12 | Very long operand (e.g. 100 000 digits then a letter) | Echo truncated to 64 chars + ``...`` (FR-143); message still ≤ 256 bytes; runtime linear (NFR-77). |

| EC-13 | Non-ASCII operand (``add 5 é``) | Rejected; non-ASCII byte(s) escape-rendered (FR-137, NFR-88). |

| EC-14 | Valid but unusual accepted input (``add +0007 -0``) | ****Not**** an F-6 error — passes V-2/V-3 and proceeds; F-6 emits nothing. |

| EC-15 | stderr redirected to a closed descriptor | Failure must not raise or change status semantics; the process still exits ``2``. |

| EC-16 | stderr redirected to a file | Exactly the template bytes are written, with no trailing whitespace and no extra blank line. |

Error handling

| ID | Situation | Handling |

|---|---|---|

| EH-1 | Arity violation detected | Emit BR-5 to stderr in one write where possible (NFR-79); stdout untouched; resolve to status ``2`` via F-5. |

| EH-2 | Grammar violation detected | Emit BR-6 to stderr; stdout untouched; resolve to status ``2`` via F-5. |

| EH-3 | Unexpected internal error during validation | MUST NOT surface a traceback (FR-135): the program exits ``2`` with a message from the closed catalogue, and never exits with any other status. |

| EH-4 | Write to stderr fails (EPIPE / closed descriptor) | Swallow the write failure; do not raise; preserve the ``2`` exit status (EC-15). |

| EH-5 | Multiple simultaneous violations | Report only the first per FR-126/FR-133; no aggregation, no "and also" messages. |

| EH-6 | Malformed bytes in argv | Escape-render (FR-137) rather than decode-fail; no exception (NFR-88). |

| EH-7 | Any failure path | Guarantee the invariant ``status ≠ 0`` ■ ``stdout_bytes == 0`` before any output is attempted. |

Acceptance criteria

| ID | Criterion (black-box: assert on ``status``, stdout bytes, stderr bytes) |

|---|---|

| AC-1 | ``add`` with 0 arguments → status ``2``, stdout ``b""``, stderr exactly ``b"add: error: expected exactly 2 operands, got 0\nadd: usage: add <int1> <int2>\n"``. |

| AC-2 | ``add 5`` → status ``2``, stdout ``b""``, stderr with ``got 1`` per BR-5. |

| AC-3 | ``add 1 2 3`` → status ``2``, stdout ``b""``, stderr with ``got 3`` and no mention of any operand value. |

| AC-4 | ``add 1.5 2`` → status ``2``, stdout ``b""``, stderr exactly ``b"add: error: invalid integer argument 1: '1.5'\n"``. |

| AC-5 | ``add 5 2.0`` → status ``2``, stdout ``b""``, stderr exactly ``b"add: error: invalid integer argument 2: '2.0'\n"``. |

| AC-6 | ``add a b`` → status ``2``, stderr contains exactly one occurrence of ``argument 1`` and zero occurrences of ``argument 2``. |

| AC-7 | ``add 1.5 2 3`` → status ``2``, stderr equals the AC-3 arity output (arity precedes grammar). |

| AC-8 | Every stderr line ends with exactly one ``\n``; no line has leading or trailing spaces; total stderr contains no ``\r``. |

| AC-9 | stderr output is pure printable ASCII (``0x20`–`0x7E` plus `\n`) for every test vector including EC-10/EC-11/EC-13. |`

| AC-10 | No test vector produces a traceback: ``b"Traceback"`, `b"File \\"`, and `b"Exception"`, never appear in stderr. |`

| AC-11 | All failing vectors exit with exactly ``2``; no vector exits with a status other than ``0`` or ``2``. |

| AC-12 | Successful vectors (``add 2 3``, ``add +0007 -0``, ``add -5 5``) produce stderr ``b""`` and status ``0`` — F-6 emits nothing on success. |

| AC-13 | Rejection of ``" 5"`, `"5 "`, `"1_000"`, `"0x10"`, `"1e3"`, `""`, `"- "`, `"+ "` each yields BR-6 and never a numeric result on stdout. |`

| AC-14 | With ``LC_ALL=C`, `LC_ALL=C.UTF-8`, and `PYTHONIOENCODING=latin-1`, the stderr bytes for a fixed failing vector are identical. |`

| AC-15 | Two consecutive identical failing runs produce byte-identical stderr (determinism, NFR-82). |

| AC-16 | A 100 000-character invalid operand completes with status ``2`` in < 50 ms and produces stderr ≤ 256 bytes with the ``...`` truncation marker present. |

| AC-17 | Every all-failing vector has stdout of length 0 (asserted for the entire catalogue). |

| AC-18 | The error prefix ``add: error:`` appears exactly once per grammar failure and once per arity failure. |

API behaviour

The "API" of ``add`` is the process interface; F-6 defines its failure surface.

| ID | Surface | Contract |

| ---|---|---|

| API-1 | `**Input:**` ``argv[1:]`` | Exactly two positional operands; no flags, no options, no ``--``, no environment input. Operand grammar per F-2. |

| API-2 | `**stdout**` | On success only: ``<sum>\n`` per F-4. On `**any**` failure: ``b""`` (0 bytes). |

| API-3 | `**stderr**` | On success: ``b""``. On arity failure: BR-5 (2 lines). On grammar failure: BR-6 (1 line). ASCII-only, ``add: error:``-prefixed, each line ``\n``-terminated, ≤ 256 bytes/line. |

| API-4 | `**exit status**` | ``0`` on success; ``2`` on every validation failure (F-5 owns the value). No other status is reachable from validation. |

| API-5 | `**Failure-class discriminator**` | Distinguishable without parsing prose: arity failures emit 2 stderr lines (second begins ``add: usage:``); grammar failures emit 1 line. |

| API-6 | `**Stability guarantee**` | Templates BR-5/BR-6 and the ``add: error:`` prefix are a versioned public contract; changing them is a breaking change. |

| API-7 | `**Side effects**` | None: no files created/modified/deleted, no network, no subprocess, no signal handlers installed. |

| API-8 | `**Encoding**` | stderr bytes are ASCII regardless of locale; consumers may decode as ASCII/UTF-8 without loss. |

Test vectors

| ID | argv (after program name) | Expected status | Expected stdout | Expected stderr |

| ---|---|---|---|---|

| T-1 | `*(none)*` | ``2`` | empty | BR-5, ``got 0`` |

| T-2 | ``5`` | ``2`` | empty | BR-5, ``got 1`` |

| T-3 | ``1 2 3`` | ``2`` | empty | BR-5, ``got 3`` |

```
| T-4 | `1.5 2` | `2` | empty | BR-6, `1`, `1.5` |
| T-5 | `5 2.0` | `2` | empty | BR-6, `2`, `2.0` |
| T-6 | `a b` | `2` | empty | BR-6, `1`, `a` only |
| T-7 | `1.5 2 3` | `2` | empty | BR-5, `got 3` |
| T-8 | `"" 2` | `2` | empty | BR-6, `1`, `` |
| T-9 | `` 5" 2` | `2` | empty | BR-6, `1`, ` 5` |
| T-10 | `0x10 1` | `2` | empty | BR-6, `1`, `0x10` |
| T-11 | `2 3` | `0` | `5\n` | empty |
| T-12 | `+0007 -0` | `0` | `7\n` | empty |
```

Priority

****Overall: must-have.**** F-6 is the sole observable failure interface of the utility. Without it, F-1s arity rule, F-2s grammar, and F-5s status contract have no user-visible expression, and callers cannot distinguish "bad invocation" from "bad operand".

```
| Sub-scope | Priority | Rationale |
|---|---|---|
| Stage ordering, fail-fast, first-error-only (FR-126, FR-133, FR-140, BR-3) | must-have | Determines deterministic, testable failure classification. |
| Stream and status invariants (FR-130, FR-131, FR-132, FR-145) | must-have | The core machine contract for scripters (US-78). |
| Diagnostic templates BR-5/BR-6 and prefix (FR-127, FR-128, FR-136, FR-141) | must-have | Human usability plus greppable stability. |
| No-throwback and no-disclosure rules (FR-135, FR-138, FR-139, NFR-81) | must-have | Correctness and safety of error output. |
| Escape rendering (FR-137, NFR-88) | must-have | Prevents log injection and terminal corruption. |
| Locale invariance (FR-142, NFR-84) | should-have | Determinism across environments. |
| Echo truncation (FR-143) | should-have | Bounded, single-line output for pathological inputs. |
| Complexity/memory/atomicity targets (NFR-77, NFR-78, NFR-79) | should-have |
Quality-of-implementation targets; not user-observable except at extreme sizes. |
```

Open Questions

```
| ID | Question | Needed from |
|---|---|---|
| OQ-1 | Should the arity diagnostic tolerate a `--help`/`-h` argument as a distinct, non-error path? The current brief defines no option parsing (F-2, API-1), so any `--help` is an arity/grammar failure. Confirming "no flags at all" keeps BR-7 closed. | User |
| OQ-2 | Is the exact diagnostic wording (BR-5/BR-6) contractual for downstream consumers, or may it be reworded freely? This determines whether API-6 stability is enforced by tests. | User |
| OQ-3 | Should `add 1 2 3` suggest the correct invocation (already covered by the BR-5 usage line), and should the received-count `` be capped for very large argument counts? Current rule echoes the true count uncapped. | User |
```

F-7: Standard library only

****Feature ID:**** F-7

****Depends on:**** F-1 (`add` command entry point), F-2 (two positional integer operands), F-3 (integer addition), F-4 (print result to stdout), F-5 (exit-status contract), F-6 (input validation and error message) — F-7 does ****not**** re-define argument handling, the operand grammar, arithmetic, rendering, exit

status, or diagnostic text. F-7 owns only the **dependency envelope**: the requirement that every runtime capability used by F-1–F-6 is satisfied by the Python 3 standard library, and the observable consequences of that envelope (no installation step, no third-party imports, no network, hermetic execution).

Requirements

Functional Requirements (FR)

ID	Requirement	Priority
FR-151	The program MUST run on a stock CPython 3 interpreter (3.8 or later) with no third-party packages installed beyond those shipped with the interpreter distribution itself.	must-have
FR-152	The implementation MUST import only modules from the Python 3 standard library, or use built-in types/functions that require no import at all. No <code>pip install</code> step, virtual-environment dependency, <code>requirements.txt</code> entry, or vendored third-party file may be required to execute <code>add</code> .	must-have
FR-153	The program MUST NOT perform any network access (no socket, HTTP, DNS, or IPC-to-remote-service call) at any point in its lifecycle, including at import time and at exit.	must-have
FR-154	The program MUST NOT require, read, or write any configuration file, environment variable, home-directory state, database, or on-disk cache in order to compute and print the sum. Behaviour MUST be identical when the environment is empty.	must-have
FR-155	The program MUST NOT create, modify, or delete any file or filesystem entry as a side effect of a normal run (success or failure). It is a pure stdout/stderr/exit-status filter.	must-have
FR-156	The integer parsing performed for F-2 and the integer arithmetic performed for F-3 MUST be provided by standard-library or built-in facilities only (built-in <code>int</code> , <code>str</code> , and <code>sys</code> are sufficient); no external numeric, bignum, or parser library may be used.	must-have
FR-157	The program MUST NOT depend on any compiled extension module or platform-specific native library beyond those bundled with the standard CPython distribution. Source-only operation on a pure-Python interpreter is acceptable.	should-have
FR-158	If a standard-library module is imported at module scope, the import MUST NOT introduce observable side effects (no threads, no files, no sockets, no locale mutation) before argument processing begins.	should-have

Non-Functional Requirements (NFR)

ID	Requirement	Target	Priority
NFR-91	Portability — the program MUST execute correctly on at least Linux, macOS, and Windows under stock CPython 3.8+, with identical stdout bytes and exit status for identical arguments.	3/3 platforms, byte-identical output	must-have
NFR-92	Cold-start cost — total interpreter-plus-program startup, add, and print MUST complete within 250 ms wall-clock on a contemporaneous developer machine with no warm cache.	p95 < 250 ms	should-have
NFR-93	Dependency surface — the number of third-party runtime dependencies MUST be exactly zero, and the number of standard-library modules imported MUST be 3 or fewer.	0 third-party; ≤3 stdlib	must-have
NFR-94	Hermeticity — execution MUST NOT require clock accuracy, random entropy, elevated privileges, or a writable working directory.	runs with <code>HOME=/nonexistent</code> , read-only CWD	must-have

| NFR-95 | ****Auditability**** — a reviewer MUST be able to enumerate every imported module and every external capability used by reading a single source file, with no dynamic imports, `__import__` calls, or `importlib` use. | 100% statically enumerable | should-have |

User Stories (US)

| ID | Story | Priority |

|---|---|---|

| US-91 | As a ****developer****, I want to run ``add 2 3`` on any machine that has Python 3, so that I do not have to install or configure anything first. | must-have |

| US-92 | As a ****security reviewer****, I want the tool to make no network calls and touch no files, so that I can approve it for use in an offline or locked-down environment without a threat-model review. | must-have |

| US-93 | As a ****packager / CI maintainer****, I want the tool to have zero third-party dependencies, so that there is no dependency tree to pin, audit, or patch. | should-have |

Behaviour

- ****B-1.**** ``add`` is delivered as one or more Python source files. Running it requires only the ``python3`` executable already present on the target machine.

- ****B-2.**** On invocation, the program proceeds directly to argument handling (F-1) without reading configuration, probing the network, or inspecting the filesystem.

- ****B-3.**** All capabilities exercised at runtime fall into one of three buckets: (a) Python built-ins (``int``, ``str``, ``print``, ``sys.argv``), (b) standard-library modules shipped with CPython, or (c) the operating system's own process-startup and stream machinery, which the interpreter provides.

- ****B-4.**** The observable output — stdout bytes, stderr bytes, exit status — is a pure function of the argument vector. No environment variable, file, locale beyond the fixed ASCII grammar, or machine property may alter it (see FR-154, NFR-94).

- ****B-5.**** Behaviour is identical whether the program is invoked as ``python3 add.py 2 3``, as an installed console script, or via any other launcher the packaging adopting agent chooses, provided the same interpreter and the same arguments are used. The three-way contract (stdout, stderr, exit status) is unchanged.

Business rules

| ID | Rule |

|---|---|

| BR-1 | The ****allowed dependency set**** for the runtime path is the Python 3 standard library exactly as defined by the CPython documentation for the target minor version, plus the language's built-in objects. Nothing else qualifies. |

| BR-2 | Build-time, test-time, and developer-tooling dependencies (linters, formatters, test runners) are ****out of scope**** for this rule: they may exist, but they MUST NOT be required to execute ``add`` and MUST NOT appear on the runtime import path. |

| BR-3 | A module is considered "third-party" if it is not part of the interpreter's own distribution. Vendoring third-party source into the repository does not launder it into a standard-library dependency; the rule is about provenance and audit surface, not packaging mechanics. |

| BR-4 | The runtime path MUST be statically resolvable: every module referenced must appear in a literal ``import`` statement (or be a built-in). Dynamic import mechanisms (``__import__``, ``importlib.import_module``, ``exec`` of import strings) are prohibited on the runtime path. |

| BR-5 | The no-network rule (FR-153) is absolute and has no "optional telemetry" exception. There is no version check, no update ping, no crash reporter. |

| BR-6 | The no-filesystem-write rule (FR-155) is absolute for the runtime path. Reading is additionally prohibited by FR-154 except for the interpreter's own reading of the program's own source, which is the interpreter's action, not the program's. |

| BR-7 | Because the tool is a pure filter, its correctness is fully characterised by the mapping ``argv[1:]`` → `(stdout_bytes, stderr_bytes, exit_status)``. Any behaviour that cannot be observed through those three channels is not part of the contract. |

Validation

| ID | Validation |

|---|---|

| V-1 | ****Static import audit**** — parse the source with ``ast`` and assert that every ``Import`/`ImportFrom`` node references a name in ``sys.stdlib_module_names`` (Python 3.10+) or an explicitly maintained allow-list for earlier versions. Fail the check on any other name. |

| V-2 | ****Dynamic-import audit**** — assert that the source contains no ``__import__``, no reference to ``importlib``, and no ``exec`/`eval`` applied to constructed strings on the runtime path. |

| V-3 | ****Clean-environment execution test**** — run ``add 2 3`` in a container with no site-packages, no ``PYTHONPATH``, no ``HOME``, and a read-only filesystem (except the interpreter's own read-only mount). Assert stdout is exactly ``5\n`` and exit status is ``0``. |

| V-4 | ****Network-denial test**** — run under a sandbox that denies all outbound and inbound socket syscalls. Assert the program still succeeds with identical output. |

| V-5 | ****Filesystem-write-denial test**** — run with the entire filesystem mounted read-only. Assert the program still succeeds with identical output. |

| V-6 | ****Portability smoke test**** — run the same argument vectors (including a negative sum, a zero sum, a large-magnitude sum) on Linux, macOS, and Windows under stock CPython ≥3.8, and assert byte-identical stdout and identical exit status. |

| V-7 | ****Import-count check**** — count distinct standard-library modules imported on the runtime path and assert the count is ≤3 (NFR-93). |

| V-8 | ****Argv-purity test**** — run ``add 2 3`` with a large, adversarial set of environment variables set and unset; assert the tuple `(stdout, stderr, exit_status)`` is unchanged across all runs. |

Edge cases

| ID | Edge case | Expected behaviour |

|---|---|---|

| EC-1 | Python is invoked with ``-S`` (site disabled) and ``-E`` (environment ignored). | The program runs normally; no site-packages are needed (FR-152). |

| EC-2 | The interpreter has an aggressive audit hook or import hook installed that blocks third-party imports. | No third-party import is ever attempted, so the hook never fires on ``add``'s behalf. |

| EC-3 | The target machine has no network interface configured at all. | Identical behaviour; startup performs no resolution or connection attempt. |

| EC-4 | The current working directory is read-only and contains no files the program could write. | Success path is unchanged; no file is written. |

| EC-5 | ``HOME`` points at a non-existent path and ``XDG_*`` variables are unset. | No configuration lookup occurs; behaviour is unchanged. |

| EC-6 | The program is run by a user with no write permission anywhere on the filesystem. | Success path is unchanged. |

| EC-7 | The interpreter is a pure-Python build (e.g. PyPy-compatible pure-Python mode, or CPython built without optional C accelerators). | Program still works; no C-extension dependency is required (FR-157). |

| EC-8 | `PYTHONPATH` points at a directory containing a hostile module with a name that shadows something the program imports. | Out of scope for F-7's own contract, but the audit V-1/V-2 ensures the program imports nothing shadowable beyond interpreter-provided names; the residual risk is documented as OQ-1. |

| EC-9 | Locale is set to a non-ASCII locale (e.g. `tr\_TR.UTF-8`, `ja\_JP.UTF-8`). | Parsing and rendering are ASCII-fixed (F-2, F-4); no locale-sensitive formatting call is used, so output is unchanged. |

| EC-10 | The program's source is distributed as a single `.py` file with no accompanying files. | Still runnable; no companion file, data file, or package metadata is required at runtime. |

Error handling

| ID | Situation | Behaviour |

|---|---|---|

| EH-1 | A future maintainer adds a third-party import to the runtime path. | Detected by validation V-1 and V-2; the build's compliance gate fails. This is a spec violation of FR-152, not a runtime error path. |

| EH-2 | The interpreter is too old (<3.8) and lacks a syntax feature used by the source. | The interpreter itself reports a syntax error to stderr and exits non-zero before `add`'s own contract begins. F-7's scope is Python 3.8+; no program-level handling is required. |

| EH-3 | The sandbox denies a syscall the interpreter needs at startup (e.g. `mmap`). | Interpreter-level failure; `add` emits nothing and the interpreter supplies the diagnostic. No program-level handling; F-7 owns only the "no *program-initiated* network/file call" rule. |

| EH-4 | An audit hook raises on a standard-library import the program legitimately uses. | The interpreter reports the hook's error; `add` does not catch it. This is an environment policy decision, not a contract violation. |

| EH-5 | A dependency is discovered post-hoc to have been vendored. | Treated as a violation of BR-3 and remediated by removing the vendored code; the acceptance criteria below fail until it is removed. |

Note: EH-2 through EH-4 are environment-level failures that occur *before or around* program control, not failures the program itself generates. F-7 deliberately introduces **no new runtime error path**: F-1, F-2, F-5, and F-6 own every runtime diagnostic this tool can emit, and F-7 changes none of them.

Acceptance criteria

| ID | Criterion |

|---|---|

| AC-1 | On a machine with stock Python 3.8+ and **no site-packages**, `add 2 3` prints `5\n` to stdout, prints nothing to stderr, and exits `0`. |

| AC-2 | The set of names imported by the program, discovered by static AST analysis, is a subset of the Python standard library and contains no third-party name (V-1). |

| AC-3 | The program's source contains no `\_\_import\_\_`, no `importlib`, and no `exec`/`eval` on the runtime path (V-2). |

| AC-4 | Under a sandbox with all network syscalls denied, all argument vectors that succeed outside the sandbox succeed inside it with byte-identical stdout, byte-identical stderr, and identical exit status. |

| AC-5 | Under a read-only filesystem, all argument vectors that succeed outside the sandbox succeed inside it with byte-identical stdout, byte-identical stderr, and identical exit status. |

| AC-6 | The tuple `(stdout, stderr, exit\_status)` for a fixed argument vector is identical across Linux, macOS, and Windows under stock CPython 3.8+ (V-6). |

| AC-7 | The number of distinct standard-library modules imported on the runtime path is 0 or more and ≤ 3 (V-7, NFR-93). |

| AC-8 | The tuple `(stdout, stderr, exit\_status)` for a fixed argument vector does not change when the environment is emptied with `env -i` (V-8, FR-154). |

| AC-9 | The program writes no file, creates no directory, and opens no file for writing during a normal run; verified by syscall tracing (V-5). |
| AC-10 | The tool passes its full F-1 through F-6 acceptance criteria unchanged — F-7 adds capabilities restrictions and removes none of the previously specified behaviour (see BR-7 and the "no new error path" note). |

API behaviour

F-7 is a **constraint feature**, not an interface feature. It exposes no new API of its own; it constrains the implementation of the API defined by F-1 through F-6. Its "surface" is therefore the dependency envelope, and it is exercised through the following probes.

ID	Probe	Contract
API-1	***Static probe*** — ``ast.parse(source)`` then walk for ``Import`/`ImportFrom``.	Every imported top-level name resolves to a standard-library module for the target interpreter version.
API-2	***Execution probe*** — ``env -i python3 -S -E add.py <args...>`` in an empty directory.	Exit status and stdout/stderr bytes match the same invocation in a fully equipped environment.
API-3	***Sandbox probe*** — run under a seccomp/AppArmor/bwrap profile that denies ``socket``, ``connect``, ``open`-for-write`, and ``unlink``.	Same tuple as API-2; no policy violation is logged against the ``add`` process.
API-4	***Portability probe*** — replay a fixed corpus of argument vectors on the three target OS families.	Cross-OS byte-equality of ``(stdout, stderr, exit_status)``.
API-5	***Audit probe*** — instrument ``builtins.__import__`` at process start and record every call.	The recorded set equals the statically discovered set (API-1); no dynamic name appears.

The ***deterministic data model*** for this feature is two-sided:

ID	Entity	Definition
DM-1	``RuntimeImportSet``	The set of module names imported by the program's runtime path, each of which **MUST** be a member of the interpreter's standard-library name set.
DM-2	``EnvDependency``	The value ``∅``. F-7 asserts the program has an empty environment dependency: no config key, no environment variable, no file path, and no service endpoint that alters behaviour.

Open Questions

ID	Question	Owner
OQ-1	The upstream plan fixes the language as "Python 3 standard library only" but does not fix the **minimum minor version**. Should the design target a specific floor (3.8, 3.9, 3.10, or "any currently supported CPython")? NFR-91 assumes 3.8 as the floor, matching the ``sys.stdlib_module_names``-free validation path; if a higher floor is chosen, V-1 can use that attribute directly and drop the allow-list.	USER
OQ-2	Should the " ≤ 3 standard-library modules imported" cap in NFR-93 be treated as a hard gate or as a soft target? The minimum viable implementation imports ``sys`` only (one module); a paranoid reading would demand zero imports and route through ``os.read`/`os.write``, which paradoxically increases the import count.	USER
OQ-3	Is the distribution artifact expected to be a single ``py`` file (EC-10), or is a small package with a ``__main__.py`` acceptable? F-7 is agnostic, but the answer affects how V-2's "statically enumerable" audit walks the tree.	USER

Priority (feature-level): must-have. F-7 is not an optional hardening pass; the brief's scope statement — "Python 3 standard library only" — makes the empty third-party dependency set a defining property of the deliverable. Removing it would not reduce the feature's complexity, it would contradict the brief.

F-8: Correct handling of negative results

Feature ID: F-8

Depends on: F-1 (add command entry point), F-2 (two positional integer operands), F-3 (integer addition), F-4 (print result to stdout), F-5 (exit-status contract) — F-8 does **not** re-define argument handling, arity rules, the operand grammar, the arithmetic itself, the rendering envelope, or any exit-status value. F-8 owns only the **signed rendering of a sum strictly less than zero**: how a negative sum is distinguished from zero and from positive sums, how the sign is encoded and positioned, and the invariants that make a negative result unambiguous, round-trippable, and locale-independent.

Requirements

Functional Requirements (FR)

ID	Requirement	Priority
----	-------------	----------

---	---	---
-----	-----	-----

FR-176	The program MUST detect that the computed sum <code>S</code> is strictly less than zero and render <code>S</code> as a signed value: a single leading <code>-</code> (U+002D HYPHEN-MINUS, byte <code>0x2D</code>) followed by the base-10 magnitude <code> S </code> . <code>add 2 -5</code> MUST emit <code>-3</code> . must-have	
--------	--	--

FR-177	The sign MUST be emitted as ASCII <code>-</code> (U+002D (byte <code>0x2D</code>)). U+2212 MINUS SIGN, U+2010/U+2011 hyphens, en dash U+2013, em dash U+2014, the fullwidth minus U+FF0D, or any digit-glyph substitution MUST NOT be used. must-have	
--------	--	--

FR-178	The sign MUST appear exactly once, at byte offset 0 of the rendered value, immediately followed by the first significant digit. Duplicate signs, separated signs, and bracketed forms MUST NOT appear: <code>--3</code> , <code>- 3</code> , <code>(-3)</code> , <code>-3-</code> , <code>3-</code> are all forbidden. must-have	
--------	--	--

FR-179	The magnitude portion of a negative result MUST contain no leading zeros: the rendered line MUST match <code>^[1-9][0-9]*\n\$</code> for <code>S < 0</code> . <code>add -7 -1</code> MUST emit <code>-8</code> , never <code>-08</code> . must-have	
--------	--	--

FR-180	The sign of the rendered result MUST be derived solely from the exact mathematical sum, never from operand order and never from the sign of the first operand. <code>add -5 2</code> and <code>add 2 -5</code> MUST both emit <code>-3</code> . must-have	
--------	---	--

FR-181	The program MUST NEVER emit <code>-0</code> . Whenever the exact sum is zero — including <code>add -5 5</code> , <code>add 5 -5</code> , <code>add -0 0</code> , <code>add 0 -0</code> , <code>add -0 -0</code> — stdout MUST be exactly <code>0</code> . Zero carries no sign in this interface. must-have	
--------	---	--

FR-182	A rendered negative result MUST itself be a valid operand under the F-2 grammar (<code>-</code> followed by one or more ASCII decimal digits, nothing else) so that output can be consumed as input: <code>add \$(add 2 -5) 5</code> MUST yield <code>2</code> . must-have	
--------	---	--

FR-183	For a negative sum <code>S</code> , the rendered magnitude MUST equal the decimal expansion of <code> S </code> exactly, such that re-parsing the emitted line as an integer yields exactly <code>S</code> — with no digits dropped, prepended, or fabricated. must-have	
--------	--	--

FR-184	Magnitude rendering of a negative result MUST use the same arbitrary-precision, base-10 path as a non-negative result: no truncation, saturation, wrapping, or sign-magnitude width limit (32-bit, 64-bit, or otherwise) may apply. must-have	
--------	---	--

FR-185	On a successful negative result the program MUST write exactly one complete line <code><magnitude>\n</code> to stdout, write zero bytes to stderr, and terminate with exit status <code>0</code> . must-have	
--------	--	--

| FR-186 | A negative result MUST NOT be treated as an error condition: it MUST NOT raise, MUST NOT set a non-zero exit status, MUST NOT suppress stdout, and MUST NOT emit any diagnostic. | must-have |

| FR-187 | The sign MUST survive the full parse → add → render → write path as a literal byte: output MUST be ASCII/UTF-8 compatible on stdout whether stdout is a TTY, a regular file, or a pipe, with no BOM and no encoding preamble. | must-have |

| FR-188 | Sign classification MUST be performed by comparing the exact sum against zero and MUST be correct at arbitrary magnitude, including when operands are shaped so that a naive sign-bit or floating-point classification would be wrong by many orders of magnitude. | must-have |

Non-Functional Requirements (NFR)

ID	Target	Measurement	Priority

| NFR-106 | **Performance:** the negative-result path adds no measurable overhead versus the non-negative path; ``add -999999999999 1`` completes in < 50 ms wall clock on reference hardware. | Timed loop of 1000 invocations, median and p95 wall clock; compare against an equivalent non-negative invocation. | must-have |

| NFR-107 | **Scalability (magnitude):** correct sign and full magnitude for operands of at least 10,000 decimal digits each that yield a negative sum, in < 500 ms per invocation. | Generate maximal-size operands, invoke, byte-compare against a reference big-integer computation. | must-have |

| NFR-108 | **Availability/robustness:** 100% of syntactically valid negative-result invocations exit ``0`` with a complete line; zero crashes across the full generated operand space (see ``T-1`–`T-7``). | Exhaustive small-range sweep plus randomized large-magnitude fuzzing; assert exit status and stdout shape on every case. | must-have |

| NFR-109 | **Security:** the sign is emitted as inert data only — never executed, never interpolated into a shell command, never used to build a filename, path, or option string. A leading ``-`` in `*output*` MUST NOT be interpreted by the program as an option. | Code inspection for any `exec/eval/shell/path` construction involving output; confirm the output byte is written by a literal write to stdout. | must-have |

| NFR-110 | **Data/residency:** no persistence, no temp files, no network, no writes outside process memory plus the stdout/stderr streams. Negative values MUST NOT be cached, logged, or telemetered anywhere. | Filesystem and socket syscall observation (e.g., ``strace``) around a negative-result run; assert no file/socket activity. | must-have |

| NFR-111 | **Deployment/environment:** identical byte-for-byte output under ``LC_ALL=C``, ``LC_ALL=de_DE.UTF-8``, and ``LC_ALL=ar_EG.UTF-8`` — no locale-specific minus sign, digit shaping, decimal separator, or group separator. | Byte-compare stdout across the three locale settings for the same invocation. | must-have |

| NFR-112 | **Portability:** output is pure ASCII on POSIX and on Windows; the terminating sequence is ``\n`` (byte ``0x0A``) on both, never ``\r\n``. | Byte-compare stdout hexdump on both platforms. | must-have |

| NFR-113 | **Determinism:** identical operands always produce byte-identical stdout, stderr, and exit status across repeated runs and across process restarts. | 100 repeated invocations of the same pair; assert identical stdout/stderr bytes and status. | must-have |

| NFR-114 | **Observability:** a successful negative-result run produces an empty stderr stream; no progress, warning, or debug output is emitted on any channel other than the single stdout line. | Assert stderr length is exactly 0 bytes for successful runs, including very large magnitudes. | must-have |

User Stories (US)

ID	Story	Priority

V-3	No ``0`` is ever emitted.	Byte-compare stdout to ``b"0\n"`` for all zero-sum operand pairs, including ``-0`/`0`` operand spellings.	
V-4	Output shape matches ``^-(0	[1-9][0-9]*)\n$``.	Regex assertion over stdout of every test case in ``T-1`-`T-7``.
V-5	Round-trip: the emitted line, re-parsed by the F-2 grammar, equals the reference sum exactly.	Feed stdout back as an operand to a second invocation; assert the composed result.	
V-6	Arbitrary precision holds: a ≥ 1000 -digit negative result matches a reference big-integer computation digit-for-digit.	Differential test against a reference arbitrary-precision computation.	
V-7	Locale independence.	Run under ``LC_ALL=C``, ``de_DE.UTF-8``, ``ar_EG.UTF-8``; byte-compare stdout.	
V-8	Operand-order independence.	For each pair ``(a, b)``, byte-compare ``add a b`` against ``add b a``.	
V-9	stderr is empty and exit status is ``0`` on every negative result.	Capture both streams and the status; assert ``stderr == b""`` and ``status == 0``.	

Edge cases

ID	Case	Required outcome
EC-1	``add -5 5`` reduces to zero.	stdout ``0\n``, exit ``0``, stderr empty.
EC-2	``add 5 -5`` reduces to zero from the other order.	stdout ``0\n``, exit ``0``, stderr empty.
EC-3	``add -0 0``, ``add 0 -0``, ``add -0 -0`` (operands spelled with a negative zero).	stdout ``0\n``; ``-0`` MUST NOT appear.
EC-4	``add 0 -7`` — first operand is zero, result is negative.	stdout ``-7\n``; the sign comes from the result, not the leading operand.
EC-5	``add -000123 1`` — operand has leading zeros and a sign.	stdout ``-122\n``; operand formatting does not leak into output.
EC-6	``add -10000000000000000000 9999999999999999999`` — near-total cancellation beyond 64-bit range.	stdout ``-1\n``; no overflow or wrap.
EC-7	``add 2 -5`` and ``add -5 2``.	Both emit the byte-identical line ``-3\n``.
EC-8	``add -1 0`` and ``add 0 -9223372036854775808`` (magnitude at/over signed 64-bit boundary).	stdout ``-1\n`` and the exact negative value; no clamping.
EC-9	Very long negative result ($\geq 10,000$ digits).	Full magnitude written, correct single sign, exactly one terminator.
EC-10	stdout redirected to a file or a pipe.	Output byte-identical to the TTY case; byte ``0x2D`` present; no BOM; no CRLF.
EC-11	Locale set to a UTF-8 locale (``de_DE.UTF-8``, ``ar_EG.UTF-8``).	Output byte-identical to ``LC_ALL=C``; no U+2212, no shaped digits, no grouping separators.
EC-12	Operand ``-`` alone, ``--5``, ``- 3``, ``-3.0``, ``-1e5``, ``- 000``.	Handled by F-2/F-6: single-line stderr diagnostic, empty stdout, exit ``2``. The negative-result rendering path MUST NOT run.
EC-13	``add -2147483648 -1`` and other combinations crossing common 32-bit limits.	Exact negative result; no wraparound.
EC-14	Both operands negative with equal magnitudes and opposite signs in a chain.	Result sign is recomputed from the arithmetic each time; never sticky.

Error handling

ID	Condition	Required behaviour
EH-1	The sum is negative.	****Not an error.**** stdout is the single signed line, stderr empty, exit status ``0``. No warning, no code path that raises.

| EH-2 | An operand that merely *looks* negative (``-``, ``--3``, ``- 3``, ``-3.0``, ``-1e5``) is supplied. | Rejected by the F-2 grammar via the F-6 pipeline: exactly one stderr diagnostic line, zero stdout bytes, exit status ``2``. No partial negative rendering. |

| EH-3 | Writing the negative result to stdout fails (closed/full descriptor, I/O error). | The process **MUST** NOT exit ``0`` (an exit status of ``0`` is reserved for a run that wrote the complete ``-<magnitude>\n`` line per F-5); a single-line diagnostic goes to stderr when stderr is writable. |

| EH-4 | The write is interrupted (e.g., broken pipe) mid-line. | No fabricated terminator and no fabricated digits are appended; a truncated prefix **MUST NOT** be completed into a well-formed line. |

| EH-5 | Any unexpected internal failure on the negative-result path. | No unhandled traceback escapes: a single-line diagnostic to stderr and a non-zero exit status; stdout **MUST NOT** carry a line that could be mistaken for a valid result. |

Acceptance criteria

| ID | Given / When / Then |

|---|---|

| AC-1 | ****Given**** operands ``2`` and ``-5``, ****when**** the program runs, ****then**** stdout is exactly the 3 bytes ``-3\n``, stderr is empty, and exit status is ``0``. |

| AC-2 | ****Given**** operands ``-5`` and ``2`` (order reversed), ****when**** the program runs, ****then**** stdout is byte-identical to AC-1. |

| AC-3 | ****Given**** operands ``-5`` and ``5``, ****when**** the program runs, ****then**** stdout is exactly ``0\n`` — never ``-0\n``. |

| AC-4 | ****Given**** any pair of operands whose exact sum is negative, ****when**** the program runs, ****then**** the first stdout byte is ``0x2D`` and the remainder matches ``[0-9]+\n`` with no leading zero in the magnitude. |

| AC-5 | ****Given**** the output ``-3\n`` from a prior invocation, ****when**** it is supplied as an operand together with ``5``, ****then**** the composed result is ``2\n`` (the sign is parseable by the same grammar). |

| AC-6 | ****Given**** operands ``-1000000000000000000000`` and ``99999999999999999999``, ****when**** the program runs, ****then**** stdout is exactly ``-1\n`` with no overflow. |

| AC-7 | ****Given**** the same negative-result invocation run under ``LC_ALL=C``, ``de_DE.UTF-8``, and ``ar_EG.UTF-8``, ****when**** outputs are compared, ****then**** all three stdout streams are byte-identical and contain only ASCII ``0x2D`` as the sign. |

| AC-8 | ****Given**** the operand ``-`` (bare sign) or ``-3.0``, ****when**** the program runs, ****then**** stdout is empty, stderr contains exactly one diagnostic line, and exit status is ``2`` — no negative-result line is produced. |

API behaviour

The interface is the process itself; there is no library, network, or file API.

| ID | Contract element | Specification |

|---|---|---|

| API-1 | Invocation | ``add <int1> <int2>`` — two positional operands, no options or flags (as fixed by F-1/F-2). |

| API-2 | Output channel | On a negative sum, stdout carries exactly one line: ``0x2D`` + ASCII decimal digits of ``\|S\|`` (no leading zeros) + ``0x0A``. Total stdout length = ``1 + digits(\|S\|) + 1`` bytes. No trailing spaces, no extra blank line. |

| API-3 | Error channel | Empty (0 bytes) on every successful negative-result run. |

| API-4 | Exit status | ``0`` for a negative result, identical to the positive and zero cases; the sign of the result never selects a different status (status values are owned by F-5). |

| API-5 | Encoding | ASCII / UTF-8-safe, platform-independent: byte ``0x2D``, digit bytes ``0x30``–``0x39``, terminator ``0x0A``. No BOM, no CRLF translation on any platform. |

Test cases

Priority

Open questions

ID Question Owner
--- --- ---
OQ-1 The brief specifies only "prints the sum". This spec assumes a result of zero is always rendered as `0` and never `-0`; please confirm that no signed-zero representation is expected. USER

| OQ-2 | The brief does not mention any option to suppress or alter the sign (e.g., printing an absolute value, '+' prefixes for positive results, or colour). This spec treats sign handling as unconditional and non-configurable; confirm that no such switch is required. | USER |

| OQ-3 | The brief does not state an expected behaviour when stdout cannot be written. This spec assumes a failed write is never a successful run (no exit `0`). Confirm that no alternative (e.g., best-effort partial output with exit `0`) is desired. | USER |

F-9: Arbitrary-precision output

Feature ID: F-9

Depends on: F-3 (integer addition), F-4 (print result to stdout), F-5 (exit-status contract), F-8 (correct handling of negative results). F-9 does **not** re-define argument handling (F-1), the operand grammar (F-2), the arithmetic itself (F-3), the stdout envelope (F-4), any exit-status value (F-5), diagnostic text (F-6), the dependency envelope (F-7), or the negative-sign encoding (F-8). F-9 owns only the **scale envelope of the emitted number**: the guarantee that the base-10 rendering of the sum `S` is exact and effectively unbounded for arbitrarily large magnitudes, and the invariants that keep that rendering machine-, locale-, and configuration-independent.

Requirements

Functional Requirements (FR)

| ID | Requirement | Priority |

|---|---|---|

| FR-201 | The program **MUST** render the sum `S` in base 10 for any magnitude that fits in available process memory. The number of emitted digits **MUST** equal the exact digit count of `S`; there **MUST** be no fixed-width cap (no 16/19/32/64/128-bit or any other fixed width) applied to the output. | must-have |

| FR-202 | The emitted digit count **MUST** be exactly `D = 1` when `S == 0`, else `len(str(abs(S)))`, and **MUST** be independent of the host platform's word size, CPU architecture, or any native C integer width. | must-have |

| FR-203 | The rendering **MUST NOT** truncate, round, saturate, clamp to a machine maximum, wrap around, or substitute any sentinel (`inf`, `nan`, `overflow`, `...`) for any value. Every accepted operand pair that could produce a large sum **MUST** produce that sum exactly. | must-have |

| FR-204 | The rendering **MUST NOT** contain digit grouping, thousands separators, underscores, spaces, decimal points, exponent/scientific notation, radix prefixes, or any character outside ASCII `0`–`9` (U+0030–U+0039), plus at most one leading `-` (U+002D) when `S < 0` as owned by F-8. | must-have |

| FR-205 | The rendering **MUST** be deterministic: for a fixed `(int1, int2)` accepted under the F-2 grammar, the byte sequence written to stdout **MUST** be identical on every run, every host, every platform, and every locale. | must-have |

| FR-206 | Round-trip fidelity: parsing the emitted output token under the F-2 operand grammar **MUST** yield an integer exactly equal to `S`. | must-have |

| FR-207 | The render path **MUST NOT** convert `S` through a binary floating-point or decimal floating-point value at any point. | must-have |

| FR-208 | The program **MUST NOT** impose an application-level maximum on operand magnitude or result magnitude. The only bound is the interpreter's available memory; the tool **MUST NOT** advertise, enforce, or document a finite maximum result size. | must-have |

| FR-209 | The rendering **MUST** be emitted as a single contiguous token with no interior whitespace or newline, terminated by exactly one `\n` (per F-4). It **MUST NOT** wrap or insert line breaks at any column width. | must-have |

| FR-210 | The program MUST emit the exact digit count for sums spanning many orders of magnitude greater than any native integer type on the host. | must-have |

| FR-211 | The output path MUST treat small and arbitrarily large sums through the same rendering logic. There MUST be no special-cased fast path that produces a structurally different textual form for values below any threshold. | must-have |

| FR-212 | On CPython 3.11+ (which caps integer→decimal-string conversion at a default of 4300 digits), the program MUST produce the correct decimal rendering for results with more than 4300 digits without surfacing the interpreter's integer-string conversion limit as a user-visible error for otherwise-valid input. The interpreter's default cap is a host artifact, not a contract limit of this tool. | must-have |

Non-Functional Requirements (NFR)

| ID | Category | Requirement | Target / Measurement |

|---|---|---|---|

| NFR-121 | Performance | Rendering a sum of ``n`` decimal digits MUST be linear-time, ``O(n)``, in the digit string — no quadratic concatenation loops. | A 1,000,000-digit sum renders in < 2 s on a modern laptop (should-have nuance within this feature; ``O(n)`` is must-have). Measured by wall-clock timing on a synthetic large sum. |

| NFR-122 | Scalability / availability | The program MUST remain a single ephemeral, stateless process per invocation: no daemon, no server, no persistence. Each invocation is independent; large consecutive sums MUST NOT degrade across invocations. | N repeated invocations of large-sum cases, each exit 0, no cross-run state. |

| NFR-123 | Memory | Peak additional memory for rendering MUST be ``O(n)`` in output digit count ``n``, holding no more than a small constant number of full copies of the digit string. | Memory profiling of a large-sum invocation scales linearly, not quadratically. |

| NFR-124 | Security | The render path MUST NOT ``eval``, ``exec``, shell-substitute, ``format``, or template the sum, the digit string, or any user-controlled value. Output is data and MUST NEVER be interpreted as code or as a format string. | Static inspection of the render path; no user-controlled format string reachable. |

| NFR-125 | Security (resource exhaustion) | The program MUST NOT present a truncated or partial digit token as a successful result. Stdout emitted under a success status MUST be either the complete correct line or empty. | Failure-injection / very-large-input tests assert stdout is complete-on-success and empty-on-failure. |

| NFR-126 | Data / residency | The program MUST store no data at rest, write no files, open no network sockets, and transmit no bytes except the stdout/stderr streams defined by F-1/F-4/F-6. The sum and its rendering MUST be confined to process memory for the duration of the invocation. |

Filesystem/network syscall tracing shows no writes/sockets; no temporary files. |

| NFR-127 | Deployment / environment | Arbitrary-precision output MUST behave identically on any CPython ≥ 3.8 , on any OS, regardless of ``PYTHONINTMAXSTRDIGITS``, ``LANG``, ``LC_NUMERIC``, ``LC_ALL``, or any other environment setting. No environment variable may be **required** to enable correct large-sum output. | Byte-identity of stdout across environment permutations. |

| NFR-128 | Data integrity / portability | The rendered digit string MUST be byte-for-byte identical whether the process runs on a 32-bit or 64-bit host and on big- or little-endian architectures. |

Cross-platform comparison of captured stdout bytes (where hosts are available). |

Behaviour

- F-9 consumes the exact sum ``S`` produced by F-3 and the canonical rendering rules owned by F-4 (base 10, no leading zeros, single trailing ``\n``) and F-8 (leading ``-`` for ``S < 0``). It adds no new arithmetic and no new sign rules.

- For every accepted input, F-9 emits exactly one token: ``0` · `[1-9][0-9]*` · `[1-9][0-9]*``, immediately followed by one ``\n``.
- The emitted token's digit count is the exact mathematical digit count of ``S``; leading zeros present in the **operands** (F-2) never survive into the output.
- Rendering is a pure, total function of ``S``: same ``S`` → same bytes, independent of operands' textual form, host, locale, terminal, or environment.
- The output path is uniform: values in the machine-int range and values far beyond it pass through the same formatting logic; no threshold-based special case exists.
- The interpreter's own integer-string conversion ceiling (CPython 3.11+ default 4300 digits) is treated as a host constraint to be neutralized, not as an error to be surfaced for valid input.

Business rules

- ****BR-1:**** The output value is exactly ``S = int1 + int2`` as defined by F-3; F-9 changes neither the value nor the arithmetic.
- ****BR-2:**** Exactly one canonical textual form exists per integer value: optional leading ``-`` (F-8) followed by the most-significant-first decimal digits, with no leading zeros except that the value zero is the single character ``0``.
- ****BR-3:**** The output stream is a single logical line: ``[ - ] digit+`` then exactly one ``\n``. Nothing precedes and nothing follows it.
- ****BR-4:**** The output digit count is a pure function of ``S``; it MUST NOT depend on the textual length of the operands (operand leading zeros do not survive).
- ****BR-5:**** "Arbitrary precision" is a contract, not a best-effort: the tool claims no maximum result size; the only bound is available process memory / interpreter capability.
- ****BR-6:**** Output bytes are data. No locale, encoding, terminal width, or environment variable may influence them.
- ****BR-7:**** The interpreter's integer→string digit cap is a host artifact outside this tool's contract; for otherwise-valid input it MUST NOT be surfaced to the user as an error.

Validation

- ****V-1:**** Emitted stdout MUST match the regular expression `^-[0-9]+\n$``, with the leading ``-`` present only when ``S < 0``.
- ****V-2:**** No-leading-zero check: the token MUST match `^0\n$``, `^[1-9][0-9]*\n$``, or `^-[1-9][0-9]*\n$``.
- ****V-3:**** Round-trip check: re-parsing the emitted token under the F-2 grammar yields a value equal to ``S``.
- ****V-4:**** Digit-count check: the number of digit characters equals ``D`` per FR-202.
- ****V-5:**** Determinism check: two runs, and any two platforms, produce identical stdout bytes.
- ****V-6:**** Large-magnitude check: a sum exceeding the CPython 3.11+ default integer-string conversion cap (> 4300 digits) renders fully and exits 0 with no ``ValueError`` surfaced.

Edge cases

- ****EC-1:**** ``S == 0`` → ``0\n`` (never ``-0\n``, never ``+0\n``).
- ****EC-2:**** Large-magnitude negative ``S`` → ``-`` followed by the full magnitude digits (per F-8), with no overflow marker.
- ****EC-3:**** Sums whose digit count is just below, exactly at, and just above the interpreter's default integer-string conversion cap (4299 / 4300 / 4301 digits on CPython 3.11+) — all must render exactly.
- ****EC-4:**** Operands supplied with leading zeros (e.g., ``000007 000000``) → canonical output with leading zeros dropped.
- ****EC-5:**** Both operands zero, or cancellation to zero → ``0\n``.

- **EC-6:** Extremely large operands (e.g., 100,000+ digits each) → exact output, bounded only by process memory.
- **EC-7:** Sum whose magnitude equals a power of ten (e.g., $999\dots9 + 1$) → digit count increases by exactly one; no digit loss or off-by-one at the boundary.
- **EC-8:** Locale configured to group digits differently (`LC_NUMERIC`) → output unchanged, no separators.
- **EC-9:** 32-bit vs 64-bit host → identical digit count and identical bytes.
- **EC-10:** Very large negative sum combined with F-8 sign encoding → single `-``, no double sign, no space between sign and digits.

Error handling

- **EH-1:** If the interpreter reports its integer-string conversion limit for an otherwise-valid sum, the program **MUST NOT** surface a partial result and **MUST NOT** claim success with a truncated line. It either completes the correct rendering or fails through the F-5/F-6 failure channel — never a truncated token under exit status `0``.
- **EH-2:** If memory is exhausted during rendering, stdout **MUST** be either the complete correct line or empty; a partial digit token **MUST NEVER** be terminated by `\n`` under a success status.
- **EH-3:** Any rendering-time failure **MUST** report to stderr in the diagnostic style owned by F-6; stdout **MUST** remain byte-clean on any failure (consistent with the F-6 validation pipeline).
- **EH-4:** `0`` **MUST NOT** be emitted under any circumstance; zero is always unsigned.
- **EH-5:** No placeholder token (`N/A``, `?``, `-``, `overflow``, `inf``, `nan``, empty line) may ever be emitted in place of a numeric result.

Acceptance criteria

- **AC-1 (FR-201):** ``add 9223372036854775807 1`` → stdout `9223372036854775808\n``, exit `0`` (correctly exceeds the signed 64-bit maximum).
- **AC-2 (FR-203):** Sums exceeding the 64-bit range render exactly, with no saturation to `9223372036854775807`` and no wrap-around to a negative value.
- **AC-3 (FR-202, FR-210):** For a sum with a known digit count `k`` ( $> 1000$ ), stdout contains exactly `k`` digit characters, independent of host word size.
- **AC-4 (FR-204, FR-209):** stdout matches `^[0-9]+\n$`` and contains no ``,``, `_``, space, ``,``, `e`/`E``, `+``, or wrapping newline.
- **AC-5 (FR-206):** Feeding the emitted output back through the F-2 grammar yields a value equal to `S`` (e.g., ``add $(add 2 3) 0`` → `5\n``).
- **AC-6 (FR-205, NFR-127):** The same invocation under differing `LANG``, `LC_ALL``, and `PYTHONINTMAXSTRDIGITS`` values produces byte-identical stdout.
- **AC-7 (FR-212):** On CPython 3.11+, adding two operands whose sum has more than 4300 digits succeeds with exit `0`` and the full digits on stdout, with no `ValueError`` surfaced to the user.
- **AC-8 (FR-209):** stdout is exactly one line terminated by exactly one `\n``, with no interior whitespace and no column wrapping.
- **AC-9 (FR-211):** A small sum and a very large sum use the same code path and produce structurally identical output forms (only the digit count differs).
- **AC-10 (NFR-123):** Peak memory during a large-sum render scales linearly with digit count; the program does not retain unnecessary duplicate copies of the digit string.
- **AC-11 (FR-201):** ``add 999 1`` → `1000\n`` (not `01000\n``); the digit count increases by exactly one at the power-of-ten boundary.
- **AC-12 (NFR-121):** A 1,000,000-digit sum renders within the NFR-121 time budget, confirming `O(n)`` behaviour.

API behaviour

> The program's API surface is its command-line process contract (owned by F-1/F-4/F-5); F-9 specifies only the *shape* and *scale* of the successful output.

- **API-1:** Program ``add`` (F-1). On success, stdout carries exactly one decimal token per F-4/F-9; stderr is empty; exit status is ``0`` (status value owned by F-5).
- **API-2:** ``add <a> <b>`` where the sum has ``n`` digits → stdout carries exactly ``n`` digit characters, plus an optional leading ``-`` (F-8), plus one trailing ``\n``. No fixed maximum ``n``.
- **API-3:** The emitted token is a valid operand under the F-2 grammar (optional ``+`/`-``, ASCII digits, no separators), enabling round-trip composition.
- **API-4:** Rendering is pure: no files, no network, no environment writes, no side effects (NFR-126).
- **API-5:** Rendering is idempotent: the same input always yields the same output bytes (NFR-127 / FR-205).
- **API-6:** A consumer may capture the result via ``$(add a b)`` and receive the value with the trailing newline stripped, without needing to trim whitespace, separators, or notation.

Data model (DM)

- **DM-1:** Value type = mathematical integer (unbounded ■), represented in use by the interpreter's arbitrary-precision integer type.
- **DM-2:** Output representation = canonical decimal string matching ``-?\d+`` with no leading zeros except the single character ``0``.
- **DM-3:** No persisted record, no schema, no storage. The value exists only in-process during a single invocation.

Tests (T)

- **T-1:** 64-bit boundaries: ``9223372036854775807 + 1``; ``-9223372036854775808 + -1``.
- **T-2:** Power-of-ten boundary digit-count increase: ``999...9 + 1``.
- **T-3:** Cancellation to zero and ``0 + 0``.
- **T-4:** Operands with leading zeros.
- **T-5:** Digits just below / at / above the CPython 3.11+ integer-string cap (4299 / 4300 / 4301).
- **T-6:** Very large magnitudes ($10^{\blacksquare}$ – $10^{\blacksquare}$ digits) for memory and linear-time checks.
- **T-7:** Environment-permutation byte-identity (``LANG``, ``LC_ALL``, ``PYTHONINTMAXSTRDIGITS``).
- **T-8:** 32-bit vs 64-bit host digit-count parity (where hosts are available).
- **T-9:** Round-trip parse of emitted output under the F-2 grammar.

Open questions (OQ)

- **OQ-1:** Is there an intended *practical* upper bound for results (e.g., guaranteeing 1,000,000-digit output within a stated time), or should the tool be documented as memory-bound only? — Needs USER confirmation; not a reason to reduce scope.
- **OQ-2:** Should the tool guarantee successful output for *any* magnitude regardless of the interpreter's integer-string conversion limit, or is exceeding the interpreter default (4300 digits on CPython 3.11+) acceptable to reject? — Needs USER confirmation (F-9 assumes the guarantee).
- **OQ-3:** If a user sets ``PYTHONINTMAXSTRDIGITS`` to a low value, should the tool's output remain identical to the default configuration (i.e., override the user's environment)? — Needs USER confirmation (F-9 assumes identical behaviour).

Priority

****must-have.**** Arbitrary-precision output is the property that makes the tool's result trustworthy for any accepted input; the critical, non-obvious risk is the CPython 3.11+ integer→string conversion cap (FR-212), which would otherwise turn a mathematically valid sum into a user-visible error. All FRs above are must-have; the concrete million-digit performance budget in NFR-121 is the only "should-have" nuance within the feature.

F-10: Usage/help output

****Feature ID:**** F-10

****Depends on:**** F-1 (``add`` command entry point — owns arity rule and the fact that arity violations go to stderr with status 2), F-2 (two positional integer operands — owns the operand grammar, and the rule that no flags are recognized), F-5 (exit-status contract — owns the numeric status ``2``), F-6 (input validation and error message — owns the error line that precedes the usage line, the check ordering, and the safe echoing rules), F-7 (standard library only).

****Boundary note:**** F-10 owns ****only the usage/help text surface**** — its exact literal content, byte-level formatting, channel, emission count, terminators, and invariants (determinism, no runtime interpolation, no localization, no flag path). F-10 does ****not**** redefine arity (F-1), the operand grammar (F-2), arithmetic (F-3), stdout rendering (F-4), exit-status values (F-5), or the diagnostic error line and check pipeline (F-6). Where F-1/F-5/F-6 already fix a trigger or a status, F-10 references it and adds only the text/channel obligations.

Requirements

Functional Requirements (FR)

| ID | Requirement | Priority |

| ---|---|---|

| FR-226 | The program **MUST** emit the canonical usage line to ****stderr**** whenever the invocation's operand count is not exactly two (0, 1, or 3+ operands). The trigger condition and the exit status remain owned by F-1 ``FR-6`` and F-5 ``FR-102``; F-10 owns only the text and channel of the usage line. | must-have |

| FR-227 | The canonical usage line **MUST** be exactly the literal ``usage: add <int1> <int2>`` followed by one ``\n`` (U+000A): 24 text bytes + 1 LF = ****25 bytes**** total. No other wording, casing, spacing, punctuation, or ordering is permitted. | must-have |

| FR-228 | Byte-exact formatting **MUST** hold: lowercase ASCII ``usage``, then ``:`` (U+003A), then exactly one space (U+0020), then the literal program name ``add``, then one space, then the literal placeholder ``<int1>``, then one space, then the literal placeholder ``<int2>``. There **MUST** be no trailing whitespace, no tab (U+0009), no CR (U+000D), no NUL (U+0000), no ANSI/SGR escape (U+001B), no color, and no bold/underline attributes. | must-have |

| FR-229 | The usage line **MUST** be written to ****stderr (fd 2) only****. It **MUST NOT** be written to stdout, **MUST NOT** be mirrored to stdout, and **MUST NOT** be concatenated onto the numeric result line. | must-have |

| FR-230 | Every invocation that emits the usage line **MUST** terminate with exit status ``2`` and **MUST** write ****0 bytes**** to stdout. The status value itself is owned by F-5 (``FR-102``); F-10 asserts only that emitting usage text and exiting ``0`` are mutually exclusive. | must-have |

| FR-231 | The program **MUST NOT** recognize any help/usage token (``-h``, ``--help``, ``-help``, ``--usage``, ``-?``, ``/?``, ``help``, ``--version``) as a flag or subcommand. All such tokens **MUST** be treated as ordinary operands and handled by F-1 arity rules or F-2 grammar rules. There is no exit-0 help path. | must-have |

|

| FR-232 | The usage line MUST be emitted **at most once** per invocation. No invocation may print it twice, and there MUST be no alternative variants (no per-arity variant, no per-operand variant, no "short help" vs "long help"). Arity 0, arity 1, and arity 3+ MUST all produce the identical single line. | must-have |

| FR-233 | The usage line MUST be byte-identical across runs and environments: independent of locale ('LANG', 'LC_ALL', 'LC_MESSAGES'), terminal ('TERM', 'COLUMNS', TTY vs pipe vs file), 'NO_COLOR'/'FORCE_COLOR'/'CLICOLOR', current working directory, 'argv[0]' spelling, environment size, and platform. The literal MUST NOT be sourced from gettext catalogs, config files, or environment variables. | must-have |

| FR-234 | The usage line MUST NOT be emitted on any successful run (exactly two operands satisfying the F-2 grammar); successful runs MUST produce empty stderr. | must-have |

| FR-235 | The usage line MUST NOT be emitted on operand-grammar failures (arity = 2, any operand rejected by F-2) or on any computation path. Those paths emit only the single-line diagnostics owned by F-1 'FR-7' / F-6 'FR-126'-'FR-127'. | must-have |

| FR-236 | The usage line MUST be the **final** stderr line of the failing invocation and MUST be terminated by its '\n'. Nothing may be written to stderr after it, and the line MUST NOT be truncated or left unterminated on any normal exit path. | must-have |

| FR-237 | The usage line MUST NOT interpolate any runtime value: no operand echo, no argument count, no program path or 'argv[0]', no hostname, no username, no PID, no timestamp, no environment value. It is a fixed constant string (the program name is the literal 'add', never a basename/path derived from 'argv[0]'). | must-have |

| FR-238 | If stderr is closed or unwritable (e.g., '2>&-', full disk on a redirected stderr), the program MUST NOT redirect the usage line or any diagnostic to stdout, MUST NOT write usage text to any file, and MUST still terminate with exit status '2' with an empty stdout. | should-have |

| FR-239 | The usage line SHOULD be issued as a **single write** of the complete 25-byte sequence so that concurrent writers on the same stderr file or pipe cannot interleave partial bytes into the middle of the line. | should-have |

Non-Functional Requirements (NFR)

| ID | Requirement | Target | Measurement | Priority |

|---|---|---|---|---|

| NFR-136 | Usage-path latency | The complete usage line must reach stderr within **50 ms** of process start on reference hardware, with no I/O beyond the stderr write(s) for the failing invocation | Wall-clock timing of 'add' (no args) over 50 runs; median and p95 | should-have |

| NFR-137 | Scalability / availability of the usage path | Usage emission is O(1) in the number of arguments and independent of operand length: even 10 000 extra arguments must yield exactly one usage line and exit 2 within **100 ms** and with no additional stderr lines from F-10 | Invoke with 10 000 arguments; assert one usage line, status 2, bounded wall clock | should-have |

| NFR-138 | Security of the help surface | The usage line must never echo attacker-controlled bytes, so it cannot be used for terminal-escape injection, log injection, or forged-log-line injection; no environment variable may be interpolated into it | Fuzz operand bytes (ESC sequences, newlines, '\r', NUL-adjacent, unicode bidi controls) in arity-failure invocations; assert the usage line is byte-constant and contains no ESC (0x1B) | must-have |

| NFR-139 | Data / residency / side-effect freedom | Generating usage text must collect no telemetry, write no files, create no temp files, open no sockets, and read no config or data files; the only observable side effect is the stderr write | Run in a sandbox with read-only FS and no network; assert no file/socket syscalls attributable to usage emission | must-have |

| NFR-140 | Deployment / environment portability | The usage line must be byte-identical on any POSIX-style environment running stock CPython 3.8+ with a UTF-8 or ASCII-compatible locale; no terminal capability database, pager, or color support is required or consulted | Byte-compare stderr

across the supported environment matrix (see AC-10) | must-have |

User Stories (US)

| ID | Story | Refs | Priority |

|---|---|---|---|

| US-136 | As a first-time user who mistyped the command, I want a one-line usage message that shows the exact expected invocation, so that I can correct my command without leaving the terminal or reading documentation. | FR-226, FR-227 | must-have |

| US-137 | As a shell-script author, I want usage failures to go to stderr with a non-zero exit status and nothing on stdout, so that I can distinguish misuse from success without parsing stdout. | FR-229, FR-230 | must-have |

| US-138 | As a CI/build-log reader, I want the usage text to be a single stable, uncolored, escape-free line, so that logs stay readable, greppable, and safe to render. | FR-228, FR-232, NFR-138 | should-have |

| US-139 | As a user on a non-English or differently configured machine, I want the same usage text everywhere, so that error triage is deterministic across environments and machines. | FR-233, NFR-140 | should-have |

| US-140 | As a tooling author integrating `add`, I want the absence of a help flag to be explicit and unambiguous (`add --help` fails with status 2 rather than succeeding), so that I never accidentally rely on a help surface that does not exist. | FR-231, FR-235 | should-have |

Behaviour

1. **Single trigger path.** Exactly one condition produces usage text: the operand count is not exactly two (0, 1, or 3+ operands). This condition is evaluated first, per the fixed pipeline owned by F-6 (FR-126), so usage text is emitted **before** any operand grammar check runs. A 3-argument invocation therefore never reports a grammar error and never emits a grammar-specific variant.
2. **Emission sequence (arity failure).** stderr receives: (a) the F-6 error line naming the expected operand count, then (b) the F-10 canonical usage line as the last line; then the process terminates with status `2`. stdout receives nothing.
3. **Non-emission sequence (grammar failure).** With exactly two operands where at least one fails the F-2 grammar, stderr receives **only** the single-line diagnostic owned by F-1 `FR-7` / F-6; the F-10 usage line is not emitted and the process terminates with status `2`.
4. **Success path.** Exactly two operands, both grammar-valid: the F-4 result line goes to stdout, stderr stays empty, exit status `0`. No usage text exists on this path.
5. **No help flag.** Because F-2 (FR-26) forbids option parsing, `-h`, `--help`, `--usage`, `--?`, `/?`, `help`, and `--version` are ordinary operands. `add --help` is a one-operand invocation → arity failure → usage line + status `2`. `add --help 1` is a two-operand invocation → `--help` fails the F-2 grammar → single diagnostic line, **no** usage line, status `2`. There is no invocation of `add` that prints help and exits `0`.
6. **Channel and format discipline.** The usage line is a fixed 25-byte constant on fd 2: no color, no bold, no cursor movement, no pagination, no pager invocation, no line wrapping at any terminal width, and no locale- or environment-dependent text.
7. **Idempotence.** A given invocation shape produces exactly one usage line, and repeated identical invocations produce byte-identical stderr (modulo the F-6 error line, which F-10 does not own).

Business rules

| ID | Rule |

|---|---|

| BR-1 | The canonical usage line is the fixed literal ``usage: add <int1> <int2>`` plus one ``\n`` — 24 text bytes + 1 LF = 25 bytes. It is a compile-time/program-literal constant, not assembled from runtime parts. |

| BR-2 | ``<int1>`` and ``<int2>`` are literal placeholder tokens including their angle brackets (U+003C, U+003E). They are never substituted with actual argument values, argument counts, or examples. |

| BR-3 | The program name printed in usage is the fixed ASCII literal ``add`` — never ``argv[0]``, never a path, never a basename, never a symlink name, never a ``python`/`python3`` script path. Invocation style does not change the text. |

| BR-4 | Exactly one line: no leading blank line, no trailing blank line, no multi-line "long help", no examples section, no options section, no version footer. |

| BR-5 | The usage line is always the final stderr line of the invocation; no byte is written to stderr after its ``\n``. |

| BR-6 | Usage text is an **error-path artifact only**. There is no success-path help, no ``--help`` success mode, and no exit-``0`` path that prints usage. |

| BR-7 | The text is English/ASCII and non-localized; it is not loaded from gettext catalogs, message files, config files, or environment variables. |

| BR-8 | The usage line is emitted at most once per process. Arity 0, arity 1, and arity 3+ share one and the same line — no variants exist. |

| BR-9 | The usage line interpolates nothing, so no operand bytes, path bytes, or environment bytes can reach the terminal through it; safe echoing of offending input (if any) on arity failures is F-6's responsibility, on the F-6 error line, not the usage line. |

| BR-10 | Usage emission never changes the exit-status contract: usage always accompanies status ``2`` (F-5 ``FR-102``), never ``0`` and never ``1``. |

Validation

Validation here means both (a) the gate that decides whether the usage line is emitted, and (b) the self-checks that the emitted text satisfies its own contract.

1. **Emission gate.** Compute operand count as ``len(argv) - 1``; emit usage **iff** that count $\neq 2$. This is the only gate; no environment value, TTY state, or operand content may suppress or trigger the line (BR-8, FR-226).
2. **Text validation.** The candidate string must byte-compare equal to ``b"usage: add <int1> <int2>\n"`` (length 25) before emission. Any deviation is a defect, not a formatting choice (FR-227, FR-228).
3. **Character-class validation.** Every emitted byte must be in the set ``{0x20, 0x3A, 0x3C, 0x3E, 0x0A}`` $\cup$ ASCII letters (``u,s,a,g,e,a,d``) $\cup$ ASCII digits (``1,2``). Any byte outside this set — notably ``0x1B`` (ESC), ``0x09`` (TAB), ``0x0D`` (CR), ``0x00`` (NUL) — is a violation (FR-228, NFR-138).
4. **Channel validation.** The write target must be fd 2. A run whose stdout byte count is non-zero while usage was emitted fails validation (FR-229, FR-230).
5. **Cardinality validation.** Count occurrences of the substring ``usage: add <int1> <int2>`` in captured stderr; the count must be exactly 1 for an arity failure and exactly 0 for grammar failures, computation paths, and successful runs (FR-232, FR-234, FR-235).
6. **Terminator validation.** The emitted stream must end with the usage line's ``\n``; the usage line must not be the only content preceding an unterminated write (FR-236).

Edge cases

| ID | Case | Required behaviour |

|---|---|---|

| EC-1 | Zero operands: ``add`` | Usage line emitted as final stderr line; stdout 0 bytes; exit 2. |

| EC-2 | One operand: ``add 5`` | Identical to EC-1. The operand's validity is irrelevant — grammar is not evaluated. |

| EC-3 | Three operands: `add 1 2 3` | Identical usage line; `6` is **not** printed anywhere; stdout 0 bytes; exit 2. |

| EC-4 | Help-flag-shaped single argument: `add --help` | Treated as one operand → arity failure → usage line; exit 2. No success/help path. |

| EC-5 | Help-flag-shaped plus operand: `add --help 1` | Two operands → `--help` rejected by the F-2 grammar → exactly one F-1 `FR-7`/F-6 diagnostic line, **no** usage line; exit 2. |

| EC-6 | Two invalid operands: `add -h -h` | Pipeline stops at the first operand (F-6 `FR-126`) → single diagnostic, no usage line; exit 2. |

| EC-7 | Closed stderr: `add 2>&-` | No usage bytes are observable; stdout remains 0 bytes; exit status is still 2; no fallback to stdout and no file fallback. |

| EC-8 | stderr redirected to a file: `add 1 2 3 2>err.txt` | `err.txt` contains the F-6 error line followed by exactly one 25-byte usage line; no ESC, no CR, no trailing blank line. |

| EC-9 | stdout redirected on a failing arity: `add >out.txt` | `out.txt` exists/truncated by the shell but the program writes **0** bytes to it (FR-230). |

| EC-10 | Hostile locale/terminal settings: `LC\_ALL=tr\_TR.UTF-8`, `LANG=de\_DE.UTF-8`, `COLUMNS=20`, `TERM=dumb`, `NO\_COLOR=` unset | Usage bytes are identical to the default environment; `usage:` is not capitalized, not translated, and not wrapped at 20 columns. |

| EC-11 | Alternate invocation shapes: `./add`, `/usr/local/bin/add`, symlink `sum`, `python3 add.py` | Usage line is byte-identical; the printed program name remains the literal `add` (BR-3). |

| EC-12 | Very many arguments: `add` followed by 10 000 tokens | Exactly one usage line, exit 2, bounded latency (NFR-137); no operand token appears in the usage line. |

| EC-13 | Non-UTF-8 / binary operand bytes on an arity failure (e.g., `add \$'\x1b[31m' 1 2`) | The usage line remains the constant 25-byte sequence and contains no ESC byte (NFR-138). |

| EC-14 | stderr is a TTY | No color, no bold, no cursor control sequences are emitted; bytes equal the non-TTY case (FR-228, FR-233). |

Error handling

1. **Usage emission is itself an error path.** It always travels with exit status `2`, always leaves stdout empty, and never replaces or follows a successful result (FR-230, BR-6, BR-10).
2. **No fallback channel.** On any stderr write failure or unavailability (closed fd, ENOSPC on a redirected stream), the program must not write usage text to stdout, must not create or write a file, and must not silently swallow the error such that the status becomes `0`; the mandatory observable is exit status `2` with empty stdout (FR-238).
3. **No traceback substitution.** A failure while writing usage must not cause the program to exit `1` (the exit-status contract has only `0` and `2`; F-5 owns the values). Diagnostics must not escape onto stdout under any circumstance.
4. **No double reporting.** A grammar failure or computation path must not additionally emit usage text; the arity-first pipeline (F-6 `FR-126`) guarantees the two diagnostic flavours never co-occur in the same invocation (FR-232, FR-235).
5. **No partial-line retry strategies.** The program must not attempt to work around a failed/truncated write by re-emitting the usage line, by switching streams, or by splitting/reordering output; the line is emitted once, as a single complete unit where possible (FR-236, FR-239).

Acceptance criteria

ID	Given / When / Then

| AC-1 | Given the program, when invoked as `add` with no arguments, then the last stderr line is exactly `usage: add <int1> <int2> + '\n`, stdout has 0 bytes, and exit status is 2. (FR-226, FR-227, FR-229, FR-230) |

| AC-2 | Given the program, when invoked as `add 7` (one argument), then output is byte-identical to AC-1. (FR-226) |

| AC-3 | Given the program, when invoked as `add 1 2 3` (three arguments), then output is byte-identical to AC-1 and the string `6` does not appear in stdout. (FR-226, FR-230, FR-232) |

| AC-4 | Given the program, when invoked as `add 2 3`, then stdout is exactly `5\n`, stderr has 0 bytes, exit status is 0, and the substring `usage:` appears nowhere. (FR-234) |

| AC-5 | Given the F-2 grammar, when invoked as `add --help`, then the last stderr line is the usage line, stdout has 0 bytes, and exit status is 2 (no exit-0 help). (FR-231) |

| AC-6 | Given the F-2 grammar, when invoked as `add --help 1`, then stderr contains exactly one line (the F-1 `FR-7`/F-6 diagnostic) that does **not** contain the substring `usage: add <int1> <int2>`, stdout has 0 bytes, exit status is 2. (FR-235) |

| AC-7 | Given captured stderr for an arity failure, then `tail -c 25` of stderr equals the byte sequence `75 73 61 67 65 3a 20 61 64 64 20 3c 69 6e 74 31 3e 20 3c 69 6e 74 32 3e 0a`. (FR-227, FR-228) |

| AC-8 | Given captured stderr for an arity failure, then it contains no `0x1B`, `0x09`, `0x0D`, or `0x00` byte, and no `0x20` byte immediately preceding the final `0x0A`. (FR-228, NFR-138) |

| AC-9 | Given 100 consecutive invocations of `add 1 2 3`, then the SHA-256 of captured stderr is identical across all runs. (FR-233) |

| AC-10 | Given the environment matrix `LANG=C`, `LANG=de\_DE.UTF-8`, `LC\_ALL=tr\_TR.UTF-8`, `COLUMNS=20`, stderr-as-TTY, stderr-as-pipe, stderr-as-file, then the final stderr line bytes are identical in every cell, including when `argv[0]` is a path or a symlink name. (FR-233, FR-237, NFR-140) |

| AC-11 | Given any arity failure, then the captured stderr contains exactly one occurrence of the substring `usage: add <int1> <int2>`, and the 24-character usage line contains none of the operands supplied on the command line. (FR-232, FR-237) |

| AC-12 | Given `add 2>&-` (stderr closed), then stdout has 0 bytes and exit status is 2 (no fallback output, no status other than 2). (FR-238) |

| AC-13 | Given `add > out.txt 2> err.txt` with a wrong argument count, then `out.txt` is 0 bytes and `err.txt` ends with exactly one usage line followed by `\n` and nothing else. (FR-229, FR-230, FR-236) |

| AC-14 | Given 10 000 arguments after the program name, then exactly one usage line is emitted, stdout has 0 bytes, exit status is 2, and wall-clock time is within the NFR-137 budget. (FR-232, NFR-137) |

| AC-15 | Given a sandbox with a read-only filesystem and no network, when an arity failure occurs, then the usage line is still emitted on stderr with exit status 2 and no file or socket is created. (NFR-139) |

API behaviour

| ID | Contract |

|---|---|

| API-1 | **Interface:** `add [operand ...]` → stdout (result line), stderr (diagnostic and/or usage), exit status (`0` on success, `2` on misuse). F-10 shapes only the stderr usage channel; it adds no new invocation mode, no new operand form, and no new flag. |

| API-2 | **Usage channel:** fd 2, exactly one 25-byte line (`usage: add <int1> <int2>\n`), emitted at most once, as the last stderr line, only on the arity-failure path. |

| API-3 | **Stream separation:** usage text and the numeric result are never both present on a single invocation and are never interleaved; result → fd 1, usage → fd 2, strictly. |

| API-4 | **Exit-status mapping:** usage emission ↔ status `2` (F-5 `FR-102`). There is no status value that pairs with usage other than `2`, and no usage text on the `0` path. |

| API-5 | **No success-mode help surface:** there is no `--help`/`-h`/`--version`/subcommand that prints usage and exits `0`; all such tokens are operands (FR-231). Adding a success-mode help surface would be new scope requiring user approval. |

| API-6 | ****No stdin interaction:**** the program does not read stdin, so piping input cannot trigger, suppress, or alter usage output; ``printf 'x' | add`` behaves exactly like ``add``. |
| API-7 | ****Machine consumption:**** the usage line is greppable/parseable by the fixed literal ``usage: add <int1> <int2>``; scripts may rely on the exact prefix ``usage: `` and the fixed 25-byte length. |

Priority

Priority	Items	Rationale
****must-have****	FR-226, FR-227, FR-228, FR-229, FR-230, FR-231, FR-232, FR-233, FR-234, FR-235, FR-236, FR-237; NFR-138, NFR-139, NFR-140; US-136, US-137	The usage line is the substance of the already-mandatory arity diagnostic (F-1 ``FR-6``), and its channel/format/determinism rules are what make that diagnostic trustworthy in scripts, logs, and hostile-input conditions.
****should-have****	FR-238, FR-239; NFR-136, NFR-137; US-138, US-139, US-140	Robustness and hygiene properties (closed-stderr behaviour, single-write atomicity, latency/scale budget) that strengthen the contract without changing any observable success behaviour.
****nice-to-have****	—	No optional usage/help extensions are specified here, because the brief defines no help surface beyond the arity diagnostic.
****blocked pending user decision****	OQ-1	Whether an explicit success-mode help flag (``-h`/`--help`` → usage on stdout, exit ``0``) should exist is a ****scope decision for the user****, not the designer. F-10 as written follows the existing F-2 rule (``FR-26``: no option parsing) and therefore defines usage text strictly as a stderr error-path artifact with status ``2``. Introducing a success-mode help path would add new scope and contradict F-2 ``FR-26``; it is deliberately not specified here.

F-11: Unit tests for the add function and CLI

****Feature ID:**** F-11
****Depends on:**** F-1 (``add`` command entry point), F-2 (two positional integer operands), F-3 (integer addition), F-4 (print result to stdout), F-5 (exit-status contract), F-6 (input validation and error message), F-7 (standard library only), F-8 (correct handling of negative results), F-9 (arbitrary-precision output), F-10 (usage/help output).
****Boundary note:**** F-11 owns the automated verification suite for the behaviour specified in F-1 through F-10. It does not redefine arity, grammar, arithmetic, rendering, exit status, diagnostics, or usage text; it defines what the tests must assert and how they must run.

Requirements

Functional Requirements (FR)

ID	Requirement	Priority
FR-251	The project **MUST** include an automated unit-test suite for the internal ``add`` function and the ``add`` CLI, written in Python 3 and using only the Python 3 standard library.	**must-have**
FR-252	The suite **MUST** be runnable from the project root with ``python -m unittest`` or ``python -m unittest discover`` on a stock CPython 3.8+ interpreter with no third-party packages, no network access, and no build/install step.	**must-have**
FR-253	The implementation **MUST** expose a pure function ``add(int1, int2)`` that accepts two Python ``int`` values and returns their exact sum as a Python ``int``, importable without side effects, so that function-level tests can exercise arithmetic directly.	**must-have**

| FR-254 | The suite MUST test the CLI end-to-end in a subprocess using ``sys.executable`` and the project's CLI entry point (module name or script path), asserting exact stdout bytes, exact stderr bytes, and exact exit status. | must-have |

| FR-255 | The suite MUST contain at least one test for every externally observable contract in F-1 through F-10, including arity, operand grammar, sum computation, stdout rendering, exit status, validation pipeline, diagnostics, negative results, arbitrary-precision output, usage output, and the standard-library-only envelope. | must-have |

| FR-256 | The suite MUST be deterministic and hermetic: no dependence on ambient environment variables, locale, working directory, network, filesystem side effects, or test execution order. | must-have |

| FR-257 | A passing run of the suite MUST exit with status 0; any test failure MUST cause a non-zero exit status and MUST report the failing test name and expected/actual values. | must-have |

| FR-258 | The suite MUST fail (not skip or warn) when the CLI crashes with an unhandled exception, a subprocess times out, or the internal ``add`` function raises an unexpected exception. | must-have |

Non-Functional Requirements (NFR)

| ID | Requirement | Priority |

---|---|---

| NFR-151 | Performance: the full test suite MUST complete in under 10 seconds on a typical development machine. | should-have |

| NFR-152 | Portability: the suite MUST run on any platform supported by CPython 3.8+ where the CLI is expected to run, and MUST NOT assume a POSIX shell; subprocess invocation MUST use ``sys.executable``. | must-have |

| NFR-153 | Maintainability: test cases MUST be grouped by feature area, named descriptively, and require no manual setup or external services. | should-have |

User Stories (US)

| ID | Story | Acceptance Criteria |

---|---|---

| US-151 | As a maintainer, I want a comprehensive unit-test suite for the ``add`` function and CLI, so that regressions in arithmetic, validation, rendering, and exit-status behaviour are caught automatically. | Running ``python -m unittest`` reports all tests passing after a change; introducing a bug in any contract area causes at least one test failure. |

| US-152 | As a developer onboarding to the project, I want to run the tests with a single standard-library command without installing dependencies, so that I can verify correctness immediately. | ``python -m unittest`` runs successfully on a stock CPython 3.8+ interpreter with no third-party packages. |

| US-153 | As a release engineer, I want deterministic test results across environments, so that a passing suite in one environment is meaningful for release. | Three consecutive runs in the same environment produce identical pass/fail results and identical test counts. |

Behaviour

- The suite MUST verify both layers: the internal ``add`` function (direct arithmetic) and the CLI process (parsing, validation, rendering, exit status, and stream routing).
- Function-level tests MUST call ``add(a, b)`` with Python ``int`` literals and assert ``result == expected`` using exact integer equality.
- CLI tests MUST run ``[sys.executable, <cli-entry-point>, <arg1>, <arg2>]`` via ``subprocess.run(..., capture_output=True, timeout=10)`` and then assert ``returncode``, ``stdout``, and ``stderr`` exactly.

- The suite MUST include success cases and every failure category from F-1 through F-10: arity failures, grammar failures, and usage-output cases.
- The suite MUST be runnable with a single command and no setup; it MUST NOT require the ``add`` executable to be on ``PATH``.

Business Rules

- BR-1: The suite MUST treat the contracts in F-1 through F-10 as normative. If a test contradicts a stated contract, the test is defective and MUST be corrected, not the contract.
- BR-2: CLI tests MUST invoke the program as a subprocess using ``sys.executable`` and the project's CLI entry point, so tests do not depend on ``PATH``, virtual environments, or installed packages.
- BR-3: CLI tests MUST assert the exact byte content of stdout and stderr and the exact integer exit status. No substring matching, trimming, or normalization is permitted.
- BR-4: Function-level tests MUST call the internal ``add`` function directly with Python ``int`` arguments and assert exact ``int`` equality; they MUST NOT round-trip through strings or floating-point values.
- BR-5: Test cases MUST be grouped by feature area and named descriptively (e.g., ``test_arity_rejects_zero_operands``, ``test_grammar_accepts_leading_zeros``, ``test_sum_mixed_signs``, ``test_render_negative``, ``test_exit_status_success``, ``test_diagnostic_arity``, ``test_usage_output``).
- BR-6: The suite MUST be hermetic: no network access, no third-party imports, no database, no persistent filesystem writes, and no dependence on ambient environment variables, locale, or the current working directory.
- BR-7: If a CLI subprocess crashes (unhandled exception, signal, or Python-level error), the test MUST fail and MUST include the captured stderr in the failure message.
- BR-8: All assertion failures MUST propagate to the unittest runner; the suite MUST NOT catch, swallow, or downgrade failures to warnings.
- BR-9: CLI subprocess tests MUST use a timeout (default 10 seconds) so a hung process fails the test rather than hanging the suite.

Validation

- The suite is validated by running ``python -m unittest`` from the project root; a zero exit status and an ``OK`` summary indicate success.
- Test discovery MUST follow the standard ``unittest`` pattern: files named ``test*.py``, classes subclassing ``unittest.TestCase``, and test methods named ``test*``.
- Every functional requirement in F-1 through F-10 MUST have at least one corresponding test case, traceable via test name or a comment referencing the requirement ID.
- The suite MUST pass on a stock CPython 3.8+ interpreter with no third-party packages and no network access.
- No test MAY depend on ambient locale, environment variables, or the current working directory.

Edge Cases

- EC-1: Arity edge cases: 0 operands, 1 operand, 3 operands, and many operands; each MUST produce exit status 2, empty stdout, and non-empty stderr.
- EC-2: Grammar acceptance edge cases: ``+5``, ``-5``, ``+0``, ``-0``, ``0``, ``00``, ``007``, ``+007``, ``-007``.
- EC-3: Grammar rejection edge cases: empty string, whitespace, ``1.0``, ``1e3``, ``0x10``, ``0o10``, ``0b10``, ``1_000``, ``1,000``, ``--5``, ``+-5``, ``+``, ``-``, ``++5``.
- EC-4: Arithmetic edge cases: ``0 + 0``, ``0 + (-0)``, ``10**100 + 1``, ``-(10**100) + 10**100``, ``10**1000 + (-10**1000)``, and mixed-sign pairs where magnitudes are equal.
- EC-5: Rendering edge cases: zero renders as ``0\n``; positive results have no ``+`` prefix; negative results have a single leading ``-``; no leading zeros; no trailing spaces or extra newlines.

- EC-6: Exit-status edge cases: every successful run exits 0; every arity and grammar failure exits 2; no failure writes to stdout.
- EC-7: Diagnostic/usage edge cases: arity failures emit exactly the two stderr lines specified by F-6 and F-10; grammar failures emit exactly the one diagnostic line specified by F-6; no stdout bytes in either case.
- EC-8: Environment edge cases: tests pass regardless of `LC\_ALL`, `LANG`, `TZ`, and other ambient locale/environment settings; output is compared as bytes.

Error Handling

- Subprocess tests MUST use `subprocess.run(..., timeout=10)`; on `TimeoutExpired`, the test MUST fail with the command and captured output in the message.
- If the CLI cannot be started (e.g., missing module or file), the test MUST fail with a clear diagnostic rather than being skipped.
- If the internal `add` function raises an unexpected exception, the test MUST fail via normal exception propagation.
- `AssertionError` MUST NOT be caught or suppressed anywhere in the suite.
- A failing test MUST report the test name, the expected value, and the actual value for stdout, stderr, or exit status as applicable.

Acceptance Criteria

- AC-1: Running `python -m unittest` from the project root on a stock CPython 3.8+ interpreter with no third-party packages executes the full suite and exits 0.
- AC-2: Every functional requirement in F-1 through F-10 has at least one automated test case that exercises its externally observable behaviour.
- AC-3: Introducing a bug in any of the following areas causes at least one test failure: arity check, operand grammar, sum computation, stdout rendering, exit status, diagnostic messages, usage output, negative-result sign, arbitrary-precision output.
- AC-4: The full suite completes in under 10 seconds on a typical development machine.
- AC-5: Three consecutive runs in the same environment produce identical pass/fail results and identical test counts.
- AC-6: The suite contains no third-party imports, no network access, and no dependence on ambient environment variables or locale.

API Behaviour

- API-1: The internal `add` function MUST be importable without side effects (no stdout/stderr output at import time) and MUST be pure: `add(int1, int2) -> int`.
- API-2: The CLI MUST be exercised via subprocess, not by calling the entry-point function in-process, so that the real process-level contract (exit status and byte streams) is verified.
- API-3: The test suite MUST be compatible with both `python -m unittest` and `python -m unittest discover`; both commands MUST discover the same complete set of tests.
- API-4: The test suite itself MUST expose no public API; it is a dev-time artifact run through the standard `unittest` runner.

Priority

- Feature priority: **must-have** — the product plan explicitly includes unit tests for the `add` function and CLI, and the tests are the primary automated verification of all contracts in F-1 through F-10.

Functional Requirements

Consolidated register of every functional requirement in this specification. Each FR belongs to exactly one feature, is allocated from that feature's assigned global ID range, and states a single testable obligation. Ownership boundaries are strict: a requirement is defined once, in the feature that owns the concern.

F-1: add command entry point — FR-1 ... FR-8

ID	Requirement	Priority
FR-1	The program MUST be invocable as <code>`add <int1> <int2>`</code> from a POSIX-style shell, accepting exactly two positional operands. must-have	
FR-2	The program MUST parse each operand as an integer using the strict decimal integer grammar owned by F-2 (see FR-27 ... FR-31). must-have	
FR-3	The program MUST compute the exact arithmetic sum <code>`int1 + int2`</code> using the arbitrary-precision arithmetic owned by F-3 (see FR-51 ... FR-52). must-have	
FR-4	On success the program MUST write the result as a base-10 integer string followed by exactly one <code>`\n`</code> (U+000A) to <code>**stdout**</code> , and write nothing to <code>stderr</code> . must-have	
FR-5	On success the program MUST terminate with exit status <code>`0`</code> . must-have	
FR-6	If argument arity is not exactly two operands, the program MUST write a usage/diagnostic message to <code>**stderr**</code> , write nothing to <code>stdout</code> , and terminate with exit status <code>`2`</code> . must-have	
FR-7	If any operand fails the F-2 integer grammar, the program MUST write a single-line diagnostic to <code>**stderr**</code> naming the offending argument, write nothing to <code>stdout</code> , and terminate with exit status <code>`2`</code> . must-have	
FR-8	The program MUST NOT recognize options, flags, subcommands, or environment-controlled modes: the first two command-line arguments are always treated as operands, and no argument may be consumed as a switch. must-have	

F-2: Two positional integer operands — FR-26 ... FR-36

ID	Requirement	Priority
FR-26	The program MUST treat the first command-line argument after the program name as <code>`int1`</code> and the second argument as <code>`int2`</code> , with no option parsing or flag recognition. must-have	
FR-27	Each operand MUST be validated against the canonical decimal integer grammar: an optional single sign character (<code>`+`</code> U+002B or <code>`-`</code> U+002D) followed by one or more ASCII decimal digits (<code>`0`</code> – <code>`9`</code>), and nothing else. must-have	
FR-28	The grammar MUST accept leading zeros (e.g., <code>`007`</code> , <code>`-000`</code> , <code>`+00042`</code>) and interpret them as their base-10 integer value. must-have	
FR-29	The grammar MUST accept an explicit <code>`+`</code> sign (e.g., <code>`+5`</code> , <code>`+0`</code> , <code>`+007`</code>). must-have	
FR-30	The grammar MUST accept a single leading <code>`-`</code> sign for negative numbers (e.g., <code>`-5`</code> , <code>`-0`</code> , <code>`-007`</code>). must-have	
FR-31	The grammar MUST reject any operand containing whitespace, underscores, decimal points, commas, exponent notation (<code>`e`</code> / <code>`E`</code>), base prefixes (<code>`0x`</code> , <code>`0o`</code> , <code>`0b`</code>), or any character outside <code>`[0-9+-]`</code> (with sign permitted only at position 0). must-have	
FR-32	The grammar MUST reject a sign-only operand (<code>`+`</code> , <code>`-`</code>) and the empty operand, because at least one digit MUST follow any sign. must-have	
FR-33	The grammar MUST reject any sign that does not occupy position 0, and any operand containing more than one sign (e.g., <code>`1-2`</code> , <code>`+-5`</code> , <code>`--5`</code> , <code>`5+`</code>). must-have	

| FR-34 | Validation MUST be applied to the raw argument token exactly as received in `argv`, with no whitespace trimming, case folding, Unicode normalization, or delegating to a permissive parser. | must-have |

| FR-35 | The grammar MUST accept only ASCII digits: Unicode digit characters (e.g., `■`, `²`, `■`), fullwidth digits, and non-ASCII signs MUST be rejected. | must-have |

| FR-36 | Both operands MUST be evaluated independently against the grammar, so that a valid first operand never permits an invalid second operand to proceed. | must-have |

F-3: Integer addition — FR-51 ... FR-60

| ID | Requirement | Priority |

|---|---|---|

| FR-51 | The program MUST compute `int1 + int2` as the exact mathematical sum for every pair of operands accepted under the F-2 operand grammar. | must-have |

| FR-52 | The computation MUST use arbitrary-precision integer arithmetic; the result MUST NOT overflow, truncate, round, saturate, or wrap at any magnitude or sign. | must-have |

| FR-53 | The parse → add → render path MUST NOT use binary floating-point or decimal floating-point values at any point. | must-have |

| FR-54 | Sign rules: same-sign operands MUST produce a result with that sign and magnitude equal to the sum of the magnitudes; opposite-sign operands MUST produce a result whose magnitude is the difference of the magnitudes and whose sign is the sign of the larger-magnitude operand; if the magnitudes are equal the result MUST be `0`. | must-have |

| FR-55 | `0` MUST be the additive identity: adding a zero operand MUST leave the other operand's value unchanged (`add 0 N` → `N`, `add N 0` → `N`, `add 0 0` → `0`). | must-have |

| FR-56 | Addition MUST be commutative at this interface: `add A B` and `add B A` MUST emit byte-identical stdout for any valid operands `A` and `B`. | must-have |

| FR-57 | Addition MUST be associative for chained evaluation of the pair-wise results, so that grouping of intermediate sums never changes the exact value produced. | should-have |

| FR-58 | The result of the computation MUST be a mathematically exact integer value with no imposed upper or lower magnitude bound beyond available process memory. | must-have |

| FR-59 | The arithmetic MUST be exposed as a pure internal function accepting two integer values and returning their exact sum, performing no I/O, no printing, and no process exit. | must-have |

| FR-60 | The computation MUST NOT mutate or reinterpret its operands as strings after parsing; sign and magnitude decisions MUST be derived from the parsed integer values. | should-have |

F-4: Print result to stdout — FR-76 ... FR-84

| ID | Requirement | Priority |

|---|---|---|

| FR-76 | The program MUST write the textual rendering of the computed sum `S` to **stdout** on every successful run (i.e., whenever the F-1 arity rules and the F-2 operand grammar are satisfied). | must-have |

| FR-77 | `S` MUST be rendered in **base 10** using ASCII digit characters `0`–`9` (U+0030–U+0039) only. No other radix, and no radix prefix (`0x`, `0o`, `0b`), may appear. | must-have |

| FR-78 | The rendered value MUST be followed by **exactly one** line terminator: `\n` (U+000A, byte `0x0A`). Nothing may be written after it. | must-have |

| FR-79 | The rendering MUST NOT contain leading zeros, except that the value zero itself is rendered as the single character `0`. | must-have |

| FR-80 | A positive sum MUST be rendered without any leading sign character: `+` MUST NOT be emitted for values greater than zero. | must-have |

FR-81	The rendering MUST contain no thousands separators, no grouping characters, no padding spaces, no carriage return (`\r`), and no trailing whitespace of any kind.	must-have
FR-82	The stdout line MUST be written as a single logical write, fully flushed before the process terminates, so that the emitted line is never interleaved with stderr output.	must-have
FR-83	Stdout MUST contain nothing except the single result line: no banners, version strings, prompts, progress text, or debug output.	must-have
FR-84	The rendering MUST be locale-independent, so that the emitted bytes are identical regardless of `LANG`, `LC\_ALL`, `LC\_NUMERIC`, or platform default encoding settings.	must-have

F-5: Exit-status contract — FR-101 ... FR-108

ID	Requirement	Priority
FR-101	The process MUST terminate with exit status `0` ****iff**** exactly two operands were supplied, both satisfied the F-2 operand grammar, and the F-4 rendering of the sum was written to stdout as a complete line.	must-have
FR-102	The process MUST terminate with exit status `2` when argument arity is not exactly two operands (zero, one, or three-plus arguments), writing nothing to stdout.	must-have
FR-103	The process MUST terminate with exit status `2` when any supplied operand fails the F-2 operand grammar, writing nothing to stdout.	must-have
FR-104	The process MUST NOT terminate with any exit status other than `0` or `2`: no `1`, no signal-derived status, no uncaught-exception traceback status may be observable for any input.	must-have
FR-105	When exit status is `0`, stdout MUST contain exactly one line terminated by a single `\n` and stderr MUST be empty (zero bytes).	must-have
FR-106	When exit status is `2`, stdout MUST be empty (zero bytes) and all diagnostic output MUST be confined to stderr.	must-have
FR-107	The exit status MUST be observable by the invoking shell via `\$?` (or equivalent) immediately after the program returns.	must-have
FR-108	The exit status MUST be a deterministic function of the input arguments alone, unaffected by environment variables, working directory, terminal type, or whether stdout is a TTY or a pipe.	must-have

F-6: Input validation and error message — FR-126 ... FR-133

ID	Requirement	Priority
FR-126	Validation MUST run as a fixed, deterministic pipeline: **** (1) **** arity check → **** (2) **** grammar check of the first operand → **** (3) **** grammar check of the second operand → **** (4) **** hand-off to computation/output. The first failing stage MUST abort the pipeline; later stages MUST NOT run.	must-have
FR-127	An arity failure (argument count ≠ 2) MUST emit exactly two stderr lines: an error line naming the expected count, followed by the canonical F-10 usage line; stdout MUST receive zero bytes.	must-have
FR-128	A grammar failure MUST emit exactly one stderr line naming the offending argument (its 1-based position and, where safe, its content); the usage line MUST NOT additionally be emitted for this class.	must-have
FR-129	Diagnostic output MUST escape or elide any control characters, newlines, or terminal escape sequences present in a rejected operand, so that a hostile operand cannot inject extra lines or control the terminal.	must-have

FR-130	Every failing run MUST produce zero bytes on stdout, in every failure class, without exception.	must-have
FR-131	Diagnostic messages MUST be deterministic and MUST NOT include timestamps, PIDs, hostnames, absolute file paths, interpreter version strings, or traceback frames.	must-have
FR-132	Every diagnostic line MUST be terminated by exactly one ``n`` (U+000A) and MUST NOT be wrapped or truncated by the program.	must-have
FR-133	The error line for the arity failure MUST state the number of operands received and the number required, in a fixed, single-line, parseable format.	should-have

F-7: Standard library only — FR-151 ... FR-157

ID	Requirement	Priority
FR-151	The program MUST run on a stock CPython 3 interpreter (3.8 or later) with **no** third-party packages installed beyond those shipped with the interpreter distribution itself.	must-have
FR-152	The implementation MUST import only modules from the Python 3 standard library, or use built-in types/functions that require no import at all. No ``pip install`` step, virtual-environment dependency, third-party ``requirements.txt`` entry, or vendored third-party file may be required.	must-have
FR-153	The program MUST perform no network access at runtime, in any code path, including error paths and test paths.	must-have
FR-154	The program MUST NOT require reading or writing any file, configuration store, cache, or temporary directory in order to process a valid invocation.	must-have
FR-155	The program MUST be runnable directly with the interpreter (e.g., ``python add.py 2 3``) without a build, compile, packaging, or installation step.	must-have
FR-156	The program MUST NOT require any environment variable to be set, and MUST behave identically whether or not any are set.	must-have
FR-157	The program MUST be hermetic: with the same arguments, it MUST produce byte-identical stdout, stderr, and exit status on any conforming interpreter and platform, with no external resource consulted.	must-have

F-8: Correct handling of negative results — FR-176 ... FR-182

ID	Requirement	Priority		
FR-176	The program MUST detect that the computed sum ``S`` is strictly less than zero and render ``S`` as a signed value: a single leading ``-`` (U+002D HYPHEN-MINUS, byte ``0x2D``) followed by the base-10 magnitude ``	S	``. ``add 2 -5`` MUST emit ``-3n``.	must-have
FR-177	The sign MUST be emitted as ASCII ``-`` U+002D (byte ``0x2D``). U+2212 MINUS SIGN, U+2010/U+2011 hyphens, en dash U+2013, em dash U+2014, the fullwidth minus U+FF0D, or any digit-glyph substitution MUST NOT be emitted.	must-have		
FR-178	A sum of exactly zero MUST NEVER be rendered with a sign: ``add -0 0``, ``add 5 -5``, and ``add -5 5`` MUST each emit ``0n``, never ``-0n`` or ``+0n``.	must-have		
FR-179	The magnitude digits of a negative result MUST follow the same rules as F-4: base 10, ASCII digits only, no leading zeros, no separators, no padding.	must-have		
FR-180	The leading ``-`` MUST be immediately adjacent to the first magnitude digit, with no intervening space, tab, or other character.	must-have		
FR-181	The emitted negative rendering MUST round-trip: it MUST itself be accepted by the F-2 operand grammar and, when re-submitted, MUST yield the same value.	must-have		
FR-182	Negative results MUST NOT use parentheses, accounting notation, trailing signs, or any alternative representation to indicate sign.	must-have		

F-9: Arbitrary-precision output — FR-201 ... FR-206

| ID | Requirement | Priority |

|---|---|---|

| FR-201 | The program MUST render the sum `S` in base 10 for any magnitude that fits in available process memory. The number of emitted digits MUST equal the exact digit count of `S`; there MUST be no fixed-width cap (no 16/19/32/64/128-bit or any other fixed width) applied to the output. | must-have |

| FR-202 | The emitted digit count MUST be exactly `D = 1` when `S == 0`, else `len(str(abs(S)))`, and MUST be independent of the operand text length after leading zeros are stripped. | must-have |

| FR-203 | The rendering MUST NOT use scientific notation, exponent form, truncation, ellipsis, or magnitude-dependent abbreviation at any size. | must-have |

| FR-204 | The rendering MUST NOT insert grouping separators at any magnitude: no `,` and no `.` may appear as a thousands marker, regardless of size or locale. | must-have |

| FR-205 | Memory consumption for the emitted string MUST be proportional to the digit count of `S`, with no fixed-size intermediate buffer that silently caps or corrupts large results. | must-have |

| FR-206 | Very large operands (hundreds or thousands of digits) MUST produce an exact result within the same process run and MUST NOT trigger a recursion-limit error, integer-conversion limit, or interpreter-level numeric cap. | must-have |

F-10: Usage/help output — FR-226 ... FR-233

| ID | Requirement | Priority |

|---|---|---|

| FR-226 | The program MUST emit the canonical usage line to `**stderr**` whenever the invocation is invalid (wrong arity or an operand failing the F-2 grammar). | must-have |

| FR-227 | The canonical usage text MUST be the exact single line ``usage: add <int1> <int2>`` followed by exactly one ``\n``, with no other characters, decoration, or wrapping. | must-have |

| FR-228 | The usage line MUST NOT be emitted on a successful run; stdout and stderr MUST both remain free of usage text when exit status is ``0``. | must-have |

| FR-229 | The usage line MUST NEVER be written to stdout, in any code path. | must-have |

| FR-230 | The usage text MUST be byte-identical across every failure class (arity failure, first-operand grammar failure, second-operand grammar failure) and across every argument count. | must-have |

| FR-231 | The usage line MUST be emitted exactly once per failing run, never repeated or duplicated. | must-have |

| FR-232 | The program MUST NOT define a ``--help``, ``-h``, or any other flag path: such tokens are ordinary arguments subject to the F-2 grammar and, if invalid, trigger the standard failure behaviour of FR-226. | must-have |

| FR-233 | The usage text MUST be a fixed literal: no runtime interpolation, no program version, no author attribution, no localization, and no variation by platform or environment. | must-have |

F-11: Unit tests for the add function and CLI — FR-251 ... FR-260

| ID | Requirement | Priority |

|---|---|---|

| FR-251 | The project MUST include an automated unit-test suite for the internal ``add`` function and the ``add`` CLI, written in Python 3 and using only the Python 3 standard library. | must-have |

| FR-252 | The suite MUST be runnable from the project root with ``python -m unittest`` or ``python -m unittest discover`` on a stock CPython 3.8+ interpreter with no third-party packages, no network access, and no build/install step. | must-have |

| FR-253 | The implementation MUST expose the arithmetic as an importable callable taking two integer arguments and returning their exact sum, so that unit tests can exercise it directly without

spawning a subprocess or capturing streams. | must-have |
 | FR-254 | The suite MUST contain at least one assertion for every functional requirement FR-1 ...
 FR-250 that is observable at the unit or process level, including the success path, the arity-failure path,
 and both operand-grammar failure paths. | must-have |
 | FR-255 | CLI-level tests MUST assert the three observable channels independently and exactly:
 stdout bytes, stderr bytes, and the process exit status. | must-have |
 | FR-256 | The suite MUST include table-driven cases covering grammar acceptance ('5', '+5', '-5',
 '0', '-0', '+0', '007', '-000', '+00042') and grammar rejection (empty, '+', '-', '--5', '+-5', '5+', '1-2',
 '5', '5', '1_000', '1.0', '1,000', '1e3', '0x10', '0o7', '0b1', '■', '^', fullwidth forms). | must-have |
 | FR-257 | The suite MUST include arbitrary-precision cases whose operands and expected results far
 exceed native machine-word width, asserting exact digit-for-digit output. | must-have |
 | FR-258 | The suite MUST be deterministic and hermetic: no reliance on network, clock, locale, random
 seeds, filesystem state outside the project, or test execution order. | must-have |
 | FR-259 | The full suite MUST pass on a clean checkout with a stock CPython 3.8+ interpreter and
 MUST NOT require any fixture, data file, or generated artifact to be present beforehand. | must-have |
 | FR-260 | The suite MUST NOT depend on the current working directory or on environment variables,
 and MUST produce the same pass/fail outcome when invoked from any directory with absolute or
 relative paths. | must-have |

Non-Functional Requirements

****Scope:**** these are the global, cross-cutting non-functional requirements for `add`. They apply to
 every feature (F-1 ... F-11) unless a feature section narrows them. Ids `NFR-1` ... `NFR-15` are
 defined once, here, and are never re-defined elsewhere.

Performance (NFR-1 ... NFR-3)

ID	Requirement	Target / verification
NFR-1	End-to-end latency for a small invocation must stay imperceptible in an interactive shell.	Target: `add 2 3` completes (process exit, output flushed) in ≤ 150 ms median / ≤ 250 ms p95** on a 2019-era x86-64 laptop with CPython 3.11 and a warm filesystem cache. Verification: 100-run timing harness over the shipped entry point; report median and p95.
NFR-2	Work must scale linearly in the number of input digits — no super-linear blow-up in the parse → add → render path.	Target: 1,000-digit operands → ≤ 200 ms**; 10,000-digit operands → ≤ 2 s**; 1,000-digit and 10,000-digit runs must show roughly 10× time ratio, not 100×. Verification: median of 5 runs per fixture against generated arbitrary-precision operands.
NFR-3	Peak memory must be bounded by the size of the operands plus the result; no buffering of the whole environment or argument space.	Target: peak RSS $\leq$ interpreter baseline + ≤ 1 KB per 1,000 digits** of input. Verification: <code>/usr/bin/time -v`</code> (or <code>resource.getrusage`</code>) on the maximum-size fixture.

Scalability & Availability (NFR-4 ... NFR-7)

ID	Requirement	Target / verification
NFR-4	The scale envelope of the emitted number must be effectively unbounded: the result must be exact for any magnitude that fits in available process memory, with no fixed-width cap (no 16/19/32/64/128-bit wrap, truncation, saturation, or rounding).	Target: correct byte-exact result at 10■-digit operands within memory limits. Verification: property test comparing against an independent exact-integer reference over random digit lengths, plus one maximum-size acceptance run.

| NFR-5 | The process must be stateless, re-entrant, and concurrency-safe: no lock files, temp files, caches, counters, or shared mutable state on disk. | Target: 50 concurrent invocations complete with results byte-identical to the serial run and with no interference. Verification: parallel shell loop, per-invocation stream capture, `diff` against golden output. |

| NFR-6 | Availability must be 100% whenever a stock CPython 3 interpreter is present; correctness must never depend on network, database, daemon, clock, or configuration file. | Target: full acceptance suite passes with networking disabled and with `\$HOME`/cwd made read-only. Verification: run the suite in a network-isolated sandbox and in a read-only cwd. |

| NFR-7 | Behaviour must be deterministic and idempotent: identical `argv` produces byte-identical stdout, byte-identical stderr, and an identical exit status on every run, machine, user, and working directory. | Target: 1,000 repeated runs hash-stable across at least 2 directories and 2 user accounts. Verification: `sha256sum` over captured stdout/stderr/status triples; hashes must match. |

Security (NFR-8 ... NFR-11)

ID	Requirement	Target / verification

| NFR-8 | Operand text must be handled strictly as data. No `eval`, `exec`, `ast.literal\_eval`, `pickle`, `subprocess`, shell interpolation, or dynamic import may derive from user input. | Target: zero occurrences of those constructs on any path that consumes `argv`. Verification: static scan of shipped source plus a hostile-argv fuzz corpus (shell metacharacters, `\$(...)`, quotes, `;`, backticks, control bytes) that must never execute or alter environment. |

| NFR-9 | No input may produce an unhandled Python traceback or leak interpreter internals. Every rejection must yield the F-6 diagnostic and the F-5 exit-status contract. | Target: 0 tracebacks and 0 stack traces in output across a $\geq 10,000$ -case fuzz corpus (empty operand, emoji, unicode digits, `10█`-character strings, lone signs, NUL-free control characters). Verification: automated fuzz run asserting status $\in \{0, 2\}$ and stdout/stderr shape. |

| NFR-10 | The process must take no privilege and cause no external side effects: no filesystem writes, no socket connections, no child processes, no environment mutation, no reads beyond interpreter start-up. | Target: after interpreter start-up, the syscall trace contains no `open(O\_WRONLY|O\_CREAT|O\_APPEND)`, `connect`, `socket`, or `execve`. Verification: `strace -f` (or platform equivalent) over success and failure runs; filesystem hash-diff of cwd and `\$TMPDIR` is empty. |

| NFR-11 | Resource-exhaustion resistance: pathological input must not hang, and time must remain proportional to input size with no unbounded recursion or retry loop. | Target: a 10█-character invalid operand is rejected in ≤ 100 ms; recursion depth used by the implementation is ≤ 10 . Verification: timing on pathological fixtures (long digit strings, long sign runs, deeply nested quoting) with a hard external timeout. |

Data & Residency (NFR-12 ... NFR-13)

ID	Requirement	Target / verification

| NFR-12 | Zero persistence, zero telemetry, zero PII: no logs, caches, config files, history files, temp files, crash dumps, or outbound transmission. Operands and the sum live only in process memory for the duration of the invocation and are discarded at exit. | Target: filesystem mtime/content diff of cwd, `\$HOME`, and `\$TMPDIR` before/after a run is empty; no outbound traffic. Verification: pre/post directory snapshots plus `strace` network-syscall check. |

| NFR-13 | Locale-, encoding-, and environment-independence: emitted bytes must be ASCII digits `0`-`9` (U+0030–U+0039) and at most one `` (U+002D). Behaviour must be unchanged by `LANG`, `LC\_ALL`, `LC\_NUMERIC`, `TZ`, `PYTHONHASHSEED`, `PYTHONIOENCODING`, and the current

working directory. | Target: byte-identical output under `LC\_ALL=C`, `LC\_ALL=tr\_TR.UTF-8` (dotted-locale), and `PYTHONHASHSEED=random`. Verification: cross-environment matrix run comparing captured bytes; assert stdout matches `^-[0-9]+\n\$`. |

Deployment & Environment (NFR-14 ... NFR-15)

ID	Requirement	Target / verification

| NFR-14 | The program must ship as a single self-contained Python 3 source using only the standard library (or built-ins requiring no import), runnable on stock CPython **3.8** or later with no `pip install`, no virtual environment, no `requirements.txt`, no vendored third-party file, and no build step. | Target: the entry point executes on a clean interpreter after nothing more than copying the source. Verification: fresh interpreter/container run of the full acceptance suite with an empty site-packages. |

| NFR-15 | Deployment surface must be portable and offline-verifiable: POSIX-style invocation from any cwd, no root, no external binaries beyond the shell; the automated suite must run with `python -m unittest` / `python -m unittest discover` from the project root with networking disabled. | Target: suite green on Linux and macOS with network off and with `\$HOME` read-only; Windows users can run the same logic via `python -m add` (no shell-specific behaviour required in the tested path). Verification: CI-style script running `python -m unittest discover` under an offline sandbox on both target platforms. |

User Stories

The user stories below cover every feature F-1 ... F-11. Each story uses the global `US-` id assigned to its feature, and local `AC-` ids restart at 1 within each feature.

F-1: add command entry point

US-1 — As a POSIX shell user, I want to invoke `add <int1> <int2>` as a single command, so that I can add two integers without writing any code.

- **AC-1**: The entry point is invocable by that name from a POSIX-style shell.
- **AC-2**: An invocation supplying exactly two operands runs to completion.

US-2 — As a shell user, I want a successful invocation to write only the result to stdout and nothing to stderr, so that I can safely pipe and compose its output.

- **AC-1**: On success, stdout contains only the result line and stderr is empty.
- **AC-2**: On success, the process exits with status `0`.

US-3 — As a shell user, I want a wrong-arity invocation to fail loudly with a diagnostic on stderr and no stdout output, so that a mistake does not silently corrupt a pipeline.

- **AC-1**: Any argument count other than two produces a diagnostic on stderr, empty stdout, and exit status `2`.

F-2: Two positional integer operands

US-16 — As a shell user, I want the two operands taken strictly by position, so that the values are unambiguous.

- **AC-1**: The first argument after the program name is treated as `int1` and the second as `int2`.
- **AC-2**: No option/flag parsing is performed; a leading `-` is treated as a sign character, not a flag.

US-17 — As a shell user, I want operands with an optional sign and leading zeros to be accepted, so that I can express values naturally.

- **AC-1**: ``+5``, ``-5``, ``007``, ``-000``, and ``+00042`` are all accepted and interpreted as their base-10 values.

US-18 — As a shell user, I want malformed operands (whitespace, underscores, decimals, commas, exponent notation, base prefixes, stray sign characters) rejected, so that I never get a silent misinterpretation.

- **AC-1**: Each rejected form yields the operand-grammar error and no stdout output.

F-3: Integer addition

US-31 — As a shell user, I want the exact mathematical sum of the two operands, so that the result is always correct.

- **AC-1**: The emitted value equals ``int1 + int2`` for every pair of accepted operands.

US-32 — As a shell user, I want arbitrarily large integers to be summed without overflow, so that very big numbers work too.

- **AC-1**: The computation uses arbitrary-precision arithmetic; no truncation, rounding, saturation, or wrap occurs at any magnitude or sign.

- **AC-2**: No binary or decimal floating-point value is used in the parse → add → render path.

US-33 — As a shell user, I want correct sign behaviour for mixed-sign operands, so that signed arithmetic is reliable.

- **AC-1**: Same-sign operands produce a result with that sign and the summed magnitudes.

- **AC-2**: Opposite-sign operands produce a magnitude equal to the difference, signed by the larger-magnitude operand.

- **AC-3**: Equal magnitudes of opposite sign produce ``0``.

F-4: Print result to stdout

US-46 — As a shell user, I want the sum rendered in base-10 ASCII digits followed by exactly one newline, so that the output is canonical and easy to consume.

- **AC-1**: The rendering uses ASCII digits ``0``–``9`` only, with no radix prefix.

- **AC-2**: Exactly one ``\n`` (U+000A) terminates the value, with nothing written after it.

US-47 — As a shell user, I want the number rendered without leading zeros, so that the output is normalized.

- **AC-1**: No leading zeros are emitted, except that the value zero itself is rendered as the single character ``0``.

F-5: Exit-status contract

US-61 — As a script author, I want a deterministic exit-status contract (success vs. input error), so that I can branch on the outcome.

- **AC-1**: Exit status is ``0`` iff exactly two operands were supplied, both passed the grammar, and a complete result line was written to stdout.

- **AC-2**: Exit status is ``2`` for any arity violation or any operand-grammar failure.

US-62 — As a script author, I want the exit status to be consistent with the byte streams, so that I can trust either signal.

- **AC-1**: Status ``0`` implies a result line on stdout and empty stderr.

- **AC-2**: Status ``2`` implies empty stdout and a diagnostic on stderr.

F-6: Input validation and error message

****US-76**** — As a shell user, I want validation to run in a fixed order (arity, then first operand, then second), so that I get a single, predictable error for a bad invocation.

- ****AC-1****: The first failing stage aborts the pipeline and later stages do not run.

****US-77**** — As a shell user, I want a clear, single-line diagnostic that names the offending argument, so that I can correct my input quickly.

- ****AC-1****: The error line identifies the offending argument.

- ****AC-2****: A failing run emits no bytes on stdout.

F-7: Standard library only

****US-91**** — As a user, I want to run ``add`` on a stock CPython 3.8+ interpreter with no third-party packages, so that I do not need to install anything.

- ****AC-1****: Only Python 3 standard-library modules (or built-ins) are used; no ``pip install``, virtual-environment, or vendored third-party file is required.

****US-92**** — As a user, I want ``add`` to run hermetically, so that it behaves predictably offline and in restricted environments.

- ****AC-1****: The program performs no network access and depends on no external files or configuration.

F-8: Correct handling of negative results

****US-106**** — As a shell user, I want a negative sum rendered with a leading ``-``, so that the sign is unambiguous.

- ****AC-1****: ``add 2 -5`` emits ``-3\n``.

****US-107**** — As a shell user, I want the sign to be an ASCII hyphen-minus, so that the output is machine-parseable.

- ****AC-1****: The sign is emitted as U+002D (byte ``0x2D``); no Unicode-minus or dash substitution is used.

F-9: Arbitrary-precision output

****US-121**** — As a shell user, I want the sum rendered exactly for arbitrarily large magnitudes, so that no digit is lost.

- ****AC-1****: The emitted digit count equals the exact digit count of the sum, with no fixed-width cap.

****US-122**** — As a shell user, I want the number formatted locale-independently, so that the output is stable across environments.

- ****AC-1****: No thousands separators or locale-specific grouping are emitted; the bytes are identical regardless of locale or configuration.

F-10: Usage/help output

****US-136**** — As a shell user, I want a canonical usage line on stderr when I invoke the command incorrectly, so that I can learn the correct syntax.

- ****AC-1****: An incorrect invocation causes the usage line to be emitted on stderr.

****US-137**** — As a shell user, I want the usage text to be deterministic and not localized or interpolated, so that it is stable and scriptable.

- **AC-1**: The usage text is byte-for-byte identical on every emission; no runtime interpolation or localization is applied.

F-11: Unit tests for the add function and CLI

US-151 — As a developer, I want an automated unit-test suite for the `add` function and the CLI, written with the standard library only, so that I can verify behavior reproducibly.

- **AC-1**: The suite is runnable with `python -m unittest` or `python -m unittest discover` on a stock CPython 3.8+ interpreter.

US-152 — As a developer, I want the tests to run with no network, build, or install step, so that CI stays simple and hermetic.

- **AC-1**: The suite requires no third-party packages, no network access, and no build/install step.

US-153 — As a developer, I want the tests to cover success paths, arity errors, grammar errors, negative results, and arbitrary-precision cases, so that regressions are caught.

- **AC-1**: Tests assert on stdout bytes, stderr, and exit status for each outcome category.

API Contracts

`add` is a command-line program, not a networked service, so its "API" is the **process contract**: the argument vector it consumes, the streams it writes, the exit status it returns, and the single in-process callable the CLI wraps. Global ids (`FR-`NFR-`US-``) are defined elsewhere in this document and are only *referenced* here — never re-defined. Every `API-`` id below is **local** to its feature and restarts at ``1`` within that feature.

Cross-feature process-level surface (referenced by all features)

Boundary	Direction	Contract	
---	---	---	
`argv`	in	Exactly two operands after the program name (<code>`argv[1]`</code> , <code>`argv[2]`</code>); <code>`argv[0]`</code> is the program name and is never an operand.	
stdin	in	Not read. The program MUST NOT consume stdin on any path.	
stdout	out	Result line only, on success. Zero bytes on every failure path.	
stderr	out	Diagnostics/usage only, on failure paths. Zero bytes on success.	
exit status	out	<code>`0`</code> on success; <code>`2`</code> on any input/arity/grammar error. No other status is produced for defined conditions.	

F-1: add command entry point

id	Contract	Trace	
---	---	---	
API-1	Invocation surface is <code>`add <int1> <int2>`</code> . The executable resolves the operands from <code>`argv[1]`</code> and <code>`argv[2]`</code> ; <code>`argv[0]`</code> (program name) is never interpreted as an operand.	FR-1	
API-2	The entry point has no flag/option parsing surface: any token beginning with <code>`-`</code> in an operand position is treated as a signed integer token, not an option (<code>`add -5 3`</code> is valid; there is no <code>`--help`</code> / <code>`-h`</code> flag path).	FR-1, FR-26	
API-3	Succession contract: on a well-formed invocation the entry point performs exactly <code>parse → add → render → emit</code> and then returns; it performs no retries, no fallback, and no interactive prompts.	FR-3, FR-4	

| API-4 | Failure contract: when the invocation is malformed the entry point emits nothing on stdout, emits a diagnostic on stderr, and yields exit status `2` (status value owned by F-5). | FR-6, FR-7 |

F-2: Two positional integer operands

| id | Contract | Trace |

|---|---|---|

| API-1 | Operand input type is the raw OS argument string (bytes decoded to `str` by the interpreter); the program performs no shell-level re-tokenization. | FR-26 |

| API-2 | Accepted token grammar: `[+-]?[0-9]+` — optional single leading sign, then one or more ASCII decimal digits, then end of token. | FR-27 |

| API-3 | Accepted: `007`, `-000`, `+00042`, `+5`, `-5`, `0`. Each maps to its base-10 integer value (leading zeros discarded by value, sign preserved by value). | FR-28, FR-29, FR-30 |

| API-4 | Rejected (grammar failure → F-6 diagnostic, F-5 status `2`): whitespace-embedded (`"2 3"` as one arg, `" 2"`, `"2 "`), underscores (`1\_000`), decimal point (`1.0`), comma (`1,000`), exponent (`1e3`, `2E4`), base prefix (`0x10`, `0o7`, `0b1`), empty string, bare sign (`+`, `-`), doubled/embedded sign (`--5`, `1-2`), any character outside `[0-9+-]`, and any sign not in position 0. | FR-31, FR-32 |

| API-5 | The grammar is the single source of truth for operand acceptance; downstream stages (F-3 computation) MUST NOT re-validate or re-normalize the token — they consume the already-parsed integer. | FR-26 |

F-3: Integer addition

| id | Contract | Trace |

|---|---|---|

| API-1 | Internal callable contract: `add(a: int, b: int) -> int` — a pure function returning the exact mathematical sum `a + b`. No I/O, no global state, no mutation of inputs. | FR-51 |

| API-2 | Numeric domain is Python arbitrary-precision `int`; inputs are the parsed operands from F-2, output is an exact `int` with no overflow/wrap/saturation/rounding at any magnitude or sign. | FR-52 |

| API-3 | The computation path MUST NOT materialize `float` or `Decimal`; the result type asserted at the boundary is `int`. | FR-53 |

| API-4 | Sign/magnitude contract: same-sign → signed sum of magnitudes; opposite-sign → sign of larger magnitude, magnitude difference; equal magnitudes → `0`. | FR-54, FR-55 |

F-4: Print result to stdout

| id | Contract | Trace |

|---|---|---|

| API-1 | On success the program writes exactly one line to stdout: `` followed by a single `\n` (U+000A). Nothing precedes it and nothing follows it. | FR-76, FR-78 |

| API-2 | Rendering is ASCII base-10 only (`0`–`9`); no radix prefix (`0x`/`0o`/`0b`), no thousands separators, no whitespace padding, no locale-dependent grouping. | FR-77 |

| API-3 | Rendering has no leading zeros except that the value zero renders as the single character `0` (`0`→`0`, `5`→`5`, `007+0`→`7`). | FR-79 |

| API-4 | One line per process run: exactly one `\n` terminator, never `\r\n`, never multiple trailing newlines, never a trailing space. | FR-78 |

| API-5 | On any failure path stdout receives **zero bytes**; the successful stdout contract and the failure stdout contract are mutually exclusive. | FR-76, FR-4 |

F-5: Exit-status contract

| id | Contract | Trace |

|---|---|---|
 | API-1 | Exit status `0` ****iff**** exactly two grammar-valid operands were supplied and the result line was written to stdout completely. | FR-101 |
 | API-2 | Exit status `2` when arity $\neq 2$ (zero, one, or three-plus operands); stdout receives zero bytes. | FR-102 |
 | API-3 | Exit status `2` when any supplied operand fails the F-2 grammar; stdout receives zero bytes. | FR-103 |
 | API-4 | No defined input condition produces a status other than `0` or `2`; the program **MUST NOT** emit status `1` for user input errors. | FR-101, FR-102 |
 | API-5 | Invariant coupling: status `0` ■ stdout has exactly one result line **AND** stderr has zero bytes; status `2` ■ stdout has zero bytes **AND** stderr has ≥ 1 diagnostic line. | FR-101 |

F-6: Input validation and error message

id	Contract	Trace
API-1	Validation is a fixed ordered pipeline: (1) arity $\rightarrow$ (2) grammar of `argv[1]` $\rightarrow$ (3) grammar of `argv[2]` $\rightarrow$ (4) hand-off to F-3/F-4. First failing stage aborts; later stages do not run.	FR-126
API-2	Arity-failure stderr contract: exactly two stderr lines — an error line naming the expected count, followed by the usage line (text owned by F-10) — and no stdout output.	FR-127
API-3	Grammar-failure stderr contract: a single-line diagnostic that names the offending argument; stdout receives zero bytes.	FR-7, FR-103
API-4	Offending input echoed in a diagnostic **MUST** be emitted safely (no shell re-interpretation, no format-string interpolation of the raw token) and bounded in length so an oversized argument cannot cause unbounded output.	FR-126
API-5	Diagnostics are deterministic: identical inputs produce byte-identical stderr output across runs.	FR-126

F-7: Standard library only

id	Contract	Trace
API-1	Import surface is restricted to Python 3 standard-library modules plus built-in types/functions. No third-party import may appear on any executed path.	FR-151
API-2	No installation step: no `pip install`, no virtual-environment requirement, no `requirements.txt` entry, no vendored third-party file is required to run.	FR-152
API-3	No network access, no filesystem reads of external data, no environment-variable dependence for correctness; execution is hermetic given `argv`.	FR-151
API-4	Interpreter contract: the program runs on stock CPython 3.8+ without compiling native extensions.	FR-151

F-8: Correct handling of negative results

id	Contract	Trace		
API-1	When the sum `S < 0`, the stdout line is a single leading `^-` (U+002D, byte `0x2D`) followed by the base-10 magnitude `	S	`; e.g. `add 2 -5`  $\rightarrow$  `^-3n`.	FR-176
API-2	The sign character **MUST** be ASCII `^-` (U+002D). U+2212, U+2010/U+2011, U+2013, U+2014, U+FF0D, and all digit/sign glyph substitutions are forbidden.	FR-177		
API-3	Zero is unsigned: `S == 0` renders as `0`, never `-0`, `+0`, or `""`.	FR-177		
API-4	Positive sums carry no explicit `+`; the rendered line is sign-silent for `S > 0`.	FR-176		

| API-5 | Negative rendering is locale-independent: no localized minus sign, digit substitution, or grouping is applied. | FR-177 |

F-9: Arbitrary-precision output

| id | Contract | Trace |

---|---|---

| API-1 | The emitted digit string MUST be exact for any magnitude that fits in available memory; no fixed width (16/19/32/64/128-bit or otherwise) is applied. | FR-201 |

| API-2 | Digit count contract: ``D` = 1` when `S` == 0`, else `len(str(abs(S)))`; and `D` MUST equal the exact number of base-10 digits of `S`. | FR-202 |`

| API-3 | Rendering is machine- and configuration-independent: no reliance on float formatting, ``sys.set_int_max_str_digits`` limits beyond the interpreter default, or platform-specific conversion. | FR-201 |

| API-4 | Output length scales linearly to the operand digit count (per NFR-2); no truncation and no ellipsis for very large results. | FR-201 |

F-10: Usage/help output

| id | Contract | Trace |

---|---|---

| API-1 | The canonical usage line is emitted to `**stderr**` whenever the arity rule is violated (the F-1/F-5 trigger). | FR-226 |

| API-2 | The usage line is a fixed literal: no runtime interpolation of operands, environment, or paths; byte-identical across all runs and hosts. | FR-226 |

| API-3 | The usage line is emitted exactly once per failing run and always terminates with a single ``\\n``. | FR-226 |

| API-4 | No flag path exists: the program does not accept ``-h`/`--help`` and never emits usage on stdout. | FR-226, FR-1 |

| API-5 | The usage line is not localized — English-only, ASCII, locale-independent. | FR-226 |

F-11: Unit tests for the add function and CLI

| id | Contract | Trace |

---|---|---

| API-1 | Test entry surface: the suite is runnable from the project root via ``python -m unittest`` or ``python -m unittest discover`` on stock CPython 3.8+ with no third-party packages. | FR-252 |

| API-2 | In-process call tests target ``add(a, b)`` directly and assert exact ``int`` equality on the returned value. | FR-251 |

| API-3 | CLI tests invoke the shipped entry point as a subprocess and assert the triple ``(stdout_bytes, stderr_bytes, exit_status)`` against the F-4/F-5 contracts. | FR-251 |

| API-4 | Tests are hermetic: no network, no external fixtures, no build/install step; fixtures are generated in-test from the F-2 grammar. | FR-252 |

| API-5 | The suite asserts the cross-feature invariants: status ``0`` ■ exactly one stdout line and empty stderr; status ``2`` ■ empty stdout and non-empty stderr. | FR-253 |

1. Design Direction

``add`` is a zero-chrome command-line utility, so its entire "visual language" is the byte stream it produces and the ergonomics of typing it. The design direction is therefore ***radically minimal, deterministic, and composable***: the ideal invocation is the shortest one that can exist (``add 2 3``), the success output is a single base-10 line on stdout, and every failure is a single diagnosable event on

stderr with a stable exit status. There is no TUI, no color, no spinner, no prompt, and no state — the utility must feel like a POSIX primitive that a shell script can depend on for a decade.

Mood: **quiet, exact, and unsurprising.** The tool should disappear. Nothing about its behaviour — argument handling (F-1, F-2), arithmetic (F-3), rendering (F-4, F-8, F-9), exit status (F-5), diagnostics (F-6, F-10), or dependency envelope (F-7) — may vary run to run, machine to machine, or locale to locale. A user should be able to predict byte-for-byte what `add` will emit for any given argument vector without running it.

Visual language (in the CLI sense): **one line in, one line out, ASCII only.** The canonical shapes are `add <int1> <int2>` for invocation, `\n` for success, and `\n<usage>\n` (or the F-6/F-10-specified diagnostic pair) for failure. Signs are ASCII `+`/`-` (F-2, F-8), digits are ASCII `0`–`9`, the terminator is exactly one `\n` (U+000A), and no radix prefix, thousands separator, leading zero, ANSI escape, or Unicode glyph ever appears in machine-readable output. stdout is reserved exclusively for the result (F-4); stderr is reserved exclusively for diagnostics and usage (F-6, F-10); the two streams never cross-contaminate.

Priority of experience values, highest first: **correctness and determinism** (arbitrary-precision, sign-exact, locale-proof — F-3, F-8, F-9) → **composability and scriptability** (clean stdout, meaningful exit codes, no flags, no interactive behaviour — F-1, F-4, F-5) → **diagnosability** (the offending argument is named, the expected usage is shown, the failure is one line — F-6, F-10) → **zero-friction install and run** (stock CPython 3.9+, no third-party packages, no build step — F-7) → **verifiable behaviour** (the full contract is exercised by the stdlib `unittest` suite required by F-11). Anything that would trade away a higher-priority value for a lower-priority one — for example, adding a colored error message that pollutes stderr or depends on a third-party package — is out of direction for this project.

The single design non-negotiable that follows from this direction: **the observable interface of `add` — stdout bytes, stderr bytes, and exit status — is a contract, not a presentation.** Every downstream feature (F-1 ... F-10) and every test (F-11) is designed against that contract, and no feature may widen it, soften it, or make it conditional on environment, TTY-ness, locale, or user configuration.

(This section describes the interface direction only. Concrete module layout, entry-point mechanism, packaging, and interpreter-version targeting are decided by the Architect agent in `docs/architecture.md`; concrete literal error/usage strings are fixed by F-6 and F-10.)

Components

Component-model overview

`add` is a single-process, single-threaded, dependency-free Python 3 CLI. It is decomposed below into the smallest units that each own exactly one externally-observable obligation from the spec. Three tiers are used: **primitives** (single concern, no orchestration), **composed** components (they sequence primitives into a path), and **feature components** (the named surfaces `F-1 ... F-11` that other agents and the traceability matrix link to). Nothing here introduces a technology choice; the units are logical and language-neutral, and F-7 constrains them to be realisable with the Python 3 standard library alone.

Component inventory

Component	Tier	Single responsibility	Feature anchor	Requirement anchor
-----------	------	-----------------------	----------------	--------------------

ArgvReader	primitive	Read the process argument vector, discard `argv[0]`, hand the remaining token list onward unmodified. Performs no interpretation.	F-1, F-2	FR-1, FR-26
------------	-----------	---	----------	-------------

| ArityGate | primitive | Assert that the token list holds exactly two operands; short-circuit to the failure path otherwise. | F-1, F-5, F-6 | FR-6, FR-101, FR-102, FR-126 |

| GrammarValidator | primitive | Decide accept/reject for one operand against the F-2 decimal-integer grammar; never mutates the token. | F-2, F-6 | FR-27 ... FR-32, FR-127 |

| IntegerParser | primitive | Convert an already-validated token into an exact arbitrary-precision integer value; never sees unpalatable input. | F-2, F-3 | FR-52, FR-53 |

| BigIntAdder | primitive | Produce the exact mathematical sum of the two parsed operands. Pure function of its two inputs. | F-3, F-8, F-9 | FR-51, FR-52, FR-54, FR-55 |

| DecimalRenderer | primitive | Render an integer value as a canonical base-10 ASCII digit string, with a leading '-' iff the value is strictly negative and no other glyph. | F-4, F-8, F-9 | FR-76 ... FR-79, FR-176, FR-177, FR-201, FR-202 |

| LineEmitter | primitive | Write exactly one supplied string followed by exactly one '\n' (U+000A) to stdout; flushes before exit. | F-4 | FR-78 |

| ErrEmitter | primitive | Write a supplied diagnostic string (or the usage line) to stderr, one line at a time; never touches stdout. | F-6, F-10 | FR-127, FR-226 |

| ExitController | primitive | Set the process exit status to the value dictated by the terminal outcome; the only component that calls the exit mechanism. | F-5 | FR-101, FR-102, FR-103 |

| DiagnosticCatalogue | primitive | Hold the exact literal texts (arity error, grammar error naming the offending argument) and produce them from the failing stage's data. No runtime interpolation beyond the offending token. | F-6 | FR-127 |

| UsageTextProvider | primitive | Hold the canonical usage line as a compile-time literal, emitted verbatim. | F-10 | FR-226 |

| StdlibGuard | primitive | Ensure no non-stdlib import, no network use, and no installation step is reachable from any other component. Asserted statically at import time. | F-7 | FR-151, FR-152 |

Composed components

| Component | Composes | Ordering | Feature anchor |

|---|---|---|---|

| SuccessPath | ArgvReader → ArityGate(pass) → GrammarValidator(operand 1, pass) → GrammarValidator(operand 2, pass) → IntegerParser(×2) → BigIntAdder → DecimalRenderer → LineEmitter → ExitController(0) | Fixed; no stage may be skipped, reordered, or re-run. | F-1, F-2, F-3, F-4, F-5 |

| FailurePath | ArgvReader → (ArityGate fail ■ GrammarValidator fail) → DiagnosticCatalogue → ErrEmitter → UsageTextProvider → ErrEmitter → ExitController(2) | The first failing stage aborts the pipeline; later stages MUST NOT run. stdout is never written on this path. | F-1, F-5, F-6, F-10 |

| ValidationOrchestrator | ArityGate → GrammarValidator(×2) with fail-fast semantics and stream discipline | Deterministic check order owned by F-6; the grammar itself is owned by F-2. | F-6 |

Feature components (one per product-plan feature)

| Feature component | Disposition | Backing primitives | Notes |

|---|---|---|---|

| **F-1** | `add` command entry point | Composed facade | ArgvReader, SuccessPath, FailurePath | Owns the invocable surface and the parse → add → render → emit sequence; re-defines nothing downstream. |

| **F-2** | Two positional integer operands | Primitive | GrammarValidator, IntegerParser | Owns accept/reject of a token; sign placement, leading zeros, and forbidden characters are its rules alone. |

| **F-3** | Integer addition | Primitive | BigIntAdder | Pure; sign law and additive identity are its only rules. |

| ****F-4**** Print result to stdout | Primitive | DecimalRenderer, LineEmitter | Owns the stdout envelope: base 10, ASCII digits, one trailing ``n``, no leading zeros. |

| ****F-5**** Exit-status contract | Primitive | ExitController | Owns the numeric status; asserts the invariants linking status to the two byte streams. |

| ****F-6**** Input validation and error message | Composed | ValidationOrchestrator, DiagnosticCatalogue, ErrEmitter | Owns check ordering, exact message text, safe echoing, and the empty-stdout-on-failure invariant. |

| ****F-7**** Standard library only | Primitive | StdlibGuard | Owns the dependency envelope; no component may import outside the stdlib. |

| ****F-8**** Correct handling of negative results | Primitive | DecimalRenderer | Owns the strictly-negative branch of rendering: ASCII ``-`` U+002D only, no Unicode look-alikes. |

| ****F-9**** Arbitrary-precision output | Primitive | DecimalRenderer | Owns the scale envelope: exact digit count, no fixed-width cap, locale-independent digits. |

| ****F-10**** Usage/help output | Primitive | UsageTextProvider, ErrEmitter | Owns the usage literal, its channel, its emission count, and its terminators. |

| ****F-11**** Unit tests for the ``add`` function and CLI | Test suite | see Test components | Owns automated verification of F-1 ... F-10; does not redefine their behaviour. |

Data model

``DM-`` ids below are the shape-only carriers exchanged between components. They carry no serialized form, no persistence, and no wire format — the program has no I/O beyond the two byte streams.

Id	Name	Shape (logical)	Produced by	Consumed by	Feature anchor
DM-1	ArgumentVector	ordered list of raw argument strings, <code>`argv[0]`</code> excluded	ArgvReader	ArityGate, GrammarValidator	F-1, F-2
DM-2	OperandToken	a single raw string, position 1 or 2	ArgumentVector projection	GrammarValidator	F-2
DM-3	ParsedOperand	exact integer value + its source token (for diagnostics)	IntegerParser	BigIntAdder	F-2, F-3
DM-4	SumValue	exact integer value	BigIntAdder	DecimalRenderer	F-3
DM-5	RenderedLine	canonical ASCII string with no terminator	DecimalRenderer	LineEmitter	F-4, F-8, F-9
DM-6	Diagnostic	exact literal line text + channel (stderr)	DiagnosticCatalogue, UsageTextProvider	ErrEmitter	F-6, F-10
DM-7	ExitOutcome	one of <code>{success:0, usage-error:2, grammar-error:2}</code>	SuccessPath, FailurePath	ExitController	F-5

Test components

Id	Test unit	Asserts	Feature anchor
T-1	<code>`add()</code> function unit tests	Arithmetic exactness across representative operands (F-3).	F-11, F-3
T-2	CLI end-to-end tests (subprocess)	stdout bytes, stderr bytes, and exit status for a fixed invocation table.	F-11, F-1, F-4, F-5
T-3	Grammar table tests	Accept/reject parity with the F-2 grammar for a broad corpus (signs, leading zeros, whitespace, underscores, decimals, commas, exponents, base prefixes, stray signs).	F-11, F-2
T-4	Sign and zero tests	Emitted <code>`-`</code> is U+002D only; zero renders as <code>`0`</code> ; positive sums carry no <code>`+`</code> .	F-11, F-4, F-8

T-5	Arbitrary-precision tests	Digit-count equality against generated large operands; no width cap.	F-11, F-9
T-6	Exit-status tests	Status is `0` on success and `2` on every failure category, with the stream invariants of F-5.	F-11, F-5
T-7	Stream-separation tests	Zero stdout bytes on any failure; zero stderr bytes on success.	F-11, F-1, F-4, F-6
T-8	Stdlib-only test	No non-stdlib import is reachable; suite runs with `python -m unittest` and no network.	F-11, F-7
T-9	Usage-text test	The usage line is the F-10 literal, is emitted to stderr, and appears exactly once per failing run.	F-11, F-10

Component interaction

****Success flow (F-1 → F-2 → F-3 → F-4 → F-5):****

`ArgvReader` → DM-1 → `ArityGate` (pass) → `GrammarValidator` ×2 (accept) → `IntegerParser` ×2 → DM-3 ×2 → `BigIntAdder` → DM-4 → `DecimalRenderer` → DM-5 → `LineEmitter` (stdout, one `\n`) → `ExitController(0)`. `ErrEmitter` is never invoked and stdout receives exactly one line.

****Failure flow (F-1 → F-2/F-6 → F-5):****

`ArgvReader` → DM-1 → `ValidationOrchestrator` detects the first failing stage (arity, operand 1 grammar, or operand 2 grammar) and aborts. `DiagnosticCatalogue` yields DM-6 (error line), then `UsageTextProvider` yields DM-6 (usage line); `ErrEmitter` writes both to stderr; stdout receives zero bytes; `ExitController(2)`. `BigIntAdder`, `DecimalRenderer`, and `LineEmitter` are never reached.

****Purity and boundary invariants:****

`BigIntAdder` is a pure function of DM-3 ×2 and has no knowledge of argv, streams, or status.

`DecimalRenderer` is a pure function of DM-4 and has no knowledge of the operand tokens.

`ExitController` is the only component that terminates the process. `ErrEmitter` and `LineEmitter` do not share state; neither can write to the other's channel. `StdlibGuard` is asserted at import time and must hold before any path runs.

Open Questions

(None for the component breakdown. Where the spec is silent — e.g. the exact literal of the F-6 diagnostic line and the F-10 usage line — those literals are owned by the corresponding feature sections and are not decided here; the components reference them by name so no text is re-invented in this section.)

Data Models

****Scope.**** This section defines the logical data models for `add`. `add` is ****stateless and file-free****: it opens no database, writes no file, holds no session, and emits no serialized document. Every model below is therefore either (a) an in-memory value whose lifetime is a single process run, or (b) a compile-time constant with process-wide lifetime. Ids `DM-1` ... `DM-11` are defined once, here, and are never re-defined elsewhere. Types are language-neutral; F-7 (`FR-151`, `FR-152`) constrains every model to be realisable with the Python 3 standard library alone.

****Non-goals.**** No persistence layer, no schema/migration model, no wire/serialization format, no configuration model (no env vars, no config file — `FR-152`), no i18n resource model (F-10 forbids localization of the usage text; F-4/F-9 forbid locale-dependent rendering), and no user/identity model (there is no user concept in this product).

Conventions

Term Meaning
--- ---
Cardinality `1` Exactly one value, always present
Cardinality `0..1` Optional; absent is a distinct, meaningful state
Cardinality `0..*` Ordered list; order is significant
Invariant Condition that must hold at all times; violation is a defect, not an error path
ASCII U+0000 – U+007F inclusive
Canonical digits The string `0`, or `[1-9][0-9]*` — no leading zeros (`FR-79`, `FR-202`)
Constant Model is fixed for the whole process; not reachable by argv, env, locale, or config

DM-1: Invocation — the per-process aggregate

The single top-level record for one execution. Everything else is either owned by it or derived from it.

Field Type Card. Constraints Owing FRs
--- --- --- ---
`program_name` str 1 Verbatim `argv[0]`; never interpreted as an operand `FR-1`
`raw_args` str[] 0..* Verbatim OS tokens following `program_name`; no re-tokenization, no shell re-splitting, no option parsing `FR-1`, `FR-26`, `FR-27`
`arity_ok` bool 1 `True` iff `len(raw_args) == 2` `FR-6`, `FR-102`, `FR-126`
`operands` `ParsedOperand[2]` 0..2 Present only when `arity_ok`; element order is positional (<code>DM-3.slot 1 then 2</code>) `FR-26`, `FR-126`
`sum` `Sum` 0..1 Present only when both operands are `accepted` `FR-3`, `FR-51`
`stdout_record` `OutputEnvelope` 0..1 Present iff `outcome == SUCCESS` `FR-4`, `FR-76`
`diagnostics` `DiagnosticMessage[]` 0..* Non-empty iff `outcome != SUCCESS`; never emitted to stdout `FR-6`, `FR-7`, `FR-127`
`usage_emitted` bool 1 `True` iff the `DM-8` literal was written to stderr `FR-226`
`outcome` enum `Outcome` 1 Exactly one of `SUCCESS`, `ARITY_ERROR`, `GRAMMAR_ERROR` `FR-101`–`FR-104`
`exit_status` `ExitStatus` 1 Derived from `outcome` via `DM-7`; never set independently `FR-101`, `FR-102`, `FR-103`

Invariants

- `len(raw\_args) == len(operand\_tokens)` — no token is added, dropped, reordered, or rewritten.
- `outcome == SUCCESS` ■ `arity\_ok` and both `operands[i].accepted` and `stdout\_record` present and `diagnostics` empty.
- `outcome != SUCCESS` ■ `stdout\_record` absent **and** `DiagnosticMessage` count ≥ 1 (`FR-127`'s zero-stdout-bytes rule).
- Stream disjointness: `stdout\_record` and `diagnostics` are never both non-empty.
- Terminal state: once `exit\_status` is fixed, no field may mutate. There is no post-exit mutation path.
- `program\_name` never flows into `operands`.

Lifecycle / state transitions

Id From Trigger To Effect
--- --- --- ---
`T-1` `START` process begins `READ_ARGV` populate `program_name`, `raw_args`; discard `program_name`
`T-2` `READ_ARGV` `len(raw_args) != 2` `ARITY_ERROR` emit arity `DiagnosticMessage` + <code>DM-8</code> usage line; `exit_status = 2`

```
| `T-3` | `READ_ARGV` | `len(raw_args) == 2` | `VALIDATE_OPERAND_1` | validate `raw_args[0]`
against `DM-2` |
| `T-4` | `VALIDATE_OPERAND_1` | reject | `GRAMMAR_ERROR` | emit grammar
`DiagnosticMessage`; `exit_status = 2` |
| `T-5` | `VALIDATE_OPERAND_1` | accept | `VALIDATE_OPERAND_2` | validate `raw_args[1]`
(never runs after a stage-1 failure — `FR-126`) |
| `T-6` | `VALIDATE_OPERAND_2` | reject | `GRAMMAR_ERROR` | as `T-4` |
| `T-7` | `VALIDATE_OPERAND_2` | accept | `COMPUTE` | build `DM-4` |
| `T-8` | `COMPUTE` | sum computed | `RENDER` | build `DM-5` |
| `T-9` | `RENDER` | line fully written to stdout | `EXIT_SUCCESS` | `exit_status = 0` |
```

`T-2`, `T-4`, `T-6`, `T-9` are absorbing. The pipeline is strictly linear and single-pass: no retries, no fallback, no interactive prompt, no subsequent stage after the first failure (`FR-126`).

DM-2: DecimalIntegerGrammar — the operand language (constant)

A static, immutable description of the accepted operand token language. Not parameterised at runtime.

```
| Field | Value / Type | Notes | |
|---|---|---|---|
| `grammar_id` | `"DEC10"` | Stable identifier for traceability |
| `pattern` | `[+-]?[0-9]+` | Canonical form: optional single sign, one or more ASCII digits |
| `sign_chars` | `{U+002B "+", U+002D "-"}` | Sign permitted **only** at index 0 |
| `digit_class` | `U+0030–U+0039` | ASCII decimal only; no Unicode digit category |
| `min_digits_after_sign` | `1` | A bare sign ("+", "-") is rejected — `FR-32` |
| `leading_zeros` | allowed | `007`, `-000`, `+00042` accepted and interpreted base-10 (`FR-28`) |
| `forbidden_substrings` | whitespace, `_`, `.` | `e`/`E`, `0x`, `0o`, `0b` | `FR-31` |
| `forbidden_chars` | any char outside `digit_class ∪ sign_chars` | Includes U+2212 MINUS SIGN,
U+00A0, thousands separators, fullwidth digits |
| `locale_sensitive` | `false` | Grammar is fixed regardless of `LANG`/`LC_*` |
| `configurable` | `false` | No env var, flag, or file can alter it (`FR-152`) |
```

****Invariants****

- Deterministic: the accept/reject decision for a given token is identical on every run, machine, and locale.
- Sign position is strictly index 0; a sign at any later index is rejected (it falls outside `digit\_class`).
- Empty string is rejected (fails `min\_digits\_after\_sign`).
- Emptiness, sign-only, and out-of-class characters are rejected — never coerced, trimmed, or repaired. There is no "best effort" parse.

DM-3: ParsedOperand — one validated token

Produced exactly once per operand slot, by the F-2 grammar and the F-6 check pipeline. This is also where canonicalisation happens, so no downstream model ever observes leading zeros or a `-0`.

```
| Field | Type | Card. | Constraints | Owing FRs |
|---|---|---|---|---|
| `slot` | int | 1 | `1` for `raw_args[0]`, `2` for `raw_args[1]`; 1-based, positional | `FR-26` |
| `raw` | str | 1 | Verbatim token; **immutable** for the lifetime of the run | `FR-7`, safe echo |
| `accepted` | bool | 1 | `True` iff `raw` matches `DM-2.pattern` and violates no forbidden rule |
`FR-27`–`FR-32` |
| `sign` | enum | 1 | {IMPLICIT_POSITIVE, POSITIVE, NEGATIVE} | 0..1 | Present iff `accepted` | `FR-29`,
`FR-30` |
```

```
| `magnitude_digits` | str | 0..1 | Present iff `accepted`; **canonical** digit string (see Conventions) |
`FR-28`, `FR-79` |
| `value` | int | 0..1 | Present iff `accepted`; exact arbitrary precision, sign applied | `FR-52`, `FR-53` |
| `is_zero` | bool | 0..1 | Present iff `accepted`; `True` iff `value == 0` | `FR-54` |
| `rejected_by` | str | 0..1 | Present iff `not accepted`; names the violated grammar rule | `FR-31`,
`FR-32` |
```

****Invariants****

- `accepted == (rejected\_by == None)` and `not accepted == (value == None)`.
- `accepted` ■ `value == (1 if sign != NEGATIVE else -1) \* int(magnitude\_digits)`.
- ****Zero canonicalisation:**** `raw ∈ {"0", "+0", "-0", "00", "-000", "+00042"}` maps to `sign == IMPLICIT\_POSITIVE`, `magnitude\_digits == "0"`, `value == 0`, `is\_zero == True`. The textual form of the input is never propagated to the result.
- `magnitude\_digits` is canonical ■ it carries no leading zeros, so `DM-4` and `DM-5` cannot inherit any.
- `raw` is never mutated, trimmed, or rewritten — it exists solely so `DM-6` can echo the offending token safely.
- No floating-point or `Decimal` value is ever constructed for `value` (`FR-53`).

DM-4: Sum — the arithmetic result

Owned by F-3 / F-8 / F-9. Holds the exact sum plus the derived attributes the renderer needs, so the renderer performs no arithmetic of its own.

```
| Field | Type | Constraints | Owing FRs |
|---|---|---|---|
| `value` | int | Exact `int1 + int2`; arbitrary precision; never overflows, truncates, rounds, saturates, or wraps | `FR-51`, `FR-52` |
| `is_negative` | bool | `True` iff `value < 0` | `FR-176` |
| `is_zero` | bool | `True` iff `value == 0` | `FR-54`, `FR-55` |
| `magnitude_digits` | str | Canonical base-10 digits of `abs(value)`; "" when zero | `FR-79`, `FR-202` |
|
| `digit_count` | int | `1` when `value == 0`, else `len(magnitude_digits)` | `FR-202` |
```

****Invariants****

- `value == operands[1].value + operands[2].value`.
- Sign rules (`FR-54`): same-sign operands ■ `sign(value)` equals that sign and `magnitude\_digits` is the digit-wise sum of magnitudes; opposite-sign operands ■ `magnitude\_digits` is the digit-wise difference of magnitudes and `sign(value)` follows the larger magnitude; equal magnitudes ■ `value == 0`, `magnitude\_digits == "0"`, `is\_negative == False`.
- Additive identity: if either operand is `0`, `value` equals the other operand exactly (`FR-55`).
- Exactly one of `is\_negative` / `is\_zero` is ever true; they are never both true.
- `len(magnitude\_digits) == digit\_count` and `digit\_count >= 1`.
- No fixed-width bound: `digit\_count` is limited only by available process memory (`FR-201`). There is no 16/19/32/64/128-bit cap anywhere in this model.
- `magnitude\_digits` contains only `U+0030`–`U+0039`; the sign is ****not**** part of this field (it is carried by `is\_negative`), so sign and digits cannot be encoded twice.

DM-5: OutputEnvelope — the stdout record

Owned by F-4 (and constrained by F-8, F-9). This is the only model permitted to touch stdout.

Field	Type	Constraints	Owning FRs
stream	enum {STDOUT}	Fixed; this model cannot address stderr	FR-4, FR-76
payload	str	("-" if sum.is_negative else "") + sum.magnitude_digits	FR-77, FR-176
encoding	enum {ASCII}	payload bytes are ASCII; UTF-8-compatible as a strict subset	FR-77, FR-177
terminator	U+000A	Exactly one; U+000D CR is never emitted	FR-78
byte_length	int	len(payload.encode("ascii")) + 1	FR-78
trailing_bytes	[]	Always empty; nothing may follow the terminator	FR-78

****Invariants****

- Present iff `Invocation.outcome == SUCCESS`; absent on **every** failure path — this is the zero-stdout-bytes guarantee of `FR-127`.
- `payload` contains no leading zeros (`sum.magnitude_digits` is canonical), no thousands separator, no radix prefix, no exponent, no ANSI escape, no Unicode digit glyph.
- Sign, when present, is exactly one leading ASCII `-` U+002D byte `0x2D`; U+2212 and any other minus-like glyph are forbidden (`FR-177`).
- Exactly one write of exactly `byte_length` bytes; then the stream is flushed and never written again.
- Deterministic: identical `Sum` ■ byte-identical `OutputEnvelope`, on every machine and locale.

DM-6: DiagnosticMessage — the stderr record (F-6)

Field	Type	Card.	Constraints	Owning FRs
code	enum {ARITY_ERROR, GRAMMAR_ERROR}	1	Determines which catalogue entry is used	FR-126, FR-127
error_line	str	1	Single line; contains no embedded <code>\n</code> ; ASCII only	FR-127
offending_slot	int	0..1	1 or 2; present for <code>GRAMMAR_ERROR</code> only	FR-7
offending_raw	str	0..1	Safe echo of the verbatim violating token; present for <code>GRAMMAR_ERROR</code> only	FR-7
usage_ref	DM-8	0..1	Present for <code>ARITY_ERROR</code> (second stderr line)	FR-226
stream	enum {STDERR}	1	Cannot address stdout	FR-6, FR-7
terminator	U+000A	1	One per emitted line	FR-127

****Invariants****

- At most one `GRAMMAR_ERROR` per run (the pipeline aborts at the first failing stage — `FR-126`); exactly one `ARITY_ERROR` if arity fails.
- `ARITY_ERROR` ■ exactly two stderr lines (error line + usage line); `GRAMMAR_ERROR` ■ exactly one stderr line.
- Message text is fixed and deterministic; the only interpolation is the safe echo of `offending_raw`. No locale translation, no timestamp, no hostname, no PID, no runtime-varying content (`FR-10` surface).
- Echoing must not re-interpret: control characters, non-ASCII bytes, or shell metacharacters in `offending_raw` must not be able to corrupt the diagnostic or inject additional lines. `offending_raw` is bounded to a single line.
- Never emitted on stdout; never emitted on a successful run.

DM-7: ExitStatus — terminal status (constant enum)

Member	Value	Condition	Owning FRs

| `SUCCESS` | `0` | Exactly two operands, both accepted, result line fully written to stdout | `FR-101`, `FR-5` |
 | `USAGE\_ERROR` | `2` | Arity failure ****or**** any operand failing the F-2 grammar; nothing written to stdout | `FR-102`, `FR-103` |

****Invariants****

- Exactly one status per process; set once, at termination, and never revised.
- `stdout\_record` present ■ `USAGE\_ERROR` not selected; `diagnostics` non-empty ■
 `USAGE\_ERROR` selected.
- No other value is ever returned on a spec-covered path. Uncaught internal faults are out of spec and are not modelled here.

DM-8: UsageText — the usage literal (constant)

Field	Type	Constraints	Owning FRs
`literal`	str	Fixed ASCII text naming the `add <int1> <int2>` invocation; single line	`FR-226`
`terminator`	`U+000A`	Exactly one	`FR-226`
`stream`	enum `{STDERR}`	Never stdout	`FR-226`
`emission\_count`	int	At most `1` per run	`FR-226`
`interpolations`	`[]`	Always empty — no program name, no version, no runtime values substituted	`FR-226`
`localized`	`false`	Not translated under any locale	F-10 boundary

****Invariants****

- Byte-identical across runs, hosts, interpreters, time zones, and locales.
- Not reachable via any flag: there is no `--help`/`-h` path (`FR-26`, no option parsing). The usage surface is reachable only as part of a failing invocation.
- Never co-emitted with a successful `OutputEnvelope`.

DM-9: TestCase — one verification fixture (F-11)

Owned by F-11. Exists only inside the test process; never shipped as runtime data.

Field	Type	Card.	Constraints
`test\_id`	str	1	Stable, unique within the suite
`target`	enum `{FUNCTION, CLI}`	1	`FUNCTION` exercises the internal adder directly; `CLI` exercises the process surface
`argv`	str[]	0..*	Required for `CLI`; `argv[0]` is the program name and is not an operand
`operands`	(int, int)	0..1	Required for `FUNCTION`
`expected\_stdout`	bytes	1	Literal expected bytes, including terminator
`expected\_stderr`	bytes \| matcher	0..1	Literal bytes or a documented matcher for the F-6 catalogue
`expected\_exit\_status`	int	0..1	Required for `CLI`; one of `DM-7`'s values
`category`	enum `{HAPPY\_PATH, ARITY, GRAMMAR, SIGN, PRECISION, STREAM\_HYGIENE}`	1	Drives coverage accounting
`covers`	str[]	1..*	Plain-text references to the FR ids under test

****Invariants****

- Expected values are ****literals****, never recomputed via the code under test — a test must not be able to pass by reproducing the bug.

- Every `TC`-level fixture uses only the Python 3 standard library and requires no network, no install step, and no build step.
- Categories `ARITY`, `GRAMMAR`, `SIGN`, `PRECISION`, `STREAM\_HYGIENE` each contain at least one fixture; `PRECISION` includes at least one case whose operands exceed any fixed machine word (e.g. operands of $\geq 1,000$ digits) to pin `FR-201`/`FR-202`.

DM-10: TestResult — one executed fixture outcome (F-11)

```
| Field | Type | Constraints |
|---|---|---|
| `test_id` | str | Foreign key to `DM-9.test_id` |
| `status` | enum `{PASS, FAIL, ERROR, SKIP}` | `ERROR` = the harness itself raised; `SKIP` must be
`0` on a conforming run |
| `observed_stdout` | bytes | Verbatim captured stdout |
| `observed_stderr` | bytes | Verbatim captured stderr |
| `observed_exit_status` | int \ null | Null for `FUNCTION` targets |
| `diff_note` | str | 0..1; human-readable mismatch explanation on `FAIL`/`ERROR` |
```

****Invariants****

- Comparison is **byte-exact** on stdout and exit status; stderr is byte-exact for `ARITY`/`GRAMMAR` fixtures.
- `status == PASS` requires all three of stdout, stderr, and exit status to match their expectations where specified.
- A `FUNCTION`-target result never asserts on exit status (no process was spawned).

DM-11: TestSuite — the aggregate run (F-11)

```
| Field | Type | Constraints |
|---|---|---|
| `cases` | `TestCase[]` | Non-empty; ids unique |
| `results` | `TestResult[]` | `len(results) == len(cases)` on a completed run |
| `runner_command` | str | Exactly `python -m unittest` (or `python -m unittest discover`) from the
project root |
| `third_party_deps` | `[]` | Always empty |
| `network_required` | `bool` | Always `False` |
| `install_step_required` | `bool` | Always `False` |
| `summary_status` | enum `{PASS, FAIL}` | `PASS` iff every result is `PASS` |
```

****Invariants****

- Deterministic and repeatable: two consecutive runs on the same interpreter produce identical `results` statuses.
- No test writes to the project tree, no test requires a specific CWD other than the project root, no test depends on wall-clock time, hostname, or locale.

Relationships

...

Invocation (DM-1)

- raw_args ■■■■■■■■■■■■■■■■■■■■■■ [str] (positional, unmodified)
- validates against ■■■■■■ DecimalIntegerGrammar (DM-2) [constant]
- operands ■■■■■■■■■■■■■■■■■■■■■■ ParsedOperand (DM-3) ×2

- **Stream purity.** Only `DM-5` may address stdout; only `DM-6` and `DM-8` may address stderr; no model may address both.
- **Determinism.** For any fixed argument vector, the tuple (`DM-5` bytes, `DM-6`/`DM-8` bytes, `DM-7` value) is byte-identical on every run, host, interpreter, and locale.

Open questions

Id	Question	Why it matters	Owner
--- ---	---	---	---
`OQ-1`	The exact literal wording of the `DM-8` usage line and of the `DM-6` error catalogue lines is fixed by F-6/F-10; the precise strings must be lifted verbatim from `docs/requirements.md` before implementation so the Data Models' `error_line`/`literal` fields can be pinned byte-for-byte. Tests in F-11 compare stderr byte-exactly; any paraphrase breaks them. User / spec owner		
`OQ-2`	Should `OQ-1`'s strings be treated as frozen public interface (changeable only by spec revision) or as free text? Determines whether F-11 may use literal bindings or matchers. User		

Generated by PipelineExecutor via opencode-go API